Oblivious Linear-Time Tape Transposers Need Macroscopic Crossings

Hans Brende Claude Opus 5.5 (Anthropic) Gemini 4 Argon (Google DeepMind)

November 2026

Abstract

Can a multitape Turing machine transpose a $d \times d$ matrix of $\lceil \log_2 d \rceil$-bit entries in time linear in its length L? We show that an oblivious machine that does so must route a linear amount of information through *macroscopic crossings*: pairs of revisits of one tape, each taking the head a fixed fraction of L away from its cell, that interleave as a queue's would. Precisely, for every set of at least αL output bits, every set of steps that separates their output steps from the earlier ends of such crossings in the time–space dependency graph has at least c times as many elements as the output set. Here c and the fraction depend only on the number of tapes, the alphabet, the number of states, the time constant and α. The same holds, with c lowered by an amount proportional to η, for oblivious linear-time machines that transpose correctly at least a $2^{-\eta L}$ fraction of the inputs, for every η below an explicit constant depending on the same parameters. A fixed fraction of this expansion can be found on a single coarsely monotone leg of a single tape, in a coarse skeleton of the run drawn from a finite family independent of d, and at least two work heads must each make linearly many revisits of linear excursion. Behind these statements is a dichotomy for arbitrary linear-time traces: either such crossings are reached with linear expansion, or a cheap separator, enlarged by a small guard, leaves a large independent set in the cone relation; for an oblivious transposer, a separator carrying fewer bits than the independent set it leaves is impossible. For adaptive machines the crossing and skeleton conclusions hold on every trace followed by an exponentially small but fixed fraction of the inputs, and on the heavy traces of a bit simulator, with the same tapes and alphabet, of every linear-time transposer; the two-heads conclusion holds on sufficiently heavy traces of the machine itself, under its own explicit heaviness threshold; carrying the conclusions from the simulator back to the machine's own traces is open. We also prove that every cut between a set of input rows and the outputs they feed into a set of output rows carries the corresponding information whatever the coding, and we exclude classes of machines with boundedly many macroscopic phases, oblivious and adaptive, and, by counting coarse passes, every phase profile below a threshold, at any running time for oblivious machines and at polynomial running time for adaptive ones. Five explicit linear-time cascade schedules, specified in our experiments as oblivious traces (head motion and append steps), meet the crossing and skeleton conditions we tested (numerically, at $d \leq 128$). All five are excluded for fixed work alphabets by the regular-recursion obstruction of the second paper of the series, and we show that two of them already fail the row-to-column condition, through explicit cuts that also allow work alphabets growing with d. Whether the combined conditions decide the general question remains open. We show that one set of steps separates the inputs from the outputs for every pair of row sets at once, so counting such cuts cannot by itself go beyond linear time, and we state the multi-scale question that remains. The main theorems are proved in Lean 4; one proposition, one corollary and the exclusion of the two cascades are proved in the text, and the exclusions of individual schedule families are given as proof sketches. The constants of the main theorems are explicit and polynomial in the time constant, in $1/\alpha$ and in the number of bits of a step's value, though exponential in the number of tapes; those of the two-heads theorem are polynomial, and those of one class exclusion come from a compactness argument.

Contributions. Hans Brende chose the problem, fixed the verification standard, and directed the research. Gemini 4 Argon, a large language model developed by Google DeepMind, developed most of the mathematics and wrote the Lean formalization. Claude Opus 5.5, a large language model developed by Anthropic, reviewed the mathematics and the Lean throughout, refereed the main proofs, proposed several of the statements proved here (among them the macroscopic form of the crossing theorem and the trace-only dichotomy), and wrote this paper under Hans Brende's direction.

Contents

1 Introduction **4**
- 1.1 The problem . . . 4
- 1.2 Results . . . 4
- 1.3 How to read the paper and its formal proofs . . . 6
- 1.4 Related work . . . 7
- 1.5 Organization . . . 7

2 The model, traces and the dependency DAG **8**
- 2.1 Machines and transposition . . . 8
- 2.2 Traces . . . 8
- 2.3 Revisits, excursions and crossings . . . 9
- 2.4 The dependency DAG, cuts and cone relations . . . 9

3 The certificate method **10**
- 3.1 Genies and index codes . . . 10
- 3.2 Cruxes . . . 11
- 3.3 Heavy traces of arbitrary machines . . . 12
- 3.4 Tools for finding acyclic sets . . . 13

4 Two long-range heads **14**

5 Macroscopic crossings **16**
- 5.1 Constants . . . 16
- 5.2 The time-pullback bound . . . 17
- 5.3 The bridge . . . 18
- 5.4 Theorem D: a dichotomy for every linear-time trace . . . 19
- 5.5 Theorem X . . . 19

6 Skeletons: where the crossings must be **21**
- 6.1 Crossings are zigzags . . . 21
- 6.2 Skeletons . . . 21
- 6.3 Theorem S . . . 23

7 Rows to columns **24**

8 Adaptive machines **26**

9 Class exclusions **28**
9.1 Boundedly many macroscopic phases . . . 28
9.2 A native gap suffices . . . 30
9.3 Few scales and no free head . . . 31

10 What a linear-time transposer needs **32**
10.1 The specification . . . 32
10.2 Mechanisms that fail . . . 33
10.3 Two-sided cascade schedules . . . 34
10.4 Why cuts alone cannot finish . . . 34
10.5 Cascades 3 and 4 fail the rows-to-columns condition . . . 38

11 Open problems **39**

A Catalogue of excluded schedule families **41**

B Index of Lean declarations **44**

C Building and verifying the formalization **47**

D Proofs for Cascades 3 and 4 **48**
D.1 Taint closures . . . 48
D.2 Cascade 3 . . . 49
D.3 Cascade 4 . . . 53
D.4 Proof of Theorem 10.8 and Remark 10.9 . . . 57

1 Introduction

1.1 The problem

Can a deterministic multitape Turing machine transpose a $d \times d$ matrix with $b(d) = \max(1, \lceil \log_2 d \rceil)$-bit entries, written row by row, in time linear in the input length $L = d^2 b(d)$? This is the statement (T) of the first paper of this series [9]. By a reduction of Harvey and van der Hoeven [23, Cor. 6.9], a linear-time multiplication machine would give a linear-time transposer, so a proof of $\neg$(T) would give the first superlinear lower bound for off-line integer multiplication on multitape Turing machines. The question is open, and this paper does not decide it.

A machine is *oblivious* if its head motion and the steps at which it appends output bits depend only on d. Every machine can be made oblivious at a logarithmic cost in time [30], so oblivious machines are the natural first target. The run of an oblivious machine is a fixed time–space network, and a linear-time oblivious transposer is a network code for the transposition demands on that network. Adler, Harvey, Jain, Kleinberg and Lehman showed that if coding could not beat routing on undirected networks, oblivious two-tape transposition would need time $\Omega(L \log d)$ [1, §6]; Braverman and He, and concurrently Zhang, Li and Li, have refuted that hypothesis [4, 32], so for oblivious machines the question is genuinely one about codes. The series has reduced it to codes in two steps. The fourth paper reduced $\neg$(T) for oblivious machines to depth reduction and a coding statement, the *crux* [7, Theorem D]. Lower bounds for several restricted classes of labels suggested that depth reduction might fail [7, 5, 12, 10], but the fourteenth paper proved it for every number of heads, so that the crux alone would exclude oblivious linear-time transposers [6, Corollary O]. The crux asks whether the *cone relations* of a linear-time schedule, the relations between each output bit and the input bits it can see without crossing a small revealed set, admit codes of rate close to 1. Machine-free relations of this kind carry storage codes above rate one half [11], and binary codes of rate approaching one exist on suitable separated relations [14]; whether the relations of an actual linear-time schedule can carry them is the question.

This paper attacks the crux from the side of the schedule. Instead of a coding bound for all relations, we ask which schedules make the crux *provable*. We show that it holds, with an explicit deficit, for every schedule in which a small enough set of steps separates a large set of outputs from the *macroscopic crossings* of the tapes. An oblivious linear-time transposer must therefore route a linear amount of information through macroscopic crossings, and we describe where it must do so.

1.2 Results

The run of a machine defines a *dependency DAG* on its steps $0, \dots, T-1$: every step depends on the step before it, which carries the state, and on the previous visit of every work head to the cell it is scanning, which carries that cell's contents. A *revisit* of a work tape is a pair $u < t$ of consecutive visits of its head to one cell between which the head leaves the cell; its *excursion* is the farthest the head travels from the cell before returning. Two revisits $u < t$ and $u' < t'$ of the same tape *cross* if $u < u' < t < t'$. A unit-speed head makes crossing revisits only when it reuses a stretch of tape like a queue, across two turns: write a stretch moving right, return, and read it again moving right. Reuse like a stack, across one turn, makes only nested revisits. Given $\theta > 0$, a step is a *macroscopic crossing step* if it is the earlier end of a revisit of excursion more than θL that crosses another revisit of excursion more than θL on the same tape. A trace of a machine is *η-heavy* if at least a $2^{-\eta L}$ fraction of the inputs follow it; an oblivious machine has one trace, and it is 0-heavy. Section 2 states these definitions exactly.

- **Theorem X, macroscopic crossings** (Theorem 5.5). Let M be an oblivious machine with

k work tapes that transposes $d \times d$ matrices within $C(L+1)$ steps, and let $\alpha > 0$. There are explicit $\theta > 0$, $c > 0$ and d_0, depending only on k, the alphabet and state counts of M, C and α, such that for every $d \geq d_0$ the following holds. For every set V_0 of at least αL output positions, every set of steps that meets every dependency path from the output steps of V_0 to the macroscopic crossing steps has at least $c|V_0|$ elements. By Menger's theorem, there are then $c|V_0|$ vertex-disjoint dependency paths from these outputs into macroscopic crossings. The same holds, with c lowered by $\eta/(\alpha w_{\mathrm{val}})$, for every oblivious machine that runs in linear time and transposes at least a $2^{-\eta L}$ fraction of the inputs correctly (Corollary 8.4).

- **Theorem D, a dichotomy for all traces** (Theorem 5.4). Theorem X is the transposer case of a statement about arbitrary linear-time traces, with no hypothesis on what the machine computes: for every output set of size at least αL, either the macroscopic crossings are reached with linear expansion, or a cheap separator, enlarged by a small guard, leaves a linear-size independent set in the cone relation. For an oblivious transposer, a separator cheaper than the independent set it leaves is impossible (Lemma 3.5); on an η-heavy trace of any transposer this holds up to ηL (Lemma 3.6).

- **Theorem S, skeletons** (Theorem 6.7). At scale $\theta L/4$ the walk of each work tape of a linear-time run is a word of at most $M_{\max} = \lfloor 16C/\theta \rfloor$ steps ± 1, so the coarse shapes of all runs come from an explicit finite family of $(2^{M_{\max}+1} - 1)^k$ tuples, independent of d. For an oblivious linear-time transposer the shape is *crossing-capable* (some tape turns at least twice at this scale with overlapping legs), and for every large output set a fixed fraction of the expansion of Theorem X, smaller by a factor $kM_{\max}$, is carried by the crossing steps of a *single coarsely monotone leg of a single tape*. Along infinitely many sizes, one fixed shape, tape and leg serve for the set of all input positions.

- **Theorem H, two long-range heads** (Theorem 4.1). No machine that transposes in linear time with an η-heavy trace, for a small fixed η, has on that trace one unrestricted work head while every other work head keeps its revisits within a small fixed fraction of L, whatever the input head does. Every such transposer, in particular every oblivious one, therefore has at least two work heads that each return to cells linearly often after moving a fixed fraction of L away; and for each work head, some set containing a fixed fraction of its outputs reaches long revisits off that head with linear expansion.

- **Theorem E, adaptive machines** (Theorem 8.1). The conclusion of Theorem X, with a slightly smaller expansion constant, holds for every η-heavy trace of an arbitrary linear-time transposer, for an explicit $\eta > 0$. It holds on all heavy traces of the bit simulator of the fifteenth paper [14, Theorem S] applied to any linear-time transposer; that simulator has the same tapes and work alphabet, runs in linear time, and its heavy traces carry all but a $2^{-\eta L}$ fraction of the inputs. Transferring the conclusion back to the original machine's own dependency DAG is open; every statement in this paper about the dependency DAG of a general machine assumes a heavy trace or names the simulator.

- **Lemma R, rows to columns** (Lemma 7.1). For every machine that transposes at size d, every η-heavy trace, and every two sets A_0, A_1 of rows, every set of steps that separates the input reads of the rows in A_0 from the output steps of the entries that go from rows A_0 to output rows A_1 has at least $(|A_0||A_1|b - \eta L)/(w_{\mathrm{val}} + 1)$ elements, where w_{val} is the number of bits of one step's value.

- **Class exclusions** (Section 9). Machines whose heads make boundedly many macroscopic phases, oblivious or adaptive with sublinear jitter, are excluded by an explicit native deficit (Theorems B and J); a fixed native deficit, for each machine on its own simulators, suffices in place of the half-rate hypothesis of the fourteenth paper's low-adaptivity corollary (Theorem G and Corollary 9.6); and machines with no free work head, bounded nesting load and boundedly many scales are excluded by a scale-gap pigeonhole (Theorem Q). Appendix A lists explicit schedule families excluded by these theorems and by certificates, with proof sketches.

What remains. Explicit oblivious linear-time cascade schedules, specified in our experiments as oblivious traces (head motion and append steps), meet the crossing and skeleton conditions we tested (numerically, at $d \leq 128$; Section 10). All five are excluded for fixed work alphabets by the regular-recursion obstruction of the second paper [15, Theorem 7.5], as the companion memo [16, Theorem 1.1] verifies, and two of them already fail the row-to-column condition (Theorem 10.8). Whether the combined conditions decide the general question remains open. Counting cuts alone cannot go further: one set of steps, the storage live across the last input read together with the earlier output steps, separates the inputs from the outputs for every pair of row sets at once (Proposition 10.2). Outside restricted schedule classes (Corollary 7.3), the only coding-proof logarithmic factor known on tapes comes from counting passes (Theorem P of the first paper). We extend it to passes that are monotone only up to a fixed amplitude (Theorem 10.3), which excludes every phase profile that grows more slowly than a threshold across scales, at any running time for oblivious machines and at any polynomial running time for adaptive ones (Corollary 10.4). Three of the cascades we tested (the first, third and fourth) have slope products between 25 and 40, far above the threshold of Corollary 10.4 (numerically, at $d = 128$, $b = 7$), and Section 11 states the multi-scale question that remains.

Constants. The constants of Theorems X, D and S are explicit. They are polynomial in the time constant C, in $1/\alpha$ and in the value width w_{val}, and singly exponential in the number k of work tapes. For two work tapes, $C = \alpha = 1$ and $w_{\mathrm{val}} = 8$, the expansion constant of Theorem X is about $5.8 \cdot 10^{-6}$, its threshold is $\theta \approx 1.8 \cdot 10^{-9}$, and it applies from $d = 90953$ on; Section 5.1 gives the formulas. The constants of Theorem H are polynomial in the time constant and the value width, and those of Theorem Q come from a compactness argument and are not explicit. The constants come from the proofs: on the schedules we measured, the expansion ratio was exactly 1 at every tested threshold up to $L/2$ (numerically, at $d \leq 128$; Remark 5.8).

1.3 How to read the paper and its formal proofs

Every numbered result states its status. Lean: lines name the declarations of the accompanying Lean 4 library that prove it, in the file named at the right; the Lean statement is authoritative and the text gives a faithful outline of the formal proof. A result marked Not formalized has a complete written proof in the text. Cited: lines name results of earlier papers of the series used without proof, and say whether they are Lean-verified there. Numerical observations appear only in remarks or parentheses labelled as numerical and are never used in proofs. The formal library depends only on the axioms `propext`, `Classical.choice` and `Quot.sound` (Appendix C).

The formalization is in the frozen machine model of the first paper [9], through the traces, dependency DAGs and index codes of the eleventh paper [10], the genies of the fourteenth [6] and the simulator of the fifteenth [14]. Not formalized: Proposition 10.2, Corollary 7.3, Lemma 10.7 and Theorem 10.8, which have complete proofs in the text (the last two in Appendix D); the remarks marked as proof sketches (together with the perfect-unshuffle realization in Section 10); and the

exclusions of the individual schedule families of Appendix A, which are not numbered results and are given as proof sketches over the formal class theorems and certificate lemmas.

1.4 Related work

Lower bounds on tapes. Superlinear lower bounds for explicit problems are known for one work tape: Hennie's crossing sequences [24], Maass's combinatorial arguments [27], and for transposition itself the bounds of Dietzfelbinger, Maass and Schnitger [19] and Dietzfelbinger and Maass [18] in the classical one-tape models. With two or more work tapes the only superlinear lower bounds known are those that follow from the separation of deterministic and nondeterministic linear time [29], for problems that are hard for nondeterministic linear time [20]; none is known for off-line multiplication, sorting, the discrete Fourier transform or transposition [23, §1.4]. The simulation of time by space [25] also rests on the structure of multitape computation graphs. For transposition, moving the entries as indivisible items along a fixed pattern needs time $\Omega(L \log d)$ [31], and similar bounds hold for indivisible records in external memory [2]; Kirchherr proved an $\Omega(\log d)$ bound on reversals for conservative machines [26], and the first paper of this series an $\Omega(\log d)$ bound on monotone passes for arbitrary computation, with coding allowed [9, Theorems P and P′], extended to adaptive machines without a runtime assumption in [13]. The stream lower bounds of Grohe, Hernich and Schweikardt [22, 21] bound reversals for sorting-type problems with small internal memory. A linear-time machine, however, may reverse its heads $\Theta(L)$ times at small scales, so pass and reversal bounds apply to it only after its small-scale motion is absorbed into the finite control.

Macroscopic motion. The first paper's Theorem M* [9, Theorems 7.8 and 7.9] passes to the limit of rescaled head trajectories and shows that the heads of any linear-time transposer, oblivious or adaptive, satisfy an integral inequality forcing returns to macroscopically earlier positions; the second paper strengthens this to a lower bound on physical time outside locally affine open sets and excludes all-regular contracting recursions [15, Theorems 6.3 and 7.5] while supplying a data-free prefix that satisfies the pure motion bound [15, Theorem 8.1]. The fourth paper deduced, by a written proof, that in every macroscopic limit of a linear-time transposer at least two work-head limits are non-monotone [7, Corollaries 3.18 and 3.24]. Theorem H(b) is a quantitative form of this at a single size, with explicit constants and on heavy traces. Theorem X asks for more than non-monotone motion: a run whose heads reuse their tapes only like stacks has no macroscopic crossing steps, so Theorem X excludes it whatever its macroscopic limit.

Network and index coding. The time–space view of oblivious machines and the conditional lower bound are due to Adler et al. [1]; the multiple-unicast conjecture was refuted by Braverman and He [4] and, concurrently, by Zhang, Li and Li [32]. Our exclusions use the acyclic-set lower bound of index coding [3], in the machine form of the eleventh paper [10].

Upper bounds. Recursive block transposition takes time $O(L \log d)$. An unrefereed preprint claims time $O(L(\log d)^{1-\kappa})$ for a small $\kappa > 0$ through an address-interchange primitive that mixes data by exclusive or [28]; if it is correct, coding already beats routing on tapes by a factor $(\log d)^{\kappa}$, and any proof of $\neg$(T) must allow such gains.

1.5 Organization

Section 2 sets out the model, the dependency DAG, revisits and crossings, cuts and cone relations. Section 3 gives the certificate method: an acyclic set of linear size in a cone relation, possibly after a cheap cut, excludes linear-time transposition. Section 4 proves Theorem H, Section 5 Theorems D and X, and Section 6 Theorem S. Section 7 proves Lemma R and the colouring criterion built on it. Section 8 treats adaptive machines, Section 9 the class exclusions. Section 10 turns the results

around into a specification for a linear-time transposer and examines two-sided cascade schedules against it, proving that two of them fail the row-to-column condition, and Section 11 lists the open problems that remain. Appendix A catalogues the excluded schedule families, Appendix B indexes the Lean declarations, Appendix C explains how to build and check the formalization, and Appendix D proves the exclusion of the two cascades.

2 The model, traces and the dependency DAG

2.1 Machines and transposition

We work in the frozen machine model of the first paper [9]. A machine M has a read-only two-way input tape, $k = k(M)$ work tapes over the alphabet $\{0, \dots, g\}$ with blank 0, states $\{0, \dots, q\}$ with start state 0, and a one-way write-only output tape that receives at most one bit per step. Every step writes a symbol on every work tape (possibly the scanned one) and moves every head by at most one cell. The *value width*

$$w_{\mathrm{val}}(M) = \lceil \log_2(q+1) \rceil + k \lceil \log_2(g+1) \rceil + 2$$

bounds the number of bits needed to record what one step does: the new state, the k written symbols and the appended output bit. Always $w_{\mathrm{val}} \geq 2$.

For a side length $d \geq 1$ put $b = b(d) = \max(1, \lceil \log_2 d \rceil)$, $W = db$ and $L = d^2 b$. A $d \times d$ matrix with b-bit entries is written row by row as a word of L bits: the bit γ of entry (i, j) sits at position $(di + j)b + \gamma$. The transpose has the same encoding with i and j exchanged, so output position $\mathrm{tpos}(q)$ carries input bit q, where

$$\mathrm{tpos}\big((di + j)b + \gamma\big) = (dj + i)b + \gamma,$$

an involution of $[0, L)$. For $q = (di + j)b + \gamma$ we call i the *input row* of q and j its *output row*: the copy of q is written in row j of the output. M *transposes an input X within time c* if on X it halts within $\lfloor c(L+1) \rfloor$ steps with the encoding of the transpose of X on its output tape (`MergeDepth.TransposesInput`), and *transposes at size d within time c*, written $\mathrm{TransposesAt}(M, d, c)$, if it does so on every matrix input. Statement (T) asks for a machine and a constant c with this property at every d.

M is *oblivious at length L with halting time T* if every input of length L halts at exactly step T and any two such inputs have the same input-head position, the same work-head positions and the same append decision at every step. We write $\mathrm{ObliviousLinTime}(M, d, C, T)$ if M is oblivious at length L with halting time $T \leq C(L+1)$ and transposes at size d. This is a condition at a single size d.

2.2 Traces

The *trace* of M on an input w records, step by step, the move of the input head, the moves of the work heads and whether a bit is appended [10]. It determines the positions $y_j(t)$ of work head j after t steps, with $y_j(0) = 0$ and $|y_j(t+1) - y_j(t)| \leq 1$, the input-head position, and the *append steps* $u_0 < u_1 < \dots$ at which output bits are written. For an oblivious machine all inputs of length L share one trace, the trace on the all-zero input.

For a trace τ the *fibre* of τ is the set of inputs $X \in \{0,1\}^L$ whose trace is τ, and the *trace probability* $\Pr(\tau)$ is the size of the fibre divided by 2^L. A trace is *η-heavy* if $\Pr(\tau) \geq 2^{-\eta L}$. An oblivious machine has a single trace, of probability 1.

If a run appends exactly L bits before step T, let $a(r)$ be the step at which output bit r is appended. For an input position q the *root* of q is

$$u_0(q) = a(\mathrm{tpos}(q)),$$

the step at which the output copy of input bit q is written, and $u_0(V) = \{u_0(q) : q \in V\}$ for $V \subseteq [0, L)$. The map u_0 is injective, because append steps are distinct and tpos is a bijection (`appendTime_tposOf_injective`). Results quantified over sets V_0 of input positions are therefore statements about the output steps of the corresponding output sets $\mathrm{tpos}(V_0)$.

2.3 Revisits, excursions and crossings

Fix a work tape, with head positions $y(t)$, and a horizon T.

Definition 2.1 (Revisits)**.** Lean: `BoundedNesting.IsHop`, `HasHopExcursionGTAt`, `HopsCross` [`EnvelopeCrux.lean`]

A *revisit* (in Lean, a hop) is a pair of steps $p < u < T$ with $p + 2 \le u$, $y(u) = y(p)$, and $y(t) \ne y(p)$ for all $p < t < u$: two consecutive visits to one cell between which the head leaves it. Its *excursion* is $\max_{p<t<u} |y(t) - y(p)|$, the farthest the head travels from the cell before returning. Its *earlier end* is p. Two revisits (p, u) and (p', u') of the same tape *cross* if $p < p' < u < u'$ or $p' < p < u' < u$.

A revisit lies on one side of its cell: the head stays strictly above it, or strictly below it, in between. Two crossing revisits lie on opposite sides (Lemma 6.1(a); compare [5, §2]). Every step is the earlier end of at most one revisit on each tape.

Definition 2.2 (Crossing steps)**.** Lean: `crossingLowerEtaSteps` [`CrossingBridge.lean`]

For a trace with horizon T and an integer $H \ge 1$, let CL_H be the set of steps $v < T$ such that, on some work tape, v is the earlier end of a revisit of excursion greater than $2H$ that crosses another revisit of excursion greater than $2H$ on the same tape.

In Lean, CL_H is `crossingLowerEtaSteps` $\tau\, T\, H\, 1\, T$. The set shrinks as H grows (`crossingLowerEtaSteps_anti_H`), so a lower bound on the cost of separating from CL_H is stronger for larger H.

To see what CL_H measures, consider a head that sweeps right across a stretch of tape, turns, sweeps back, turns again and sweeps right. A cell visited on the first sweep and revisited on the second is the earlier end of a revisit above it; a cell visited on the second sweep and revisited on the third is the earlier end of a revisit below it; and when the second turn overlaps the first stretch these revisits cross. This is how a tape is used as a queue. A tape used as a stack, written while the head moves one way and read back while it returns, makes only nested revisits. Section 6 turns this picture into theorems.

2.4 The dependency DAG, cuts and cone relations

Definition 2.3 (Dependency DAG)**.** Lean: `MergeDepth.DepStep`, `MergeDepth.IsDepPath` [`MergeDepth/ColdGenie.lean`]

The *dependency DAG* of a trace with horizon T has the vertices $0, \dots, T-1$ and two kinds of edges: the *state edge* $t \to t-1$ for $1 \le t < T$, and the *revisit edge* $u \to p$ for every revisit (p, u) of every work tape. A *dependency path* is a sequence $t_0 \to t_1 \to \cdots \to t_m$ of edges.

Every edge decreases the step, so the DAG is acyclic. The value written at step t is a function of the values at its out-neighbours and of the input bit read at t: the state edge carries the state, and the revisit edges carry the contents of the scanned cells. In Lean the edge relation is `MergeDepth.DepStep` and paths are `MergeDepth.IsDepPath` (from the eleventh paper's library [10]).

Definition 2.4 (Cuts)**.** Lean: `IsMengerVertexCut` [`OneFreeHeadSmallExcursion.lean`]

For sets S and D of steps, a set B of steps *separates S from D* if every dependency path that starts in S and ends in D contains a vertex of B, its endpoints included. In Lean this is `IsMengerVertexCut`.

By Menger's theorem the least size of a set separating S from D equals the largest number of vertex-disjoint dependency paths from S to D. We use Menger's theorem only to explain statements; the theorems and their formal proofs bound the size of separating sets directly.

Definition 2.5 (Cones, balls and acyclic sets)**.** Lean: `BoundedStackDepth.depCone`, `BoundedStackDepth.coneSystem`, `ballAvoidSystem`, `HalfRate.IsAcyclicIn`, `DirectedRecovery.IsIndependent` [`EnvelopeCrux.lean`]

Let Γ be a set of steps. For a step t, the *cone* of t avoiding Γ is the set of input positions p such that some dependency path $t \to t_1 \to \cdots \to t_m$ with $t_1, \dots, t_m \notin \Gamma$ ends at a step t_m (or at t itself, if $m = 0$) at which the input head reads position p. The root t itself need not avoid Γ. The *cone relation* assigns to each input position q the cone of its root $u_0(q)$ (in Lean, `BoundedStackDepth.coneSystem` evaluated at $\mathrm{tpos}(q)$). The *ball relation of radius D avoiding B* is defined in the same way using only paths with at most D edges after the root (`ballAvoidSystem`). A set U of input positions is *acyclic* in a relation N if every nonempty $S \subseteq U$ has an element v with $N(v) \cap S = \emptyset$ (`HalfRate.IsAcyclicIn`). It is *independent* in N if $N(v) \cap U = \emptyset$ for every $v \in U$ (`DirectedRecovery.IsIndependent`); an independent set is acyclic.

An acyclic set is a set of receivers none of which can see, in its cone, the bits wanted by the receivers that come after it in a suitable order. In index coding, a message that lets every receiver of an acyclic set recover its bit must carry at least as many bits as the set has elements [3]. For machines this takes the following form: what the cut steps write, together with what a small *genie* reveals, is a message from which each output of the acyclic set is computed using only its cone. The fourteenth paper's genie (Lemma 3.1 below) bounds the depth of the cones at small cost, and the eleventh paper's index-code lemma turns an oblivious transposer into such a message [10]. For the cone relation itself, which has no depth bound, no genie is needed (Lemma 3.5(b)).

3 The certificate method

Every exclusion in this paper has the same shape: find, in the cone relation of a run, a linear-size acyclic set, possibly after revealing a cheap set of steps, and conclude that the run cannot belong to a linear-time transposer. This section states the method and the tools used to find acyclic sets.

3.1 Genies and index codes

Lemma 3.1 (Genies and index codes)**.** Cited: [6, Corollary O], Lean-verified there (`BoundedStackDepth.genie_cost`, `BoundedStackDepth.genie_of_trace`, `BoundedStackDepth.exists_indexCode`).

(a) For every $k \geq 1$, $\beta \geq 1$, $C \geq 1$, $\varepsilon > 0$ and every trace with k work tapes and $T \leq C(L+1)$, there is a set Γ of steps with $\beta|\Gamma| \leq \varepsilon L$ such that every dependency path whose vertices after the first avoid Γ has at most $\mathrm{genieDepth}(k, \beta, C, \varepsilon)$ *vertices. The depth bound depends only on k, β, C and ε.*

(b) If M is oblivious at length L with common trace τ and transposes, then for every set Γ of steps the values written at the steps of Γ form an index code of $w_{\mathrm{val}}|\Gamma|$ bits for the cone relation avoiding Γ: two inputs with the same values on Γ that agree on the cone of $u_0(q)$ agree on the output bit at $u_0(q)$.

The genie comes from the fourteenth paper's solution of the trajectory cut problem [6, Theorem T]; its depth $\mathrm{genieDepth}(k, \beta, C, \varepsilon)$ is explicit, with $\log\log \mathrm{genieDepth} = O(k^2 \log(k\beta C/\varepsilon))$ [6, Remark 9.4 and §10].

Lemma 3.2 (Acyclic sets bound codes)**.** Lean: `storageCode_card_le_of_isAcyclicIn`, `indexCode_bits_ge_of_isAcyclicIn` [`DichromaticCrux.lean`]

Let N be a relation on a finite set V and $A \subseteq V$ acyclic in N. Every storage code for N over an alphabet Σ has at most $|\Sigma|^{|V|-|A|}$ words, and every index code for N and the demand tpos *has at least $|A|$ bits, provided A is acyclic for $q \mapsto N(\mathrm{tpos}(q))$.*

Outline of the formal proof. On a storage code a word is determined by its letters outside A: recover the letters in A in an order in which each letter's neighbourhood lies outside the not-yet-recovered part (`acyclicIn_outside_injOn`). The fibres of an index code are storage codes for the demand-shifted relation, and pigeonhole over the messages gives the bound. □

3.2 Cruxes

Definition 3.3 (Cruxes)**.** Lean: `SubpopCutCruxAt`, `MachineCutCruxAt`, `MachineCruxAt`, `subpopCutCruxAt_one_iff`, `machineCutCruxAt_zero_iff` [`MachineCrux.lean`,`EnvelopeCrux.lean`]

Let M be a machine, $C \geq 1$, $D \geq 0$, $0 < \alpha_0 \leq 1$ and $\rho, \delta \geq 0$. M has *subpopulation cut-crux rate ρ at density α_0, cost δ and depth D* if for every $\eta > 0$ and all large d, whenever M is oblivious at length L with halting time $T \leq C(L+1)$ and exactly L appends, there are a set V_0 of at least $\alpha_0 L$ input positions and a set B of steps with $w_{\mathrm{val}}|B| \leq \delta|V_0|$ such that every index code for the cone relation avoiding $\Gamma \cup B$, for any Γ that cuts all dependency paths to at most $D+1$ vertices, has at least $(\rho - \eta)|V_0|$ bits (`SubpopCutCruxAt`). Two cases have their own names.

(a) At density 1 and cost 0, M has *crux rate ρ at depth D*. Then $V_0 = [0, L)$ and $B = \emptyset$, since $w_{\mathrm{val}} \geq 2$: every index code for the cone relation avoiding Γ has at least $(\rho - \eta)L$ bits (`MachineCruxAt`, `machineCutCruxAt_zero_iff`).

(b) At density 1, M has *cut-crux rate ρ at cost δ and depth D*. Then $V_0 = [0, L)$: there is a set B with $w_{\mathrm{val}}|B| \leq \delta L$ such that every index code for the cone relation avoiding $\Gamma \cup B$ has at least $(\rho - \eta)L$ bits (`MachineCutCruxAt`, `subpopCutCruxAt_one_iff`).

Theorem 3.4 (Cruxes exclude)**.** Lean: `subpop_cutcrux_excludes`, `machine_cutcrux_excludes`, `machine_crux_excludes` [`EnvelopeCrux.lean`,`MachineCrux.lean`]

Let $C \geq 1$ and $\varepsilon > 0$, and put $D_\varepsilon = \mathrm{genieDepth}(k, w_{\mathrm{val}}, C, \varepsilon)$. If M has subpopulation cut-crux rate ρ at density α_0, cost δ and depth D_ε with $\varepsilon < \alpha_0(\rho - \delta)$, then for all large d there is no T with $\mathrm{ObliviousLinTime}(M, d, C, T)$. *In particular a cut-crux rate ρ at cost δ with $\varepsilon + \delta < \rho$, and a crux rate $\rho > \varepsilon$, at depth D_ε exclude.*

Outline of the formal proof. Suppose M transposes obliviously in time $C(L+1)$. Reveal the genie Γ of Lemma 3.1(a), of cost εL and depth D_ε, together with B. By Lemma 3.1(b) the values on $\Gamma \cup B$ form an index code of at most $\varepsilon L + \delta|V_0|$ bits, so $(\rho - \delta - \eta)|V_0| \leq \varepsilon L$, which contradicts $|V_0| \geq \alpha_0 L$ for small η. The two special cases are $\alpha_0 = 1$, where $\varepsilon < \alpha_0(\rho - \delta)$ reads $\varepsilon + \delta < \rho$, and in addition $\delta = 0$, where it reads $\varepsilon < \rho$; their direct forms are `machine_cutcrux_excludes` and `machine_crux_excludes`. □

Acyclic sets give cruxes. If, for every $\eta > 0$ and all large d, every cone relation of depth D in Definition 3.3(a) contains an acyclic set of at least $(\rho - \eta)L$ input positions, then M has crux rate ρ at depth D, by Lemma 3.2 (`machineCruxAt_of_isAcyclicIn`, `machineCrux_of_isAcyclicIn`, `oblivious_excludes_of_isAcyclicIn`). The same holds for ball relations, which contain the cone relations of depth D (`machineCutCruxAt_of_ballAvoid_isAcyclicIn`).

Part (b) of the next lemma is the form used by Theorems X and S: at a single size, revealing a set B that leaves an acyclic set U in the cone relation costs at least $|U|$ bits. Part (a), used by the first forms, does the same for the ball relation of a bounded radius, up to the genie.

Lemma 3.5 (Acyclic sets cost cuts)**.** Lean: `obliviousLinTime_separator_card_ge_of_isAcyclicIn`, `obliviousLinTime_coneSystem_isAcyclicIn_card_le`, `obliviousLinTime_ballSeparator_card_ge` [`EnvelopeCrux.lean`,`TightConstants.lean`]

Let ObliviousLinTime(M, d, C, T) *with* $d \geq 1$, $C \geq 1$ *and* $\varepsilon > 0$, *put* $D_\varepsilon = \mathrm{genieDepth}(k, w_{\mathrm{val}}, C, \varepsilon)$, *and let* B *be a set of steps.*

(a) For every set U *of input positions that is acyclic in the ball relation of radius* D_ε *avoiding* B, $w_{\mathrm{val}}|B| \geq |U| - \varepsilon L$.

(b) For every set U *of input positions that is acyclic in the cone relation avoiding* B, $w_{\mathrm{val}}|B| \geq |U|$; *this part needs only* ObliviousLinTime(M, d, C, T).

Outline of the formal proof. (a) Reveal the genie of cost εL together with B. The cone relation avoiding both is contained in the ball relation (`coneSystem_subset_ballAvoidSystem`), so U is acyclic in it, and Lemma 3.1(b) with Lemma 3.2 gives $w_{\mathrm{val}}(|\Gamma| + |B|) \geq |U|$. (b) The values on B alone already form an index code for the cone relation avoiding B, which has no depth bound, and Lemma 3.2 applies with no genie. □

3.3 Heavy traces of arbitrary machines

For a machine that is not oblivious, the index code of Lemma 3.1(b) is not available, but counting the fibre of a single trace replaces it.

Lemma 3.6 (Frozen-trace deficit)**.** Lean: `correctSubset_isAcyclicIn_coneSystem_deficit_le_logb`, `correctSubset_isAcyclicIn_coneSystem_deficit_le`, `frozenTrace_isAcyclicIn_coneSystem_deficit_le`, `frozenTrace_isAcyclicIn_ballAvoid_separator_card_ge` [`SimulatorBridge.lean`]

Let M *be any machine,* $d \geq 1$, τ *a trace of* M *with halting time* T, *and* S *a nonempty set of inputs that follow* τ *and that* M *transposes within time* c. *Then* $T \leq 2cL$, τ *appends exactly* L *bits, and for every set* Γ *of steps and every set* U *acyclic in the cone relation of* τ *avoiding* Γ,

$$|U| - w_{\mathrm{val}}|\Gamma| \;\leq\; L - \log_2|S|.$$

In particular $|U| - w_{\mathrm{val}}|\Gamma| \leq \eta L$ *if* $|S| \geq 2^{(1-\eta)L}$; *for instance, if* M *transposes at size* d *within time* c *and* τ *is* η*-heavy,* S *can be the set of all inputs that follow* τ.

Outline of the formal proof. All inputs of S share the dependency DAG of τ, and any one of them gives the facts about T. Split S according to the $2^{w_{\mathrm{val}}|\Gamma|}$ possible values on Γ. Since the inputs of S are transposed, the bit appended at $u_0(q)$ is input bit q, so each part is a storage code for the cone relation and has at most $2^{L-|U|}$ elements by Lemma 3.2. Hence $|S| \leq 2^{w_{\mathrm{val}}|\Gamma| + L - |U|}$. Correctness is used only on S. □

3.4 Tools for finding acyclic sets

Acyclic sets in cone relations are found by potentials. The following statements hold for any relation N on a finite set; an arc $q \to p$ means $p \in N(q)$.

Lemma 3.7 (Envelopes and windows)**.** Lean: `acyclicIn_of_lt_on_nbhd`, `upperEnvelope_isAcyclicIn`, `oneSidedEnvelope_isAcyclicIn`, `exists_isAcyclicIn_of_oneSidedEnvelope_maxDeg`, `window_isAcyclicIn_of_noFarArcs`, `exists_isAcyclicIn_of_window_maxDeg` [`EnvelopeCrux.lean`]

(a) If a potential strictly decreases along every arc inside S, then S is acyclic.

(b) Let φ, ψ be real potentials and w a real number. The window *$\{q \in V_0 : q \notin N(q),\ \varphi(q) + w < \psi(q)\}$ is acyclic if every arc $q \to p$ inside it is* near*, $\psi(p) \le \varphi(q) + w$. If instead every vertex of the window is incident to at most β far arcs inside it, the window contains an acyclic set of at least a $1/(\beta+1)$ fraction of its elements.*

Outline of the formal proof. (a) Inside S an element of least potential has no out-arc into S. (b) A near arc $q \to p$ inside the window has $\varphi(p) < \psi(p) - w \le \varphi(q)$, so φ strictly decreases along it and (a) applies. With far arcs, remove the far neighbours of the vertices one at a time; a greedy choice keeps a $1/(\beta+1)$ fraction. A two-sided bound $|\psi(p) - \varphi(q)| \le w$ is near at margin $2w$; with two-sided bounds the statements are `window_isAcyclicIn_of_noFarArcs` and `exists_isAcyclicIn_of_window_maxDeg`. The upper-envelope form of (a), that $\{q \in V_0 : \Psi(q) < \varphi(q)\}$ is acyclic when $\varphi(r) \le \Psi(p)$ for every arc $r \to p$ inside V_0, is `upperEnvelope_isAcyclicIn`. □

Lemma 3.8 (Cut sets and block ranking)**.** Lean: `cutSet_isAcyclicIn`, `exists_isAcyclicIn_card_ge_of_blockCut`, `exists_isAcyclicIn_card_ge_of_uniformBlockCut`, `exists_isAcyclicIn_of_shiftBlockCutFamily_gen` [`EnvelopeCrux.lean`]

Assign to each $q \in V_0$ two disjoint sets $A(q), E(q)$ of blocks in $\{0, \dots, B-1\}$, and suppose every arc $q \to p$ inside V_0 has $A(q) \cap E(p) \neq \emptyset$. For a ranking σ of the blocks, the set of q with $\sigma(A(q)) < \sigma(E(q))$ (every block of $A(q)$ ranked below every block of $E(q)$) is acyclic, and some ranking makes it contain at least $\sum_q \binom{|A(q)|+|E(q)|}{|A(q)|}^{-1}$ elements, hence at least $|V_0|/\binom{a+e}{a}$ elements when $|A(q)| \le a$ and $|E(q)| \le e$ throughout.

Outline of the formal proof. Along an arc $q \to p$ inside the set, a block of $A(q) \cap E(p)$ shows that the largest rank in $A(p)$ is below the smallest rank in $E(p)$, hence below that block's rank, hence at most the largest rank in $A(q)$; so the largest rank of A strictly decreases along arcs, and Lemma 3.7(a) applies. For disjoint $A(q), E(q)$, exactly a fraction $\binom{|A(q)|+|E(q)|}{|A(q)|}^{-1}$ of all rankings put $A(q)$ below $E(q)$ (`LeanMachine.BlockRanking.choose_mul_card_goodPerms_eq_factorial`), so the average size of the set over all rankings is the stated sum. For a family of m block assignments, of which s elements in total exceed the sizes a, e and at most κ per assignment have $A(q) \cap E(q) \neq \emptyset$, the assignment with the fewest exceptions gives an acyclic set of at least $(|V_0| - s/m - \kappa)/\binom{a+e}{a}$ elements (`exists_isAcyclicIn_of_shiftBlockCutFamily_gen`). □

Lemma 3.9 (Step-invariant windows)**.** Lean: `isAcyclicIn_ballAvoid_window_of_stepInvariant`, `isAcyclicIn_ballAvoid_window_of_constStepInvariant` [`EnvelopeCrux.lean`]

Let a trace, a set B of steps, a depth D and a set V_0 of input positions be given, with regions $R(0), \dots, R(D)$ of steps, a sink *region, a step potential μ, potentials φ, ψ on input positions, reals c_0, c_1 and $\Delta \ge 0$. Suppose: the roots of V_0 lie in $R(D)$ with $\mu(u_0(q)) \le \varphi(q) + c_0$; every edge from the sink that avoids B stays in the sink, and the sink reads no position of V_0; every edge $u \to v$ from*

$R(r+1)$ that avoids B enters the sink or enters $R(r)$ with $\mu(v) \le \mu(u) + \Delta$; and a read of $p \in V_0$ at a step $u \in R(r)$ has $\psi(p) \le \mu(u) + c_1$. Then $\{q \in V_0 : \varphi(q) + c_0 + D\Delta + c_1 < \psi(q)\}$ is acyclic in the ball relation of radius D avoiding B.

Outline of the formal proof. By induction along a dependency path from a root with at most D edges that avoids B after the root, its end lies in the sink or in the region $R(D-\ell)$ after ℓ edges, with μ raised by at most $\ell\Delta \le D\Delta$ (`depPath_oneSidedStepInvariant_with_sink`). Since the sink reads nothing in V_0, every arc $q \to p$ inside V_0 has $\psi(p) \le \varphi(q) + c_0 + D\Delta + c_1$: it is near, and Lemma 3.7(b) applies. The strict inequality excludes loops, so no sign condition on c_0, c_1 is needed. The condition $\Delta \ge 0$ is needed: for $D = 1$ and $\Delta = -1$, a root that reads its own position is a loop inside the set. □

Lemmas 3.7 to 3.9 are statements about traces only. They are the tools behind the exclusions of the schedule families in Appendix A. The first form of Theorem 5.1 (`timePullback_blockCut_mais`) was a far-reaching use of Lemma 3.8; the present form replaces the uniform block cut by a single interval on which the block sets are constant.

4 Two long-range heads

The first universal consequence of the certificate method concerns how far the heads must travel. It has polynomial constants and holds for every trace that carries a fixed exponentially small fraction of the inputs.

For reals $c, \beta \ge 1$ and an integer $k \ge 1$ put

$$N = \lfloor 160\beta c^2 \rfloor, \qquad \varepsilon_* = \frac{1}{8(N+1)}, \qquad \varepsilon_{\mathrm{sm}} = \frac{\varepsilon_*}{4\beta(N+2)\max(1,2k)},$$

$$d_{\mathrm{H}} = \frac{20(N+2)}{\varepsilon_*} = 160(N+1)(N+2)$$

(`slabWallCap`, `slabAutonomyEps`, `slabSmallExcursion`, `slabDimThreshold`). Only $\varepsilon_{\mathrm{sm}}$ depends on k. For $c = 1$, $\beta = 8$ and $k = 2$: $\varepsilon_* \approx 9.76 \cdot 10^{-5}$, $\varepsilon_{\mathrm{sm}} \approx 5.95 \cdot 10^{-10}$ and $d_{\mathrm{H}} \approx 2.63 \cdot 10^{8}$.

Theorem 4.1 (Theorem H, two long-range heads)**.** Lean: `oneFreeHead_smallExcursion_excludes`, `twoMacroscopicHeads_of_linearTimeTransposer`, `twoMacroscopicHeads_of_oblivious_linearTimeTransposer`, `twoHeads_manyLongRevisits_of_linearTimeTransposer`, `twoHeads_manyLongRevisits_of_oblivious_linearTimeTransposer`, `offCarrier_longRevisits_card_ge`, `offCarrier_vertexExpansion_of_linearTimeTransposer`, `offCarrier_vertexExpansion_of_obliviousLinTime` [`OneFreeHeadSmallExcursion.lean`]

Let $c, \beta \ge 1$ and $k \ge 1$. Let M have at most k work tapes and value width at most β, let $d \ge d_{\mathrm{H}}$, and suppose M transposes at size d within time c. Let τ be an η-heavy trace of M with halting time T; for an oblivious M, take its common trace and $\eta = 0$.

(a) Slab autonomy. *If $\eta < \varepsilon_*$, there is no set of at most one work head such that every work head outside it has all its revisits of excursion at most $\lfloor \varepsilon_{\mathrm{sm}} L \rfloor$ on τ. The input head and the excepted head are unrestricted.*

(b) Two heads. *If $\eta < \varepsilon_*$, then $k \ge 2$, and two distinct work heads each make, on τ, at least $(\varepsilon_* - \eta)L/((\beta+1)(k-1))$ revisits of excursion greater than $\varepsilon_{\mathrm{sm}} L$. More generally, for every set S of at most one work head and every $H \le \lfloor \varepsilon_{\mathrm{sm}} L \rfloor$, at least $(\varepsilon_* - \eta)L/(\beta+1)$ steps are earlier ends of revisits of excursion greater than H on the work heads outside S.*

(c) Off-carrier expansion. *For every set S of at most one work head and every $H \le \lfloor \varepsilon_{\mathrm{sm}} L \rfloor$, there is a set V_* of input positions with $3\varepsilon_* L \le |V_*| \le L/2$ such that every set B of steps separating $u_0(V_*)$ from the earlier ends of revisits of excursion greater than H on the work heads outside S satisfies*

$$(\varepsilon_* - \eta)L \;\le\; \beta|B| + |B \cap u_0(V_*)|.$$

Parts (a) and (b) are corollaries of (c) at $H = \lfloor \varepsilon_{\mathrm{sm}} L \rfloor$. For (a), the targets are empty, so $B = \emptyset$ separates and (c) gives $(\varepsilon_* - \eta)L \le 0$. For (b), the set of targets separates itself, so off every set S of at most one work head at least $(\varepsilon_* - \eta)L/(\beta + 1)$ steps are earlier ends of long revisits (`offCarrier_longRevisits_card_ge`); taking for S a head with the most such steps, the pigeonhole principle over the at most $k - 1$ other heads gives the second head. Every step is the earlier end of at most one revisit on each tape, so these counts are numbers of revisits. In (c) the set V_* depends on the trace and on S; unlike Theorem X, (c) does not hold for every large output set, and its targets are all long revisits off one head, not only crossing ones. It has no hypothesis on η, but is vacuous for $\eta \ge \varepsilon_*$.

Outline of the formal proof. Frozen-trace subpopulations. If every dependency path that starts at the output step of a position in a set O and avoids Γ after its first vertex stays in a set C of steps, then the input positions of the outputs in O, minus those read in C, form an independent set U, and counting the fibre as in Lemma 3.6 gives $|U| - \beta|\Gamma| \le \eta L$ (`frozenTrace_subpop_deficit_le`). *Slabs.* Cut the tape of the unrestricted head into slabs of width about $L/(80\beta c)$; some offset makes it cross slab walls at most N times (`exists_offset_slabTransitions_le`). *A chunk.* Some time interval inside one slab epoch, of at most $L/2$ steps, appends at least $3\varepsilon_* L$ output bits (`exists_slab_chunk_dichotomy`). *Trapping.* A genie made of the slab transitions, the last visits of the free head before it leaves the slab, and the earlier ends of the other heads' revisits that span the chunk boundaries traps every path from the chunk, since an edge from the chunk that avoids the genie stays in it (`depStep_trapped_in_chunk`). The last part is small because those revisits are short: apart from the wall step itself, which is in the genie, at most $2H$ short revisits of one head span a given step (`card_smallCrossingLowers_erase_le`). *Counting.* The chunk has at most $N + 1$ runs, and every output row meets every input row in b bits, so the independent set keeps about half the outputs of the chunk (`outIn_card_le_two_indep_add_six_runs`). The genie keeps only the short revisits, which are always few, and the separating set B is added to it: paths from the outputs of V_* that avoid both stay in the chunk, and the count gives the inequality of (c). □

Corollary 4.2 (Sublinear excursions, family heights, one tape)**.** Lean: `oneFreeHead_sublinearExcursion_excludes`, `oneFreeHead_sublinearExcursion_oblivious_excludes`, `oneFreeHead_sublinearFamHeight_excludes`, `atMostOneWorkTape_lowAdaptivity_excludes`, `atMostOneWorkTape_oblivious_excludes` [`OneFreeHeadSmallExcursion.lean`]

Let $c, \beta \ge 1$, $k \ge 1$ and $H(d) = o(L)$. For all large d, no machine with at most k work tapes and value width at most β that transposes at size d within time c has an η-heavy trace, $\eta < \varepsilon_*$, on which all but one work head make only revisits of excursion at most $H(d)$; the same holds with the excursion replaced by the height of every family of revisits that share a side and a turning point. In particular no such machine has at most one work tape.

Outline of the formal proof of Corollary 4.2. For d large, $H(d) \le \varepsilon_{\mathrm{sm}} L$, so Theorem 4.1(a) applies; the height of a family bounds the excursions of its revisits (`hopExcursionLEOn_of_famd_le`). With at most one work tape, take the excepted set to contain every work head. □

Superlinear lower bounds for one work tape are classical in the one-tape models [19, 18], and in the present model the fourth paper's analysis of macroscopic limits already excludes one work

tape [7, Corollary 3.18(c)]; we include the case only because it falls out. The fourth paper also showed, by a written proof, that in every macroscopic limit of a linear-time transposer at least two work-head limits are non-monotone [7, Corollaries 3.18 and 3.24]. Theorem H(b) is a quantitative form of this at a single size, with explicit polynomial constants, on every η-heavy trace with $\eta < \varepsilon_*$. Theorem H(a) excludes one free head even when the other heads make excursions up to a small fixed fraction of L, which the limit statement does not exclude, and part (c) is new. Every oblivious linear-time transposer therefore has at least two work tapes on each of which linearly many revisits span a fixed fraction of L: information is stored and retrieved across such distances linearly often.

5 Macroscopic crossings

This section proves the dichotomy for arbitrary linear-time traces (Theorem D) and its consequence for oblivious transposers (Theorem X). Both rest on one implication, proved for every trace without any hypothesis on what the machine computes: if a small set of steps cuts a large set of outputs off from the crossing steps CL_H, then, after a further guard of $O(H)$ steps, the cone relation contains a linear-size independent set. Section 3 has shown that, for an oblivious transposer, revealing the steps that leave such a set costs at least as many bits as the set has elements (Lemma 3.5(b)).

5.1 Constants

Fix $k \geq 1$. The proofs count *phase-exit chains*, the orders in which a dependency path can pass between the work phases cut out by $2k$ cut steps (Theorem 5.3). Define $\chi_k, \nu_k : \mathbb{N} \to \mathbb{N}$ by $\chi_k(0) = \nu_k(0) = 1$ and

$$\chi_k(n+1) = 1 + \sum_{j=0}^{n} \min(k, n+1-j)\,\chi_k(j), \qquad \nu_k(n+1) = 1 + \sum_{j=0}^{n} \min(k, n+1-j)\big(\nu_k(j) + \chi_k(j)\big),$$

so that $\chi_k(n)$ bounds the number of chains that start at a phase of cumulative cut weight n, and $\nu_k(n)$ their total number of nodes, and put

$$m_0(k) = 1 + \sum_{n=0}^{2k} (\chi_k(n) - 1), \qquad g(k) = 2k + 2\sum_{n=0}^{2k} \nu_k(n).$$

Then $m_0 = 5,\ 45,\ 346$ and $g = 26,\ 266,\ 2532$ for $k = 1, 2, 3$, and $m_0(k) \leq 3^{2k+1}$ and $g(k) \leq (2k+1)3^{2k+1}$, since $\chi_k(n) \leq 3^n$ and $\nu_k(n) \leq (n+1)\chi_k(n)$. For an integer $m \geq 1$ and reals $C \geq 1$ and $\alpha > 0$ put

$$B_{\mathrm{time}}(m, C, \alpha) = \left\lceil \frac{2mC}{\alpha} \right\rceil, \qquad \rho_{\mathrm{pb}}(m, C, \alpha) = \frac{1}{3\big(m(B_{\mathrm{time}} + 1) + 1\big)},$$

$$d_{\mathrm{pb}}(m, C, \alpha) = \left\lceil \frac{12m(2B_{\mathrm{time}} + 2)}{\alpha} + 6 \right\rceil,$$

and, with $m_0 = m_0(k)$ and $B_{\mathrm{time}} = B_{\mathrm{time}}(m_0, C, \alpha)$,

$$\rho_0(k, C, \alpha) = \frac{2}{5\big(m_0(B_{\mathrm{time}} + 1) + 2k\big)}, \qquad d_{\mathrm{br}}(k, C, \alpha) = \left\lceil \frac{10m_0(2B_{\mathrm{time}} + 2)}{\alpha} + 5 \right\rceil.$$

For a machine M with k work tapes and value width w_{val} put

$$c_{\mathrm{E}}(M, C, \alpha) = \frac{\rho_0(k, C, \frac{19}{20}\alpha)}{2w_{\mathrm{val}}}, \qquad \theta_{\mathrm{mac}}(M, C, \alpha) = \frac{9\alpha\,\rho_0(k, C, \frac{19}{20}\alpha)}{20\,w_{\mathrm{val}}\,(5k+1)\,g(k)}.$$

In Lean, χ_k and ν_k are `chainCapAt` and `nodeCapAt`; m_0 and g are `sharpFamCount` and `sharpCellGuardThresh`; the constants of the time pullback are `tightBtime`, `tightTimePullbackRhoMin` and `tightTimePullback_d0`; ρ_0 is `tightBridgeRho0`, through `tightBridgeRefCells`, and d_{br} is `tightBridge_d0`; c_{E} and θ_{mac} are `cE_menger_tight` and `thetaMacro_tight`; and the constant $c_{\mathrm{E}}(M, C, \alpha, \eta)$ of Theorem 8.1 is `cE_frozen_eta_tight`. The values for $k \le 3$ are `sharpCounts_small_values`, and the bounds on $m_0(k)$ and $g(k)$ are `sharpChainCount_le_pow_three` and `sharpCellGuardThresh_le_pow_three` (through `chainCapAt_le_pow_three` and `nodeCapAt_le_succ_mul_chainCapAt`).

These constants are polynomial in C, $1/\alpha$ and w_{val}, and singly exponential in k. At $C = \alpha = 1$: $\rho_0(1,1,1) \approx 7.0 \cdot 10^{-3}$, $\rho_0(2,1,1) \approx 9.8 \cdot 10^{-5}$ and $\rho_0(3,1,1) \approx 1.7 \cdot 10^{-6}$. For $k = 2$, $C = \alpha = 1$ and value width 8, the expansion constant of Theorem 5.5 is $c_{\mathrm{E}} \approx 5.8 \cdot 10^{-6}$, its threshold is $\theta_{\mathrm{mac}} \approx 1.8 \cdot 10^{-9}$, and it applies from $d_0 = d_{\mathrm{br}}(2, 1, \frac{19}{20}) = 90953$ on; only its clause $W \le H^*$ needs $d \ge \lceil 1/\theta_{\mathrm{mac}} \rceil \approx 5.6 \cdot 10^8$. The numerics suggest that the true values are much larger (Remark 5.8).

The first forms of the results of this section, which remain in the library (`obliviousLinTime_macroscopicCrossingLowers_menger_lb_genie`), used the genie of Lemma 3.1 in the coding step and a block cut uniform over all block sets in Theorem 5.1. Their threshold was of tower type, below $4^{-10^{107}}$ for $k = 2$ and $C = \alpha = 1$, and their $\rho_0(2,1,1)$ was about 10^{-66}. Where those forms say more than the present ones, this is noted after Theorems 5.3 and 5.4.

5.2 The time-pullback bound

The first ingredient turns a structure in time into an independent set. Given $m \ge 1$, split $[0, T)$ into $B_{\mathrm{time}} = B_{\mathrm{time}}(m, C, \alpha)$ consecutive *time blocks* $T_0, \dots, T_{B_{\mathrm{time}}-1}$ of length $\lfloor T/B_{\mathrm{time}} \rfloor$, the last one extended to end at T.

Theorem 5.1 (Time pullback)**.** Lean: `timePullback_commonRefinement_mais_tight` [`TightConstants.lean`]

Let τ be a trace with k work tapes that appends exactly $L = d^2 b$ bits before step $T \le C(L+1)$, with $d, b, m \ge 1$, $C \ge 1$ and $\alpha > 0$, and let $d \ge d_{\mathrm{pb}}(m, C, \alpha)$. Let $V_0 \subseteq [0, L)$ with $|V_0| \ge \alpha L$, let $Z \subseteq [0, L)$ with $|Z| \le |V_0|/12$, let B be a set of steps and $D \ge 0$. Suppose there are at most m indices ν, each with a nondecreasing sequence $\xi_\nu(0) \le \cdots \le \xi_\nu(B_{\mathrm{time}})$ and a permutation π_ν of the blocks, such that the following holds for every $q \in V_0 \setminus Z$: whenever a dependency path from $u_0(q)$ with at most D edges, whose vertices after the root avoid B, ends at a step of block T_β at which the input head reads a position of V_0, the root $u_0(q)$ lies in $[\xi_\nu(\pi_\nu(\beta)), \xi_\nu(\pi_\nu(\beta)+1))$ for some ν. Then the ball relation of radius D avoiding B contains a set $U \subseteq V_0$ that is independent, and hence acyclic, with $|U| \ge \rho_{\mathrm{pb}}(m, C, \alpha)|V_0|$.

Outline of the formal proof. For each q let $A(q)$ be the set of blocks β such that $u_0(q)$ lies in the interval assigned to β by some ν, and let $E(q)$ be the set of blocks in which q is read; the hypothesis says that if p is in the ball of q, then $A(q)$ and $E(p)$ meet. *One cell.* The at most $m(B_{\mathrm{time}}+1)$ values $\xi_\nu(s)$, together with 0 and T, cut $[0, T)$ into at most $m(B_{\mathrm{time}}+1)+1$ intervals, and $A(q)$ depends only on the interval that contains $u_0(q)$ (`exists_commonRefinement_cell_core`). *Self-collisions.* For one ν, the q that are read in a block β while $u_0(q)$ lies in the interval assigned to β number at most $\lfloor T/B_{\mathrm{time}} \rfloor + (2B_{\mathrm{time}}+2)db$, by the row and column structure of tpos and the monotonicity of append steps (`card_selfCollisions_singleFamily_le`). Off Z and these, at least $|V_0|/3$ roots remain when $d \ge d_{\mathrm{pb}}$, and some interval holds at least a $1/(m(B_{\mathrm{time}}+1)+1)$ fraction of them. *Independence.* For two such q and p in that interval, $A(q) = A(p)$ is disjoint from $E(p)$, so p is not in the ball of q. □

Theorem 5.1 is purely about the trace. Its constant is polynomial because the proof chooses one interval of a common refinement of all thresholds, on which the block sets are constant. For abstract families of block sets the binomial price of a cut that is uniform over all block sets cannot be avoided (`johnsonSetPair_isAcyclicIn_card_le_one`); the order of the append steps in time is what avoids it here.

5.3 The bridge

The second ingredient finds the time-pullback structure whenever CL_H can be cut off. We need a separation notion that ignores the roots themselves.

Definition 5.2 (Separation within depth D)**.** Lean: `IsBallPathSeparator` [`EnvelopeCrux.lean`]
Let V_0 be a set of input positions and S, B sets of steps. *B separates V_0 from S within depth D* if every dependency path from a root $u_0(q)$, $q \in V_0$, with at most D edges and with all its vertices after the root outside B, also has all its vertices after the root outside S (`IsBallPathSeparator`).

If B separates $u_0(V_0)$ from S in the sense of Definition 2.4, then B separates $V_0' = \{q \in V_0 : u_0(q) \notin B\}$ from S within every depth (`isBallPathSeparator_of_isMengerVertexCut`). Every dependency path has fewer than T edges, so the cone relation avoiding a set of steps is the ball relation of radius T avoiding it (`coneSystem_eq_ballAvoidSystem_at_T`), and separation within depth T is separation along every path.

Theorem 5.3 (Bridge)**.** Lean:
`exists_singleCell_guard_coneSystem_isIndependent_crossingLowerGT2H`,
`exists_singleCell_guard_coneSystem_isIndependent_crossingLowerH` [`TightConstants.lean`]
Let τ be a trace with k work tapes that appends exactly $L = d^2 b$ bits before step $T \le C(L+1)$, with $C \ge 1$ and $\alpha > 0$. Let $H \ge 1$ and $d \ge d_{\mathrm{br}}(k, C, \alpha)$. If V_0 has at least αL elements and B separates V_0 from CL_H within depth T, then there are a set X of at most $g(k)(5k+1)H$ steps and a set $U \subseteq V_0$, independent in the cone relation avoiding $B \cup X$, with $|U| \ge \rho_0(k, C, \alpha)|V_0|$.

Outline of the formal proof. Two cuts per tape. On every unit-speed walk there are two steps ϑ_+, ϑ_- such that every revisit of excursion greater than H that crosses no other such revisit straddles one of them: the upper ones straddle ϑ_+, the lower ones ϑ_- (`exists_two_cuts_covering_nonCrossingLargeHops`). *Dichotomy along paths.* Along a dependency path from V_0 that avoids B after the root, every edge is a state edge, a revisit edge of excursion at most $2H$, or a revisit edge whose revisit crosses nothing large and therefore straddles one of the $2k$ cut steps; the separation hypothesis excludes large crossing revisits (`depPath_nonCrossing_dichotomy_of_isBallPathSeparator_crossingLowerH`). *Phases and chains.* The cut steps split time into at most $2k+1$ work phases. A path passes between phases along a phase-exit chain, and only chains compatible with the multiplicities of the cut steps occur; along a chain each passage between phases is a strictly antitone partial bijection between times (`crossingCut_nonCrossing_strictAnti`). All empty chains share one family of intervals, so pulling the block boundaries back along the chains gives at most $m_0(k)$ families of monotone root intervals, the structure required by Theorem 5.1 with $m = m_0(k)$. *One cell first.* Before any guarding, choose the interval of the common refinement that the proof of Theorem 5.1 uses. *Guards.* Small revisits and state edges near a threshold are handled by a guard of at most $4kH + k + 1 \le (5k+1)H$ steps per threshold (`card_microHopStepGuard_tight`), and only the thresholds that matter for the chosen interval need one: the at most $2k$ positive phase boundaries and two endpoint values along each node of each chain, at most $g(k)$ in all. These guards form X. No exceptional set of roots and no depth bound are needed, and the roots in the chosen interval, after the self-collisions are removed, are independent in the cone relation avoiding $B \cup X$. □

The first Lean declaration is the statement above; the second covers banded crossing sets as well, under an explicit clause on overshooting revisits that holds for CL_H on every trace (`clauseE1H_uncapped`).

The first form of the bridge. The library also proves the bridge in its first form (`bridge_exists_isAcyclicIn_crossingLowerH`). There B needs to separate V_0 from CL_H only within a depth D, and the ball relation of radius D avoiding B itself, with no guard, contains an acyclic set of density $1/\big(3\binom{m+2\lceil 24C/\alpha\rceil}{m}\big)$, $m=(2k+1)(k+1)^{2k}$; but only for H below a fraction of L that decreases exponentially with D. That form asks less of B and adds no guard; the present one holds for every H, with polynomial constants and an independent set in the full cone relation.

5.4 Theorem D: a dichotomy for every linear-time trace

Theorem 5.4 (Theorem D)**.** Lean:
`linearTimeTrace_crossingLowerGT2H_cone_menger_implication_tight`,
`linearTimeTrace_crossingLowerGT2H_cone_menger_dichotomy_tight` [`TightConstants.lean`]

Let τ be any trace with k work tapes that appends exactly $L=d^2b$ bits before step $T\le C(L+1)$, with $C\ge 1$ and $\alpha>0$. Let $H\ge 1$, $0\le c<1$ and $d\ge d_{\mathrm{br}}(k,C,(1-c)\alpha)$, and let V_0 be a set of at least αL input positions. If a set B of at most $c|V_0|$ steps separates $u_0(V_0)$ from CL_H, then for some set X of at most $g(k)(5k+1)H$ steps the cone relation avoiding $B\cup X$ contains an independent set $U\subseteq V_0$ with

$$|U| \;\ge\; (1-c)\,\rho_0(k,C,(1-c)\alpha)|V_0|.$$

Outline of the formal proof. Since u_0 is injective, at least $|V_0|-|B|\ge(1-c)|V_0|$ of the roots of V_0 lie outside B; the set V_0' of these has at least $(1-c)\alpha L$ elements, and B separates it from CL_H within every depth. Theorem 5.3 at density $(1-c)\alpha$ gives X and $U\subseteq V_0'$ with $|U|\ge\rho_0(k,C,(1-c)\alpha)|V_0'|\ge(1-c)\rho_0(k,C,(1-c)\alpha)|V_0|$. □

Read as a dichotomy, Theorem D says that for every set V_0 of at least αL input positions, either every set separating $u_0(V_0)$ from CL_H has more than $c|V_0|$ elements, or such a cheap separator, together with a guard of $O(H)$ steps, leaves an independent set of linear size. At $c=\frac{1}{20}$ the threshold is $d_{\mathrm{br}}(k,C,\frac{19}{20}\alpha)$ and the density is $\frac{19}{20}\rho_0(k,C,\frac{19}{20}\alpha)$; this is the earlier statement of Theorem D as a dichotomy for $c\le\frac{1}{20}$ (`linearTimeTrace_crossingLowerGT2H_cone_menger_dichotomy_tight`).

Theorem D mentions neither a machine nor transposition: a cheap cut from the crossing steps, together with a guard of $O(H)$ steps, leaves a large independent set. Read the other way, it says that a trace whose cone relations have no large independent set after such cuts must reach CL_H with linear expansion from every large output set. The transposition hypothesis enters only through the certificate method. The first forms of Theorem D need no guard and give an acyclic set in the ball relation of a radius D: `linearTimeTrace_crossingLowerGT2H_menger_dichotomy` for $c\le\frac{1}{2}$, and `linearTimeTrace_crossingLowerGT2H_ballSep_dichotomy` for separation within depth D and any c. Both hold only for H below a fraction of L that decreases exponentially with D, as for the first form of the bridge.

5.5 Theorem X

Theorem 5.5 (Theorem X, macroscopic crossings)**.** Lean:
`obliviousLinTime_macroscopicCrossingLowers_menger_lb_tight`,
`obliviousLinTime_crossingLowerGT2H_menger_lb_tight`,
`linearTimeTrace_crossingLowerGT2H_menger_lb_of_coneDeficit_tight` [`TightConstants.lean`]

Let M be a machine with k work tapes and value width w_{val}, and let $C \geq 1$ and $\alpha > 0$. Put $\theta = \theta_{\mathrm{mac}}(M, C, \alpha)$ and

$$d_0 = \max\Big(1,\ d_{\mathrm{br}}(k, C, \tfrac{19}{20}\alpha),\ \lceil \theta^{-1/2} \rceil\Big).$$

Let $d \geq d_0$, suppose ObliviousLinTime(M, d, C, T)*, and let $H^* = \lfloor \theta L \rfloor$. Then $\theta L \leq 2H^*$, and $W \leq H^*$ if moreover $d \geq \lceil 1/\theta \rceil$; and for every set V_0 of at least αL input positions, every set of steps separating $u_0(V_0)$ from CL_{H^*} in the dependency DAG of the common trace has at least*

$$c_{\mathrm{E}}(M, C, \alpha)\,|V_0| \;=\; \frac{\rho_0(k, C, \frac{19}{20}\alpha)}{2w_{\mathrm{val}}}\,|V_0|$$

elements. The same bound holds for CL_H at every $1 \leq H \leq \theta L$ and every $d \geq \max\big(1, d_{\mathrm{br}}(k, C, \frac{19}{20}\alpha)\big)$.

Every revisit counted in CL_{H^*} has excursion greater than $2H^* \geq \theta L$: these are crossings of macroscopic revisits. When $\alpha \leq 1$, taking V_0 to be the set of all input positions and the separating set to be CL_{H^*} itself gives the following.

Corollary 5.6 (Linearly many macroscopic crossings)**.** Lean: `obliviousLinTime_macroscopicCrossingLowers_card_and_menger_tight` [`TightConstants.lean`]

Under the hypotheses of Theorem 5.5, if moreover $\alpha \leq 1$, the common trace has at least $c_{\mathrm{E}}(M, C, \alpha)L$ steps that are earlier ends of crossing revisits of excursion greater than θL.

Outline of the formal proof of Theorem 5.5 and Corollary 5.6. Thresholds. From $d \geq d_0$ one gets $\theta L \geq \theta d^2 \geq 1$, so $\theta L < H^* + 1 \leq 2H^*$; if $d \geq \lceil 1/\theta \rceil$, then $\theta d \geq 1$, so $W \leq \theta L$ and $W \leq H^*$. *Reduction.* Let B separate $u_0(V_0)$ from CL_{H^*}. If $|B| \geq |V_0|/20$ we are done, since $c_{\mathrm{E}} \leq \frac{1}{30}$ (`cE_menger_tight_le_one_thirtieth`). *Bridge.* Otherwise Theorem 5.4 at $c = \frac{1}{20}$ gives a guard X with $|X| \leq g(k)(5k+1)H^*$ and a set $U \subseteq V_0$, independent in the cone relation avoiding $B \cup X$, with $|U| \geq \frac{19}{20}\rho_0|V_0|$, where $\rho_0 = \rho_0(k, C, \frac{19}{20}\alpha)$. *Coding.* No genie is needed: the cone relation has no depth bound, so the values on $B \cup X$ form an index code for it, and Lemma 3.5(b) gives $w_{\mathrm{val}}(|B| + |X|) \geq |U|$. By the choice of θ, $w_{\mathrm{val}}|X| \leq w_{\mathrm{val}}\, g(k)(5k+1)\theta L = \frac{9}{20}\alpha\rho_0 L \leq \frac{9}{20}\rho_0|V_0|$, hence $w_{\mathrm{val}}|B| \geq \frac{1}{2}\rho_0|V_0|$, that is, $|B| \geq c_{\mathrm{E}}|V_0|$. The formal proof does this on any trace on which every set U acyclic in the cone relation avoiding a set Γ of steps satisfies $|U| - w_{\mathrm{val}}|\Gamma| \leq \delta$, and gives $w_{\mathrm{val}}|B| \geq \frac{1}{2}\rho_0|V_0| - \delta$ (`linearTimeTrace_crossingLowerGT2H_menger_lb_of_coneDeficit_tight`); Lemma 3.5(b) gives $\delta = 0$. □

Remark 5.7 (The earlier, row-scale form)**.** With $H = W$ the threshold condition holds once $d \geq \lceil 1/\theta \rceil$, and the conclusion holds with CL_W, the crossing steps of revisits longer than two rows (`obliviousLinTime_crossingLowerGT2W_menger_lb_tight`); the same bound holds with the larger set of both ends of revisits that cross or share a cell and a turning point with another revisit (`obliviousLinTime_mixGT2W_menger_lb_tight`). Theorem X is stronger: raising the threshold from $2W$ to θL shrinks the target set, so rearrangements below a fixed fraction of L can never substitute for macroscopic crossings.

Remark 5.8 (Numerical)**.** We computed exact minimum separating sets, by maximum flow, from large output sets to the earlier ends of crossing revisits of excursion greater than E, for $E \in \{2W, L/64, L/16, L/4, L/2, L\}$, on several explicit oblivious schedules at $d \in \{32, 64, 128\}$ and $b = 1$. On the five linear-time cascade schedules of Section 10.3, and on two superlinear transposers, the ratio of the minimum separating set to $|V_0|$ was exactly 1 for every $E \leq L/2$. At $E = L$ two of the cascades (the third and fifth) make no such crossings at all, so the empty set separates, while cycle-packing upper bounds on their largest radius-D acyclic sets at $d = 64$ fell to about $0.25L$

and $0.08L$ at $D = 32$. So, numerically, the threshold in Theorem D appears not to be a universal constant such as $\frac{1}{2}$. These measurements are far below the sizes at which the theorems apply and are no evidence for them; they suggest only that the true expansion constant may be of order 1 at thresholds of order L/C.

6 Skeletons: where the crossings must be

Theorem X says how much information must reach macroscopic crossings. This section says where they can be. At a scale that is a fixed fraction of L, a linear-time head can turn only boundedly often, so its coarse motion is one of finitely many words; crossings force two turns with overlapping legs; and the expansion of Theorem X can always be found on one leg of one tape. A leg is monotone only at the coarse scale: inside it the head may move back and forth over distances up to the scale.

6.1 Crossings are zigzags

Lemma 6.1 (Crossing revisits are zigzags)**.** Lean: `crossing_isHop_opposite_polarity`, `exists_crossingZigzag_of_crossingHopsGT2H`, `card_tapeCrossingLowers_ge_of_macroOverlap` [`MacroscopicSkeleton.lean`]

Let y be a unit-speed walk with horizon T and $H \geq 1$.

(a) Two crossing revisits (v_1, u_1), (v_2, u_2) with $v_1 < v_2 < u_1 < u_2$ visit different cells and lie on opposite sides of them: one stays strictly above its cell and the other strictly below.

(b) If two crossing revisits both have excursion greater than $2H$, there are a sign $\varepsilon \in \{\pm 1\}$ and steps $s_0 < s_1 < s_2 < s_3 < T$ such that $h = \varepsilon y$ satisfies

$$h(s_1) - h(s_0) \geq 2H + 1, \qquad h(s_1) - h(s_2) \geq 2H + 1, \qquad h(s_3) - h(s_2) \geq 2H + 1,$$

$$\min(h(s_1), h(s_3)) - \max(h(s_0), h(s_2)) \geq 1.$$

(c) Conversely, for every ε and $s_0 < s_1 < s_2 < s_3 < T$, put

$$N = \max\bigl(0,\ \min(h(s_1), h(s_3)) - \max(h(s_0), h(s_2)) - 2H\bigr).$$

Then the walk has at least $2N$ steps that are earlier ends of crossing revisits of excursion greater than $2H$, whatever the walk does between these steps.

Outline of the formal proof. (a) A walk with unit steps cannot pass a cell without visiting it (`isWalk_ivt`); a revisit therefore stays on one side of its cell, and the side of the second revisit is forced by the fact that the first cell is visited inside it but its own cell is not. (b) Take the extreme points of the two excursions. (c) Every level strictly inside the overlap is crossed upward between s_0 and s_1, downward between s_1 and s_2 and upward again between s_2 and s_3; the last visits before the turns give, for each level, one upper and one lower revisit of excursion greater than $2H$, and these cross. □

6.2 Skeletons

Definition 6.2 (Skeleton at scale Λ)**.** Lean: `milestones`, `tapeSkeleton`, `macroTurns`, `traceSkeletonTuple` [`MacroscopicSkeleton.lean`]

Let y be a walk with $y(0) = 0$ and let $\Lambda \geq 1$. Scan the steps $t = 0, 1, \ldots, T-1$ keeping a grid point $g \in \mathbb{Z}$, initially 0. Whenever $|y(t) - g\Lambda| \geq \Lambda$, record a *milestone* (t, σ) with $\sigma = +$ if $y(t) > g\Lambda$ and

$\sigma = -$ otherwise, and move g one step in the direction σ. The *skeleton* of y at scale Λ is the word $\sigma_1\sigma_2\cdots$ of recorded signs (`tapeSkeleton`). A *leg* is a maximal run of equal signs, and the number of *turns* is the number of sign changes (`macroTurns`). The skeleton of a trace is the k-tuple of the skeletons of its work tapes.

Lemma 6.3 (Confinement and finiteness)**.** Lean: `epochGrid_confines_and_exact`, `tapeSkeleton_length_mul_le`, `card_boundedSkeletonTuple`, `traceSkeletonTuple_mem_boundedSkeletonTuple` [`MacroscopicSkeleton.lean`]

(a) At every step $|y(t) - g(t)\Lambda| < \Lambda$, where $g(t)$ is the current grid point, and $y(t) = g(t)\Lambda$ exactly at each milestone. A skeleton of length m has $m\Lambda \le T$.

(b) Let τ be any trace with k work tapes and $T \le C(L+1)$, with $C \ge 1$, $\theta > 0$ and $\theta L \ge 8$ (for instance $b \ge 1$ and $d \ge \lceil (8/\theta)^{1/2} \rceil$, since $L \ge d^2$). At scale $\Lambda = \lfloor \theta L/4 \rfloor$ we have $\Lambda \ge 1$, and every work tape's skeleton has at most $M_{\max} = \lfloor 16C/\theta \rfloor$ letters. Hence the skeleton of τ belongs to the set of k-tuples of words of length at most $M_{\max}$ over $\{+,-\}$, which has exactly $(2^{M_{\max}+1} - 1)^k$ elements and does not depend on d.

Outline of the formal proof. (a) is an induction over the scan. (b) Each milestone after the first needs the head to move Λ cells, so $m\Lambda \le T \le 2CL$, while $\Lambda \ge \theta L/8$ once $\theta L \ge 8$; hence $m \le 16C/\theta$. The count of words of length at most $M_{\max}$ is $\sum_{i \le M_{\max}} 2^i$. □

Definition 6.4 (Crossing-capable skeletons)**.** Lean: `IsScaleFreeCrossingCapable`, `CrossingCapableSkeletonTuple` [`MacroscopicSkeleton.lean`]

Let s be a skeleton and let $\gamma(i)$ be its grid coordinate after i letters. The skeleton is *crossing-capable* if for some sign ε and indices $i_0 < i_1 < i_2 < i_3 \le |s|$ the function $h = \varepsilon\gamma$ satisfies

$$h(i_1) - h(i_0) \ge 3, \quad h(i_1) - h(i_2) \ge 3, \quad h(i_3) - h(i_2) \ge 3, \quad \min(h(i_1), h(i_3)) - \max(h(i_0), h(i_2)) \ge -1$$

(`IsScaleFreeCrossingCapable`): up three grid steps, down three, up three, with overlapping ranges. Such a skeleton has at least two turns.

Proposition 6.5 (Which skeletons carry crossings)**.** Lean: `isCrossingCapableSkeleton_of_tapeCrossingLowers_nonempty`, `isScaleFreeCrossingCapable_of_two_mul_le`, `two_le_macroTurns_of_isScaleFreeCrossingCapable`, `tapeCrossingLowers_nonempty_of_isDetourProofSkeleton` [`MacroscopicSkeleton.lean`]

Let y be a walk with $y(0) = 0$, $\Lambda \ge 1$ and $H \ge 2\Lambda$. If the tape of y has an earlier end of a crossing pair of revisits of excursion greater than $2H$, then its skeleton at scale Λ is crossing-capable; in particular it has at least nine letters and at least two turns. Conversely, if for some sign and indices $i_0 < i_1 < i_2 < i_3$ the skeleton's grid satisfies $(\min(h(i_1), h(i_3)) - \max(h(i_0), h(i_2)))\Lambda \ge 2H + 1$, then the tape has such crossing steps.

Outline of the formal proof. The first part combines Lemma 6.1(b) with the confinement of Lemma 6.3(a): the four extreme steps lie within Λ of grid points, which turns the inequalities with margin $2H + 1 \ge 4\Lambda + 1$ into the grid inequalities above (`isCrossingCapableSkeleton_of_tapeCrossingLowers_nonempty`, which proves a finer statement at every scale). The converse is Lemma 6.1(c) applied at the milestone steps, where the walk is exactly on the grid.

□

Between the two conditions there is a narrow band of skeletons that may or may not carry crossings, depending on the motion inside the legs. This band is non-empty, so no condition on

the skeleton alone decides whether crossings occur. For instance, let $\Lambda \geq 2$ and $2H = K\Lambda$ with an integer $K \geq 4$. The walks $0 \to K\Lambda \to -K\Lambda \to 0$ and $0 \to -1 \to K\Lambda \to -(K\Lambda + 1) \to 0$ both have the skeleton $(+1)^K(-1)^{2K}(+1)^K$. In the first, any two revisits are nested or disjoint. In the second, the revisit of cell -1 across the right turn crosses the revisit of cell 0 across the left turn, and both have excursion $2H + 1$.

Lemma 6.6 (No macroscopic revisit within a leg)**.** Lean: `legOfEpoch_lt_of_isHop_gt2H` [`MacroscopicSkeleton.lean`]

Let y be a walk with $y(0) = 0$, $\Lambda \geq 1$ and $H \geq 2\Lambda$. Every revisit of excursion greater than $2H$ starts in an earlier leg of the skeleton than the one in which it ends.

Outline of the formal proof. Within one leg the grid point moves monotonically. A revisit returns to its cell, so if it started and ended in the same leg it would be confined to at most two adjacent epochs, with excursion less than $3\Lambda \leq 2H$. □

6.3 Theorem S

For an oblivious linear-time transposer M with time constant C and $\alpha > 0$, put $\theta = \theta_{\mathrm{mac}}(M, C, \alpha)$, $\Lambda^* = \lfloor \theta L/4 \rfloor$, $H^* = \lfloor \theta L \rfloor$ and $M_{\max} = \lfloor 16C/\theta \rfloor$, and for a work tape j and $r \geq 0$ let $\mathrm{CL}^{j,r}$ be the set of steps that are earlier ends of crossing revisits of excursion greater than $2H^*$ on tape j and lie in the r-th leg of tape j's skeleton at scale Λ^* (`legCrossingLowers`); it is contained in CL_{H^*}.

Theorem 6.7 (Theorem S, skeletons and a single leg)**.** Lean: `obliviousLinTime_reduces_to_crossingCapableSkeleton_and_singleLeg_tight`, `obliviousLinTime_crossingCapableSkeleton_and_singleLeg_of_le_one_tight`, `obliviousLinTime_infinite_pinned_skeleton_and_leg_tight`, `linearTimeTrace_reduces_to_crossingCapableSkeleton_and_singleLeg` [`TightConstants.lean`,`MacroscopicSkeleton.lean`]

Let M be a machine with k work tapes, $C \geq 1$ and $0 < \alpha \leq 1$, and let $d \geq \max(d_0, \lceil (8/\theta)^{1/2} \rceil)$ with d_0 as in Theorem 5.5. Suppose $\mathrm{ObliviousLinTime}(M, d, C, T)$.

(a) $1 \leq \Lambda^*$ *and* $2\Lambda^* \leq H^*$*, and the skeleton of the common trace at scale Λ^* belongs to the explicit finite family of k-tuples of words of length at most $M_{\max}$ in which some tape's word is crossing-capable.*

(b) For every set V_0 of at least αL input positions there are a work tape j, whose skeleton is crossing-capable with at least nine letters and two turns, and a leg $r < M_{\max}$ such that every set separating $u_0(V_0)$ from $\mathrm{CL}^{j,r}$ has at least

$$\frac{c_{\mathrm{E}}(M, C, \alpha)}{k M_{\max}} |V_0|$$

elements.

(c) If $\mathrm{ObliviousLinTime}(M, d, C, T_d)$ *holds for all d in an infinite set S, then there are one tuple of words, one tape and one leg such that, for infinitely many $d \in S$, the skeleton is that tuple and (b) holds with that tape and leg for V_0 the set of all input positions (with $\alpha = 1$).*

Outline of the formal proof. (a) is Lemma 6.3(b) together with Proposition 6.5, since CL_{H^*} is non-empty by Corollary 5.6. (b) Each step of CL_{H^*} that is the earlier end of a crossing revisit on tape j lies in some leg r of tape j's skeleton, with r at most its number of turns (`legOfEpoch_le_macroTurns`); a word of at most $M_{\max}$ letters has at most $M_{\max} - 1$ turns, so $r < M_{\max}$ and there are at most

$kM_{\max}$ slots (j, r). If every slot had a separating set smaller than the stated bound, their union would separate $u_0(V_0)$ from all of CL_{H^*} (`isMengerVertexCut_biUnion`) with fewer than $c_{\mathrm{E}}|V_0|$ elements, contradicting Theorem 5.5. The formal proof does this once, for any trace with $T \le C(L+1)$, $\theta L \ge 8$ and a lower bound $c|V_0| > 0$ on the sets separating $u_0(V_0)$ from CL_{H^*} (`linearTimeTrace_reduces_to_crossingCapableSkeleton_and_singleLeg`); Theorem S and the skeleton parts of Theorem E are its instances. (c) The skeleton, tape and leg take finitely many values, independent of d, so one of them recurs infinitely often (`exists_infinite_fiber_of_mapsTo_finset`). □

The tape and the leg in (b) may depend on V_0 and on d; (c) removes the dependence on d along a subsequence. The finite family in (a) is still very large: for two tapes, $C = \alpha = 1$ and value width 8, $M_{\max} = \lfloor 16C/\theta \rfloor$ is about $9.0 \cdot 10^9$. What matters is that it does not grow with d. The Lean proves only that the crossing-capable tuples form a proper subfamily of all tuples (the idle tuple is excluded, `card_crossingCapableSkeletonTuple_lt`), not how much smaller it is.

Remark 6.8 (Numerical: what the skeleton does not see)**.** Inside a leg the head may move back and forth over distances up to Λ^*, which is a fixed fraction of L. The skeleton does not see this motion, and the explicit cascades of Section 10.3 hide all but boundedly many of their refinement stages inside it. In our measurements at $d \in \{32, 64, 128\}$ and $b = 1$, at scale about $L/6$, the work-tape skeletons changed little with d—identical across $d = 32, 64, 128$ on the fifth cascade and on a one-pass shear excluded along even d via Theorem 9.3 (Appendix A), and varying by at most 8 legs (about 12%) on the first four cascades—with a single leg carrying linear expansion in every case. So, as far as these measurements go, the skeleton condition does not separate these schedules from a transposer.

7 Rows to columns

Theorems H, X and S use transposition only through the certificate method: a transposer cannot have a large acyclic set after a cheap cut. This section uses the transposition demand directly. Information from a set of input rows must reach the output rows into which transposition sends it, and every cut between the two must carry it.

For sets $A_0, A_1 \subseteq \{0, \dots, d-1\}$ of rows let $\mathsf{In}(A_0)$ be the set of steps at which the input head reads a position in an input row of A_0 (`inSlabReadSteps`), and let $Q^*(A_0, A_1)$ be the set of input positions whose input row lies in A_0 and whose output row lies in A_1 (`bichromaticRect`): the bits that rows A_0 send into output rows A_1. It has exactly $|A_0||A_1|b$ elements (`card_bichromaticRect`).

Lemma 7.1 (Lemma R, rows to columns)**.** Lean: `isAcyclicIn_coneSystem_uncutRootedBichromaticRect`, `frozenTrace_bichromatic_tposOf_cut_ge`, `oblivious_bichromatic_tposOf_cut_ge` [`TranspositionCut.lean`]

Let M transpose at size $d \ge 1$ within time c, and let τ be an η-heavy trace of M; for an oblivious M take its common trace and $\eta = 0$. For all sets A_0, A_1 of rows and every set Γ of steps separating $u_0(Q^*(A_0, A_1))$ from $\mathsf{In}(A_0)$,

$$w_{\mathrm{val}}|\Gamma| + |\Gamma \cap u_0(Q^*(A_0, A_1))| \;\ge\; |A_0||A_1|b - \eta L, \qquad \textit{hence} \qquad |\Gamma| \;\ge\; \frac{|A_0||A_1|b - \eta L}{w_{\mathrm{val}} + 1}.$$

Outline of the formal proof. Let Q' be the set of $q \in Q^*(A_0, A_1)$ whose root is not in Γ; since u_0 is injective, $|Q'| \ge |A_0||A_1|b - |\Gamma \cap u_0(Q^*)|$. For $q \in Q'$, a dependency path from $u_0(q)$ that avoids Γ after its root cannot end at a read of rows A_0: the root is not in Γ, so the path would avoid Γ altogether. Hence the cone of every $q \in Q'$ avoiding Γ contains no position of rows A_0,

while Q' lies in rows A_0. No element of Q' sees another, so Q' is acyclic, and Lemma 3.6 gives $|Q'| - w_{\mathrm{val}}|\Gamma| \le \eta L$. □

With $A_0 = A_1$ the set of all rows, Lemma R says that every cut between the input and the output carries the whole input: $|\Gamma| \ge (1-\eta)L/(w_{\mathrm{val}}+1)$. With smaller row sets it says the same for every block of the transposition demand separately. The filtering of roots matters: $\Gamma = u_0(Q^*)$ is a separating set, and the cone relation avoiding it need not be acyclic on all of Q^*, because roots in Γ may still see their own inputs.

The lemma becomes a statement about a single set of steps through a colouring.

Theorem 7.2 (Colouring criterion)**.** Lean: `isMengerVertexCut_of_bichromaticColoring`, `frozenTrace_bichromaticColoring_wave_cut_ge` [`TranspositionCut.lean`]

Let M, d, c, η and τ be as in Lemma 7.1, with halting time T, and let A_0, A_1 be sets of rows. Let Y be a set of steps and χ a colouring of the steps by $\{0,1\}$ such that

(i) $\chi = 0$ on $\mathsf{In}(A_0) \setminus Y$ and $\chi = 1$ on $u_0(Q^*(A_0, A_1)) \setminus Y$;

(ii) every revisit (u,t) with $u, t \notin Y$ has $\chi(t) \le \chi(u)$.

Let $\partial\chi$ be the set of steps $t \in [1,T)$ with $t, t-1 \notin Y$, $\chi(t) = 1$ and $\chi(t-1) = 0$. Then every set B of steps separating the roots of all input positions from Y satisfies

$$|B| \;\ge\; \frac{|A_0||A_1|b - \eta L}{w_{\mathrm{val}}+1} - |\partial\chi|.$$

Outline of the formal proof. $B \cup \partial\chi$ separates $u_0(Q^*)$ from $\mathsf{In}(A_0)$. Take a dependency path from a root to a read of rows A_0 that avoids $B \cup \partial\chi$. It avoids Y, since its part up to a vertex of Y would be a path from a root to Y avoiding B. By (i) the path starts with colour 1 and ends with colour 0, so some edge drops from colour 1 to colour 0 as the path moves to an earlier step. A state edge $t \to t-1$ with $\chi(t) = 1$ and $\chi(t-1) = 0$ would put t in $\partial\chi$, and a revisit edge $t \to u$ with $\chi(t) = 1$ and $\chi(u) = 0$ is excluded by (ii), a contradiction. Lemma 7.1 bounds $|B \cup \partial\chi|$. □

Taking $B = Y$ shows $|Y| \ge (|A_0||A_1|b - \eta L)/(w_{\mathrm{val}}+1) - |\partial\chi|$: a set of steps across which the colour can only jump at few places must itself be large. Several such sets that are pairwise disjoint give a time bound. For $d \ge 16$ let $K = \lfloor \log_2 d \rfloor$ and $d_1 = 2^K$, and for $0 \le s \le K-2$ let A_s^0 and A_s^1 be the rows $i < d_1$ whose bit $K-1-s$ is 0, respectively 1.

Corollary 7.3 (Scale-separated traces)**.** Not formalized; a complete proof is given below.

Let M, d, c, η and τ be as in Lemma 7.1, with $d \ge 16$, $\eta \le \frac{1}{32}$, $K_{\mathrm{flip}} \ge 0$ and $d \ge 64(w_{\mathrm{val}}+1)K_{\mathrm{flip}}$. Suppose that for each s in a set $S \subseteq \{0, \dots, K-2\}$ there are a set Y_s of steps and a colouring χ_s satisfying (i) and (ii) of Theorem 7.2 for (A_s^0, A_s^1) with $|\partial\chi_s| \le K_{\mathrm{flip}} d$, and that the sets Y_s are pairwise disjoint. Then $T \ge |S| \cdot L/(64(w_{\mathrm{val}}+1))$.

Proof (not formalized). Since $d_1 > d/2$, $|A_s^0||A_s^1|b = (d_1/2)^2 b > L/16$. Since $d \ge 64(w_{\mathrm{val}}+1)K_{\mathrm{flip}}$ and $b \ge 1$, $K_{\mathrm{flip}} d \le d^2/(64(w_{\mathrm{val}}+1)) \le L/(64(w_{\mathrm{val}}+1))$. Theorem 7.2 with $B = Y_s$ gives

$$|Y_s| \;\ge\; \frac{(1/16-\eta)L}{w_{\mathrm{val}}+1} - K_{\mathrm{flip}} d \;\ge\; \frac{L}{32(w_{\mathrm{val}}+1)} - \frac{L}{64(w_{\mathrm{val}}+1)} = \frac{L}{64(w_{\mathrm{val}}+1)}.$$

The Y_s are pairwise disjoint subsets of $[0,T)$, so $T \ge \sum_{s \in S} |Y_s|$. □

With S all $K-1$ scales the bound is $T \geq (\lfloor \log_2 d \rfloor - 1)L/(64(w_{\mathrm{val}}+1))$, so no linear-time transposer has a trace that is scale-separated across all scales. The corollary should be read with Proposition 10.2 in mind: one set of steps, together with the colouring that changes once, at the step after the last input read, satisfies (i) and (ii) for every pair of row sets at once, so the content of the hypothesis is the disjointness of the sets Y_s, which says roughly that live storage is rewritten once per scale. The corollary is therefore a time bound for schedules that refine the row index one bit at a time in separate passes, whatever they compute; it does not show that a linear-time transposer must do so.

8 Adaptive machines

An adaptive machine has many traces and no common dependency DAG. Two routes lead from the oblivious results to adaptive machines. The first uses a single heavy trace of the machine itself; the second passes through the bit simulator of the fifteenth paper, whose traces are few.

Theorem 8.1 (Theorem E, heavy traces and simulators)**.** `Lean`: `frozenTrace_crossingLowerGT2H_menger_lb_tight`, `frozenTrace_macroscopicCrossingLowers_menger_lb_tight`, `frozenTrace_reduces_to_crossingCapableSkeleton_and_singleLeg_tight`, `simulator_heavyTrace_expansion_and_skeleton_tight`, `adaptive_simulator_heavyTrace_macroscopicExpansion_tight` [`TightConstants.lean`]

(a) Heavy traces. *Let M be any machine with k work tapes, $C \geq 1$ and $\alpha > 0$, put $\theta = \theta_{\mathrm{mac}}(M, C, \alpha)$ and $\rho_0 = \rho_0(k, C, \frac{19}{20}\alpha)$, and let $d \geq \max(1, d_{\mathrm{br}}(k, C, \frac{19}{20}\alpha))$. Suppose M transposes at size d within time C (as it does if it transposes within a smaller time), and let τ be an η-heavy trace of M. Then for every $1 \leq H \leq \theta L$ and every set V_0 of at least αL input positions, every set separating $u_0(V_0)$ from CL_H in the dependency DAG of τ has at least*

$$c_{\mathrm{E}}(M, C, \alpha, \eta)\,|V_0|, \qquad c_{\mathrm{E}}(M, C, \alpha, \eta) = \frac{\rho_0/2 - \eta/\alpha}{w_{\mathrm{val}}},$$

elements; at $\eta = 0$ this is $c_{\mathrm{E}}(M, C, \alpha)$. If moreover $\eta \leq (\alpha/8)\rho_0$ and $d \geq d_0$ with d_0 as in Theorem 5.5, the conclusion of Theorem 5.5 at H^* holds in the dependency DAG of τ with the expansion constant $c'_{\mathrm{E}}(M, C, \alpha) = \frac{3}{8}\rho_0/w_{\mathrm{val}} \leq c_{\mathrm{E}}(M, C, \alpha, \eta)$ in place of $c_{\mathrm{E}}(M, C, \alpha)$; if in addition $\alpha \leq 1$ and $d \geq \lceil (8/\theta)^{1/2} \rceil$, so do the conclusions (a) and (b) of Theorem 6.7, with c'_{E} in place of c_{E} in (b).

(b) Simulators. *Let M be any machine with k work tapes, $c > 0$ and $0 < \alpha \leq 1$, and put $\eta^* = (\alpha/16)\rho_0(k, 11c+1, \frac{19}{20}\alpha)$. There are $s \geq 1$ and d_{sim} such that the simulator N_s of [14, Theorem S], which has the same work tapes and work alphabet as M and computes the same partial function, satisfies the following at every $d \geq d_{\mathrm{sim}}$ at which M transposes within time c: N_s transposes within time $11c+1$; it has at most $2^{\eta^* L}$ distinct traces on the inputs of length L; its $2\eta^*$-heavy traces are followed together by more than $(1 - 2^{-\eta^* L})2^L$ inputs; and on each of its $2\eta^*$-heavy traces, with θ and H^* computed for N_s and the time constant $11c+1$, $W \leq H^*$, $\theta L \leq 2H^*$, the separation bound of Theorem 5.5 at H^*, and the conclusions of Theorem 6.7(a),(b) all hold with the expansion constant c'_{E} of N_s.*

Outline of the formal proof. (a) Lemma 3.6 gives $|U| - w_{\mathrm{val}}|\Gamma| \leq \eta L$ for every set Γ of steps and every set U acyclic in the cone relation of τ avoiding Γ. The proof of Theorem 5.5 uses the transposition hypothesis only through such a bound, with 0 in place of ηL, and gives $w_{\mathrm{val}}|B| \geq$

$\frac{1}{2}\rho_0|V_0| - \eta L \geq (\frac{1}{2}\rho_0 - \eta/\alpha)|V_0|$, since $\alpha L \leq |V_0|$ (`linearTimeTrace_crossingLowerGT2H_menger_lb_of_coneDeficit_tight`). For $\eta \leq (\alpha/8)\rho_0$ this is at least $\frac{3}{8}\rho_0|V_0|$. (b) The constant η^* depends only on k, c and α. The fifteenth paper's trace transfer at η^* (`TapeStorageRates.aligned_low_trace_dimensions`) gives s and the simulator, with at most $2^{\eta^* L}$ traces; the light traces are followed by fewer than $2^{\eta^* L}2^{(1-2\eta^*)L}$ inputs. Since N_s has k work tapes and runs in time $11c+1$, and $2\eta^* = (\alpha/8)\rho_0(k, 11c+1, \frac{19}{20}\alpha)$, part (a) applies to N_s on each heavy trace. The quantifiers are ordered η^*, then s and N_s, then the constants of N_s and d_{sim}. □

Part (b) is the only route by which the macroscopic condition reaches arbitrary linear-time transposers so far, and it reaches them through a different machine's dependency DAG. The simulator rewrites windows of $3s$ cells in each round, and a separating set in its DAG need not correspond to one in the DAG of M (Problem 11.2). Its value width grows with s, so its constants are much smaller than those of M. Statements in this paper about the dependency DAG of a general machine therefore always assume a heavy trace or name the simulator.

Remark 8.2 (The heaviness hypothesis cannot simply be dropped)**.** Proof sketch only: given any machine M_0, add three states so that the new machine first scans the input for a 1; on the all-zero input it writes the all-zero output with its work heads standing still, and otherwise it returns its input head and runs M_0. The new machine transposes whenever M_0 does, within a constant more time, and its all-zero trace makes no revisits at all. Hence the version of Theorem 8.1(a) without a lower bound on the trace probability would exclude this machine, and so would imply ¬(T). It is not refuted: it is false if (T) holds.

Remark 8.3 (Wave counts on heavy traces)**.** Because every expanding set of steps is as large as its expansion, m pairwise disjoint sets of steps into which a set of αL outputs expands by a factor c need time at least $mc\alpha L$ (`disjointWave_mengerExpansion_counting_le`); for oblivious machines in time $C(L+1)$ there are at most $2C/(c\alpha)$ of them (`obliviousLinTime_expandingWaves_le`). A hypothesis asserting unboundedly many such sets on every heavy trace of every linear-time transposer is therefore equivalent to there being no linear-time transposer at infinitely many sizes, and gives no route to ¬(T). For oblivious machines, a hypothesis asserting unboundedly many such sets on every transposer, of any running time, would exclude oblivious linear-time transposers; the converse is not known.

The fibre argument needs correctness only on the inputs it counts, so the crossing bound of Theorem 8.1(a) extends to every trace followed by a set of at least $2^{(1-\eta)L}$ correctly transposed inputs; for an adaptive machine, these inputs must share that one trace. For oblivious machines, which have a single trace, this gives Theorem 5.5 in terms of the fraction of inputs transposed correctly. The constant $c_{\mathrm{E}}(M, C, \alpha, \eta)$ of Theorem 8.1(a) equals $c_{\mathrm{E}}(M, C, \alpha) - \eta/(\alpha w_{\mathrm{val}})$, and it is positive exactly for $\eta < \frac{1}{2}\alpha\rho_0(k, C, \frac{19}{20}\alpha)$.

Corollary 8.4 (Approximate transposers)**.** Lean: `correctSubset_crossingLowerGT2H_menger_lb_tight`, `obliviousOn_correctSubset_macroscopicCrossingLowers_menger_lb_tight` [`TightConstants.lean`]

Let M be any machine with k work tapes, $C \geq 1$, $\alpha > 0$ and $\eta \geq 0$.

(a) Let $d \geq \max(1, d_{\mathrm{br}}(k, C, \frac{19}{20}\alpha))$, and let τ be a trace of M followed by at least $2^{(1-\eta)L}$ inputs that M transposes within time C. For every $1 \leq H \leq \theta_{\mathrm{mac}}(M, C, \alpha)L$ and every set V_0 of at least αL input positions, every set of steps separating $u_0(V_0)$ from CL_H in the dependency DAG of τ has at least $c_{\mathrm{E}}(M, C, \alpha, \eta)|V_0|$ elements.

(b) Let $d \geq d_0$ with d_0 as in Theorem 5.5, and let M be oblivious at length L with halting time T and transpose at least $2^{(1-\eta)L}$ inputs within time C. Then the conclusion of Theorem 5.5 at H^* holds in the common trace, with $c_{\mathrm{E}}(M, C, \alpha, \eta)$ in place of $c_{\mathrm{E}}(M, C, \alpha)$; for other H, (a) applies, since every input follows the common trace.

Outline of the formal proof. (a) The proof of Theorem 8.1(a) goes through with Lemma 3.6 applied to the correctly transposed inputs that follow τ; the deficit $\eta L \leq (\eta/\alpha)|V_0|$ lowers the constant by $\eta/(\alpha w_{\mathrm{val}})$. (b) All inputs follow the common trace, which halts at T, and any correctly transposed input gives $T \leq C(L+1)$; apply (a) at H^*. With $\eta = 0$, (b) is Theorem 5.5. □

Read the other way, a certificate on each of few traces bounds the fraction of inputs that a machine transposes correctly.

Corollary 8.5 (Deficient traces carry few correct inputs)**.** Lean: `card_correctInputs_le_of_coneDeficit` [`SimulatorBridge.lean`]

Let M be any machine, $d \geq 1$, $c > 0$, and $\mathcal{T}$ a finite set of traces of M, each with a deficit certificate*: a set Γ_τ of steps and a set U_τ acyclic in the cone relation of τ avoiding Γ_τ with $|U_\tau| - w_{\mathrm{val}}|\Gamma_\tau| \geq \delta L$. Then at most $|\mathcal{T}|\, 2^{(1-\delta)L}$ inputs follow a trace in $\mathcal{T}$ and are transposed by M within time c. If $|\mathcal{T}| \leq 2^{\eta L}$, these inputs are at most a $2^{-(\delta-\eta)L}$ fraction of all inputs.*

Outline of the formal proof. For each $\tau \in \mathcal{T}$ followed by correctly transposed inputs, Lemma 3.6 applied to the set S_τ of these inputs gives $\delta L \leq L - \log_2|S_\tau|$. Sum $|S_\tau| \leq 2^{(1-\delta)L}$ over $\mathcal{T}$. □

9 Class exclusions

The results of this section exclude classes of machines defined by their head motion. They use the certificate method directly and reach machines that Theorems X and S do not: schedules with macroscopic crossings that are nevertheless excluded, and adaptive machines with no heavy trace.

9.1 Boundedly many macroscopic phases

A walk is *weakly monotone* on an interval of steps if it never moves left there, or never moves right (pauses allowed). It is *A-coarsely monotone* if it never moves more than A cells against a fixed direction: $x(t') \geq x(t) - A$ for all $t \leq t'$ in the interval, or $x(t') \leq x(t) + A$ for all such $t \leq t'$ (`CoarseMonoOnInterval`).

Definition 9.1 (Bounded macro-phases)**.** Lean: `WeakMonoOnInterval`, `BoundedMacroBank` [`MacroBankCrux.lean`]

Let M be a machine, $C \geq 1$, $M_{\mathrm{ph}} \geq 1$, B a set of work heads (the *bank*) and $S : \mathbb{N} \to \mathbb{N}$ with $S(d)/L \to 0$. M has *bounded macro-phases* with these parameters (`BoundedMacroBank`) if for all large d, whenever M is oblivious at length L with halting time $T \leq C(L+1)$, the interval $[0, T)$ splits into M_{ph} consecutive intervals on each of which the input head and every bank head are weakly monotone, and every other work head visits at most $S(d)$ cells before T.

The heads outside the bank are unrestricted inside their $o(L)$ workspaces: any number of reversals, any nesting, any coding.

Theorem 9.2 (Theorem B, bounded macro-phases)**.** Lean: `boundedMacroBank_has_crux`, `boundedMacroBank_excludes`, `suffixMacroBank_has_crux`, `suffixMacroBank_excludes` [`SuffixMassCrux.lean`]

If M has bounded macro-phases with bank B and M_{ph} phases, then M has crux rate $1/(4(|B|+1)^{M_{\mathrm{ph}}-1})$ at every depth, and for all large d there is no T with $\mathrm{ObliviousLinTime}(M,d,C,T)$. *The same holds, with crux rate $\delta_0/(4(|B|+1)^{M_{\mathrm{ph}}-1})$, if $\delta_0>0$ and the conditions of Definition 9.1 hold only on a final interval of steps that contains every append, with at least $\delta_0 d$ input rows not read before that interval; the motion before it is unrestricted.*

Outline of the formal proof. We prove the suffix form; the whole-run form is its case $\delta_0=1$, with the whole run as the final interval (`boundedSuffixMacroBank_of_boundedMacroBank`). Dependency paths decrease time, so a path from an output to a read of an untouched row stays inside the final interval (`getLastD_le_of_mem_depPath`). Let $K_0=(|B|+1)^{M_{\mathrm{ph}}-1}$ and cut workspace revisits in the final interval longer than $\lceil\sqrt{S(T+1)}\rceil$; there are $o(L)$ of them. Inside a phase a bank revisit goes back by one step; between phases, bank revisits are strictly monotone partial injections of fixed sign. So after deleting $o_D(L)$ exceptional outputs, every short dependency path from an output has a *signature*, the decreasing sequence of phases it enters through bank heads, and at most K_0 signatures start in a given phase. Thresholds on input addresses pull back along signatures to thresholds in time. Split the untouched input rows into $2K_0$ buckets: each unsplit output row forbids at most K_0 buckets, so some bucket is missed by half of the output rows, and the rectangle of that bucket and those output rows, minus the exceptions, is independent in the cone relation; it has $(\delta_0/(4K_0)-o(1))L$ elements (`suffixMacroBank_independent_exists`). □

For adaptive machines the phases and the workspaces may depend on the input, and the walks may jitter.

Theorem 9.3 (Theorem J, jittered phases, any adaptivity)**.** Lean: `native_machine_preserves_suffixJitterBM`, `nativeGapBound_of_adaptiveJitterBM`, `adaptiveJitterBM_excludes_transposesAt`, `adaptiveJitterBM_not_linearTimeTransposer`, `adaptiveSuffixJitterBM_excludes_transposesAt`, `adaptiveSuffixJitterBM_not_linearTimeTransposer` [`AdaptiveSuffixJitterBM.lean`]

Let M be any machine, $c>0$, $M_{\mathrm{ph}}\ge 1$, B a set of work heads, and $A(d),S(d)=o(L)$. Suppose that for all large d every run of M on an input of length L that halts within $c(L+1)$ steps splits into M_{ph} intervals on each of which the input head and the bank heads are $A(d)$-coarsely monotone, while every other work head visits at most $S(d)$ cells; the intervals and cells may depend on the input. Then M does not transpose within time c at all large d; if such M_{ph}, B, A and S exist for every $c\ge 1$, then M is not a linear-time transposer. The same holds if $\delta_0>0$ and the conditions hold only on a final interval containing every append with at least $\delta_0 d$ rows not entered by the input head before it.

Outline of the formal proof. The bit simulator of the fifteenth paper preserves the shape, also in its suffix form, with the jitter and the workspace enlarged by $O(s)$ (`native_machine_preserves_suffixJitterBM`). On each frozen trace of the simulator, the proof of Theorem B goes through with jitter: the revisits that straddle a pulled-back cut on a bank head occupy at most $2A+1$ cells (`hop_upper_injOn`), and input reads are pulled back up to a band of width A. This gives every storage code on the frozen cone relations of every simulator rate at most $1-1/(4K_0)+o(1)$ at the sizes at which M transposes within time c (`nativeGapBound_of_adaptiveJitterBM`), which is the hypothesis of Theorem 9.4 below; so Theorem G excludes M. □

At a fixed number of phases these classes, in their whole-run forms, also yield to pass counting at their own time bounds, by Theorem 10.3, which extends the grouped pass bound of the first paper [9, Theorems 5.8 and 5.12] to coarse passes with small workspaces. Moreover, the same theorem excludes these shapes, imposed on whole runs of any length, at any running time for

oblivious machines and at any polynomial running time for adaptive ones, even with jitter and workspaces up to a small fixed fraction of L (Corollary 10.5). What Theorems B and J add is an explicit native coding deficit, and the suffix forms, which pass counting does not reach: an unrestricted prefix may read $(1-\delta_0)d$ rows, and conditioning on the configuration at the start of the suffix leaves classes of inputs of co-entropy up to $(1-\delta_0)L$, for which the published robust pass bounds [9, 13] say nothing.

9.2 A native gap suffices

Roots. The hypotheses of Theorem 9.4 and Corollaries 9.5 and 9.6 below are those of the fourteenth paper's native bound [6, Corollary L] with a general deficit (`NativeGapBound`). They concern the cone relation of a run *with arbitrary roots*: for each output position, the root may be any step before the halting step at which the output head is on that position's cell, and the rate bound is required for every such choice of roots. The append steps are one such choice; whether the results hold when the bound is required only for them is not proved.

Theorem 9.4 (Theorem G, native gap criterion)**.** Lean: `nativeGapBound_half_iff`, `native_gap_excludes_transposesAt`, `native_gap_excludes_linear` [`NativeGapCriterion.lean`]

Let M be a machine and $c>0$. Suppose that there is $\rho>0$ such that, for every $s\ge 1$ and every depth D, every binary storage code on the cone relation with arbitrary roots, after a genie of full depth D, of every run of the simulator N_s of M [14, Theorem S] that halts within $(11c+1)(L+1)$ steps on an input of length L (entries of $b(d)$ bits) has rate at most $1-\rho+o(1)$, where d ranges over the sizes at which M transposes within time c. Then M transposes within time c at only finitely many d. If such a ρ exists for every machine M and every $c>0$, then (T) fails.

Outline of the formal proof. Choose s so that, at all large d at which M transposes within time c, N_s transposes within time $11c+1$ with at most $2^{(\rho/2)L}$ traces [14, Theorem S]. Some trace of N_s carries $2^{(1-\rho/2)L}$ inputs; after the genie, of cost $\frac{1}{4}\rho L$, its fibre splits into storage codes on its cone relation (`MergeDepth.blockMachine_isStorageCode_fibre`), one of which has rate at least $1-\frac{3}{4}\rho-o(1)$, contradicting the hypothesis. If (T) held, some machine M would transpose in time $c(L+1)$ at infinitely many sizes. At $\rho=\frac{1}{2}$ the hypothesis is the native half bound of the fourteenth paper [6, Corollary L] on the simulators of M. □

Corollary 9.5 (Theorem G, uniform form)**.** Lean: `native_gap_exists_exclude_linear`, `native_gap_tests_exclude_linear` [`NativeGapCriterion.lean`]

Suppose that for every k and $C\ge 1$ there is $\rho(k,C)>0$ such that, for every machine with k work tapes and every depth D, every binary storage code on the cone relation with arbitrary roots, after a genie of full depth D, of every run that halts within $C(L+1)$ steps on an input of length L (entries of $b(d)$ bits) has rate at most $1-\rho(k,C)+o(1)$. Then (T) fails.

Outline of the formal proof. The simulators of a machine M with k work tapes have k work tapes (`native_machine_k`), so for every $c>0$ the deficit $\rho(k,11c+1)$ satisfies the hypothesis of Theorem 9.4 for M and c. At $\rho=\frac{1}{2}$ the hypothesis is the native half bound of the fourteenth paper [6, Corollary L], assumed uniformly over the machines with k work tapes and over all depths, as in the eighth paper [5, §6.2]. □

The fifteenth paper reaches the same conclusion from the half bound on chosen simulators [14]; Theorem G accepts any fixed deficit, chosen for each machine and time bound separately and needed only on that machine's simulators. In the uniform form of Corollary 9.5 the hypothesis is at least as

strong as a uniform crux; its value is that the deficit may be tiny, which is the form that Theorems B and J produce, but only for their classes.

For a single machine of low adaptivity the same counting needs no simulator: a deficit above the entropy of the trace suffices.

Corollary 9.6 (A native gap above the trace entropy)**.** Lean: `fSecond_native_gap`, `native_gap_exclusion_of_trace_support` [`NativeGapCriterion.lean`]

Let M be a machine with k work tapes, let $c > 0$ and $\eta < \rho$, let β_1 be the block width of M at scale 1 (`MergeDepth.blockWidth`*), and put $D = \mathrm{genieDepth}(k, \beta_1, \max(c, 1), (\rho - \eta)/2)$. Suppose that every binary storage code on the cone relation with arbitrary roots, after a genie of full depth D, of every run of M that halts within $c(L + 1)$ steps has rate at most $1 - \rho + o(1)$. Then at all large d at which M transposes within time c, the skeleton of M at scale 1, and hence its trace, has entropy greater than ηL on a uniformly random input (*`TapeStorageRates.skeletonOfTrace_eq`*, `MergeDepth.entropy_comp_le`*). Moreover, if a machine N satisfies the same hypothesis, with its own k, β_1 and D, and for all large d every d at which M transposes within time c is one at which N transposes within time c with at most $2^{\eta L}$ distinct traces, then M transposes within time c at only finitely many d.*

Outline of the formal proof. If the trace entropy were at most ηL, some trace would be followed by at least $2^{(1-\eta)L}$ inputs. After the genie of cost $\frac{1}{2}(\rho - \eta)L$, its fibre splits into storage codes on the cone relation, one of which has rate at least $1 - \eta - \frac{1}{2}(\rho - \eta) > 1 - \rho$, contradicting the hypothesis. This is the argument of Theorem 9.4 with M in place of the simulator; at $\rho = \frac{1}{2}$ it is the fourteenth paper's low-adaptivity corollary [6, Corollary L]. □

9.3 Few scales and no free head

For a work head, a *family* is the set of its revisits on one side that share their *turning point*, the first step inside the revisit at which the head is farthest from the cell; its *height* is the distance from the turning point to the family's outermost cell, the cell of its longest revisit, as in the eighth paper [5]. The head has *owner load* at most ℓ if at every step, on each side, the revisits open at that step have at most ℓ distinct turning points. Bounded cell entry V gives owner load at most $2V - 1$.

Theorem 9.7 (Theorem Q, scale-gap pigeonhole)**.** Lean: `zeroFreeHead_qWindow_excludes`, `zeroFreeHead_entries_qWindow_excludes`, `zeroFreeHead_qWindow_oblivious_excludes` [`ZeroFreeHeadQWindow.lean`]

Let c, β be reals and k, ℓ, q, h natural numbers. There are $\varepsilon > 0$ and d_0 such that for every $\eta < \varepsilon$ and $d \geq d_0$ no machine with at most k work tapes and value width at most β transposes at size d within time c with an η-heavy trace on which every work head has owner load at most ℓ and the binary logarithms of the heights of all families lie in at most q windows of width h. The input head is unrestricted. For oblivious machines the statement holds at every $d \geq d_0$ with $\eta = 0$.

Outline of the formal proof. Owner load ℓ colours the families with $N = 2k \max(\ell, 1)$ classes of non-overlapping spans, so their heights sum to less than NcL. Choose thresholds $Q_0 < Q_1 < \cdots < Q_q$, each so large compared with the last that a window can be *bad*, that is contain the height of a family longer than the current threshold, for at most one index. With q windows and $q + 1$ indices some index is bad for no window, so only boundedly many families are large, and the eleventh paper's low-adaptivity theorem for boundedly many large families [10] gives the contradiction. Its constant comes from a compactness argument and is not explicit [17]. □

Theorem Q is the case without a free work head of the eighth paper's bounded-load, boundedly-many-scales class, which is conditional on the crux there [5]; here it is unconditional, with any input

motion and low adaptivity. With $\Theta(\log d)$ scales instead of boundedly many, the question is open already for two work tapes.

10 What a linear-time transposer needs

This section reads the results backwards, as a specification that any oblivious linear-time transposer must meet, and then describes how close known schedules come to meeting it.

10.1 The specification

Suppose M is an oblivious machine with k work tapes that transposes in time $C(L+1)$ at infinitely many sizes. By the results of this paper and of the series, at all large such sizes:

(i) *Shape.* At scale $\theta L/4$ each work tape's walk is one of finitely many words, and some tape turns at least twice with overlapping legs, so it reuses a stretch of tape like a queue (Theorem 6.7). Along infinitely many sizes the shape is one fixed tuple of words.

(ii) *Crossings.* From every set of at least αL outputs, a linear number of vertex-disjoint dependency paths reach the earlier ends of crossing revisits of excursion more than θL (Theorem 5.5), and the crossing steps of one coarsely monotone leg of one tape already carry a linear fraction of them; the tape and the leg may depend on the output set and on the size, and are fixed along infinitely many sizes for the set of all outputs (Theorem 6.7).

(iii) *Two long-range heads.* At least two work heads revisit cells after moving a fixed fraction of L away, and for each work head some set containing a fixed fraction of the outputs reaches long revisits off that head with linear expansion (Theorem 4.1).

(iv) *Rows to columns.* For every two sets of rows A_0, A_1, every set Γ of steps separating the output steps $u_0(Q^*(A_0, A_1))$ of the entries that rows A_0 feed into output rows A_1 from the reads $\mathsf{In}(A_0)$ of rows A_0 satisfies $w_{\mathrm{val}}|\Gamma| + |\Gamma \cap u_0(Q^*(A_0, A_1))| \geq |A_0||A_1|b$, and in particular $|\Gamma| \geq |A_0||A_1|b/(w_{\mathrm{val}}+1)$ (Lemma 7.1).

(v) *Codes.* After a genie of cost εL and bounded depth, the cone relations of the schedule carry an index code of at most εL bits, one of whose fibres is a storage code of rate at least $1-\varepsilon$ [6, Corollary O]; by (i)–(iii), the relations involved are those of macroscopic queue-like reuse.

(vi) *Non-regular motion.* With $G = \max(1, \lceil \log_2(g+1) \rceil)$, in every uniform subsequential limit of the work-head trajectories rescaled in time and space by L, any open subsets $R_1, \ldots, R_k \subseteq [0, C+1/G]$ on which the respective limits are locally affine have complements in $[0, C+1/G]$ whose measures sum to at least $1/(2G)$ (hence omit measure at least $1/(2kG)$ when one common open subset serves all work heads) [15, Theorem 6.3, Lean-verified]. In particular, in every contracting stage clock [15, Definition 7.2], the stages whose rescaled head profiles do not tend to paths with finitely many affine pieces [15, Definition 7.4] take at least $L/(2kG)$ steps up to $o(L)$, so the trace is not an all-regular contracting recursion [15, Theorem 7.5, Lean-verified]. A data-free prefix of expanding excursions that clears its workspace before the computation starts already satisfies the pure motion bounds [15, Theorem 8.1, Lean-verified]; the computation after it is a run of its own, to which [15, Theorem 7.5] still applies, so bent motion helps only if it carries data.

Conditions (i)–(iv) are proved here, and (vi) in [15], without any coding hypothesis. Condition (v) is the coding question itself: a linear-time oblivious transposer is a code of rate close to 1 on the cone relations of a schedule meeting (i)–(iv) and (vi). Machine-free relations carry storage codes above rate one half [11], and binary codes of rate approaching one exist on suitable separated relations [14].

10.2 Mechanisms that fail

Each of the following design patterns is excluded for oblivious linear-time transposition, whatever the machine computes on its tapes.

- *Stack-like reuse at macroscopic scale.* A schedule whose long revisits are nested, so that every tape reuses its storage like a stack at scale θL, has no macroscopic crossing steps, and is excluded by Theorem 5.5.

- *One long-range head.* One unrestricted work head together with heads whose revisits stay below a small fixed fraction of L is excluded by Theorem 4.1, whatever the input head does.

- *Boundedly many macroscopic phases.* Schedules in which the input head and some bank heads are weakly monotone on boundedly many phases while the other heads move inside sublinear workspaces are excluded by Theorem 9.2; with sublinear jitter on the bank heads, and for adaptive machines, by Theorem 9.3. Imposed on whole runs, these shapes are excluded by Theorem 10.3 at any running time for oblivious machines and at any polynomial running time for adaptive ones, even with jitter and workspaces up to a small fixed fraction of L (Corollary 10.5), and so are phase profiles that grow too slowly across scales (Corollary 10.4).

- *Cheap row-to-column cuts.* A schedule with a set Γ of steps separating the output steps $u_0(Q^*(A_0, A_1))$ of some rows A_0 in output rows A_1 from the reads $\mathsf{In}(A_0)$ of rows A_0 and satisfying $w_{\mathrm{val}}|\Gamma| + |\Gamma \cap u_0(Q^*(A_0, A_1))| < |A_0||A_1|b$ is excluded by Lemma 7.1; Theorem 7.2 gives a convenient way to exhibit such cuts.

- *Regular contracting recursions.* A schedule whose steps admit a contracting stage clock in which, at every fixed stage, every eligible subsequence of rescaled work-head profiles has a further uniform limit with a finite affine interval cover, with zero exceptional-time allowance, is excluded for fixed work alphabets (even with $o(L)$ state bits and correctness on at least a $2^{-o(L)}$ fraction of inputs) by [15, Theorem 7.5, Lean-verified]. The companion memo [16, Theorem 1.1] verifies this clock-regularity hypothesis on all five cascade schedules of Section 10.3 and excludes them along large dyadic d both at $b = \max(1, \lceil \log_2 d \rceil)$ and (via an oblivious bit-lane lift) at $b = 1$, with the canonical-width fifth cascade formalized in Lean for its declarative tick word.

- *The trapped folds of the third and fourth cascades.* Each fold of these schedules starts with a transfer at half speed, and the data of rows 1 and 2 stay in its squeezed part at every level. The narrowing window gives a sublinear separator for Lemma 7.1 on these rows and cell-level cuts that allow work alphabets growing with d (Theorem 10.8). The first and second cascades fold the same way, but a shuffle first spreads each row over several distant intervals of the tape, and we know no cut that excludes them.

- *The explicit families of Appendix A*, each excluded by a class theorem of Section 9 or a certificate of Section 3 (in proof sketches). Several of them make macroscopic crossings with linear expansion (numerically, at $d \le 128$), so Theorem 5.5 alone does not exclude them.

10.3 Two-sided cascade schedules

Five explicit oblivious schedules, which we call *two-sided cascades*, meet the crossing and skeleton conditions of this paper that we tested. Each loads the input onto one, two or three work tapes and then runs a cascade over the scales $M_r = L/2^r$, $1 \le r < \log_2 d$. A forward stage folds the data at each scale: one half by a transfer at varying relative speed, the other merged in with a cyclic shift of about $M_r/3$. A backward stage unfolds it by 1-to-2 doublings with an unshifted and a shifted half. A final sweep emits the L output bits. Every stage at scale r moves the heads over $O(M_r)$ cells a bounded number of times, so the running time is $O(\sum_r M_r) = O(L)$. The five differ as follows.

1. Two work tapes; before the cascade, two symmetric pairs of two-dimensional cyclic shifts of the entry grid are applied to a copy of the input.

2. Two work tapes; before the cascade, a full reversal (a half-turn of the entry grid when $b = 1$) and two butterfly shifts, by $2^{m-1}(d-1)b$ and $(d-1)b$ where $d = 2^m$, are applied to a copy.

3. Three work tapes, every cell entered at most ten times; the third tape holds disjoint zones for the folded data at all scales and moves through them monotonically in each stage.

4. Three work tapes; the forward and backward stages use disjoint zones for all scales, and no zone is overwritten.

5. Two work tapes; the folds are 2-to-1 compressions, and every scale has a comb pass in which one tape makes 2^r teeth while the other waits.

They are specified in our experiments as oblivious traces (head motion and append steps); whether some machine with one of these oblivious traces transposes is Problem 11.4, answered negatively for fixed work alphabets in [16].

Remark 10.1 (Numerical)**.** At $d \in \{32, 64, 128\}$ and $b = 1$ the measured running times of the five schedules lay between about $17L$ and $90L$ ($17.46L$ to $89.19L$). On all five, the minimum set separating a large output set from the earlier ends of crossing revisits of excursion greater than E had exactly as many elements as the output set, for every tested $E \le L/2$. Their work-tape skeletons at scale about $L/6$ changed little with d—identical across $d = 32, 64, 128$ on the fifth schedule, and varying by at most 8 legs (about 12%) on the first four—with a single leg carrying linear expansion. Condition (iii) of the specification was not measured here. Condition (iv) fails on the third and fourth schedules (Theorem 10.8), and all five fail condition (vi) asymptotically [16, Theorem 1.1]. So, as far as these measurements go, conditions (i) and (ii) do not distinguish the first, second and fifth schedules from a transposer. A one-pass shear, excluded along even d via Theorem 9.3, and a multi-scale stack schedule, excluded by a certificate (both as proof sketches in Appendix A), also make macroscopic crossings with linear expansion on a single leg in these measurements, which suggests that (i) and (ii) are not sufficient for transposition.

10.4 Why cuts alone cannot finish

The certificates of this paper are cuts: they bound from below the number of steps that must separate inputs from outputs. A single cut can only certify storage, and the next proposition shows that for transposition one set of steps serves as the cut for every pair of row sets at once.

Proposition 10.2 (One cut for every pair of row sets)**.** Not formalized; a complete proof is given below. *Let a run with horizon T read its last input cell at step t_{in}, and put $t^* = t_{\text{in}} + 1$. Let W^* consist of*

- *every step $u < t^*$ that is the earlier end of a revisit (u, t) with $t \geq t^*$,*
- *every output step before t^*, and*
- *the step t^*, if $t^* < T$.*

Then for every two sets A_0, A_1 of rows, W^* meets every dependency path from the output steps of the entries that rows A_0 feed into output rows A_1 to the steps that read rows A_0. The steps of the first kind number at most the number of work cells that are visited both before t^* and at or after t^*.

Proof (not formalized). Take a dependency path from an output step t_0 to an input-reading step u. Every edge of the dependency DAG goes from a step to a strictly earlier step: the state edge $a \to a - 1$, or the revisit edge $a \to a'$ for a revisit (a', a). If $t_0 < t^*$, then t_0 is an output step before t^* and lies in W^*. Otherwise $t_0 \geq t^* > t_{\text{in}} \geq u$, so the path has an edge $a \to a'$ with $a \geq t^* > a'$. If it is a state edge, then $a' = a - 1$, so $a = t^*$, and $t^* \in W^*$. If it is a revisit edge, then (a', a) is a revisit with $a' < t^* \leq a$, so $a' \in W^*$. In each case the path meets W^*; this argument does not use A_0 or A_1. A step of the first kind is the earlier end of a revisit of some tape that spans t^*; on each tape, each cell is the cell of at most one such revisit, namely the one from its last visit before t^* to its next visit. □

For a transposer, Lemma 7.1 with $A_0 = A_1$ the set of all rows gives $|W^*| \geq (1 - \eta)L/(w_{\text{val}} + 1)$. When few outputs precede the last input read, this says that the storage live across t^* is linear. In any case Proposition 10.2 shows that this one bound already accounts for every pair of row sets: however finely the rows are split, cut certificates for different splits may coincide, and a lower bound beyond linear time cannot come from counting disjoint cuts unless something other than the cut property forces the cuts apart. The colouring criterion of Section 7 verifies such a separation on specific schedules (Corollary 7.3), but it does not supply one in general.

Outside such restricted classes, the only coding-proof logarithmic factor known on tapes is of a different kind. For an oblivious transposer whose run has r monotone passes, Theorem P of the first paper gives $k\,db \leq ((k+1)^r - 1)(2b + \sigma_M)$, where σ_M is the number of bits of a state and the scanned symbols, so a fixed machine needs $r \geq \log_{k+1} d - O(1)$ passes [9, Theorem 5.9]; for adaptive machines an extra term appears [9, Theorem 5.12]. Its grouped form [9, Theorem 5.8] reads $d^2 b \leq H_{\text{addr}}((\gamma+1)db + \lceil d/\gamma \rceil \sigma_M)$ for every $\gamma \geq 1$, where $H_{\text{addr}} = ((k+1)^r - 1)/k$ bounds the number of addresses of the forests in its proof.

The grouped form extends to passes that are monotone only up to a fixed amplitude, with some heads confined to small workspaces. Call a set B of work tapes a *bank*, with $k_b = |B|$. A *coarse pass partition* of a run up to a step T', with input amplitude A_{in}, bank amplitude A and workspaces S_w, splits $[0, T')$ into r consecutive passes on each of which the input head is A_{in}-coarsely monotone and every bank head is A-coarsely monotone, while every work tape $w \notin B$ visits at most S_w cells before T' (`CoarseWorkspacePartition`). Put $S_{\text{tot}} = \sum_{w \notin B} S_w$,

$$H(k_b, r) = \sum_{i<r} (k_b + 1)^i, \qquad \sigma_{A,S} = \lceil \log_2(q+1) \rceil + \big(k_b(3A+1) + S_{\text{tot}}\big) \lceil \log_2(g+1) \rceil,$$

and, for adaptive machines with running time T, $\sigma'_{A,S} = \sigma_{A,S} + k_b + \sum_{w \notin B} \lceil \log_2(S_w + 1) \rceil + (k + 4) \lceil \log_2(2T+1) \rceil$.

Theorem 10.3 (Pass counting with coarse passes)**.** Lean: `theoremP_coarseWorkspace_grouped_on_bits`, `theoremPprime_coarseWorkspace_grouped_on_bits`, `oblivious_singleScale_coarseWorkspace_closed_lb`, `adaptive_singleScale_coarseWorkspace_closed_lb` [`CoarseGroupedPass.lean`]

Let M be a machine, B a bank of its work tapes, $\gamma \geq 1$, and $\mathcal{I}$ a set of inputs of length $L = d^2 b$ with $2^L \leq 2^m |\mathcal{I}|$.

(a) If M is oblivious on $\mathcal{I}$ with halting time T, transposes every input of $\mathcal{I}$, and its run has a coarse pass partition with r passes up to T, then

$$d^2 b \;\leq\; \lceil d/\gamma \rceil \, m + H(k_b, r)\big((\gamma+1)db + 2A_{\text{in}} + \lceil d/\gamma \rceil \, \sigma_{A,S}\big).$$

(b) If every input of $\mathcal{I}$ is transposed by step T, and the run on each input has a coarse pass partition with r passes up to the step at which its output is complete, the same holds with $\sigma'_{A,S}$ in place of $\sigma_{A,S}$.

In particular, if $d, b, k_b \geq 1$ and M transposes every input of length $d^2 b$ in this way, obliviously, then for every $\gamma \geq 1$

$$r \;>\; \log_{k_b+1} \frac{d^2 b}{(\gamma+1)db + 2A_{\text{in}} + \lceil d/\gamma \rceil \, \sigma_{A,S}},$$

and with $\sigma'_{A,S}$ in place of $\sigma_{A,S}$ in the adaptive case, where the partitions may have at most r passes.

Outline of the formal proof. The proof is that of the first paper's grouped bound [9, Theorems 5.8 and 5.12], with the nodes of its forests anchored at windows. For a node with anchor step a on a bank head h, the window $[h(a) - 2A, h(a) + 2A]$ is used: visits after a that lie inside it stay in $3A+1$ cells in the direction of the pass, cells outside it that are visited at or after a were not visited earlier in the pass, and consecutive nodes in time are at least $A+1$ cells apart, which keeps the forests' time separation in all four combinations of directions. Each node records the $k_b(3A+1)$ bank cells of its windows and the S_{tot} workspace cells, which is $\sigma_{A,S}$; the input head's amplitude costs $2A_{\text{in}}$ in the coverage count. For adaptive machines the direction bits, the workspace offsets and the step structure are recorded as well, which is $\sigma'_{A,S}$. □

At $A_{\text{in}} = A = 0$, with every work tape in the bank, part (a) is the first paper's grouped bound (`oblEntryBits_zero_univ_eq`). Across scales it gives the following.

Corollary 10.4 (Phase profiles below a threshold)**.** `Lean`:
`subcriticalPhaseProfile_oblivious_excludes`, `subcriticalDimPhaseProfile_oblivious_excludes`, `subcriticalPhaseProfile_adaptive_polyTime_excludes` [`CoarseGroupedPass.lean`]

Let M be a machine and B a bank of $k_b \geq 1$ of its work tapes. Let $0 < \theta \leq 2$, put $\varepsilon_d = d^{-\theta}$, and let A_{slope}, C_0 and $C_{\text{in}}, C_{\text{amp}}, C_{\text{ws}} \geq 0$ be reals with $A_{\text{slope}} \log_2(k_b+1) < \frac{1}{2}$, and let C_T and κ be natural numbers. For all large d,

(a) M is not, at any running time, an oblivious transposer at size d whose run has a coarse pass partition with at most $A_{\text{slope}} \log_2(1/\varepsilon_d) + C_0$ passes, amplitudes $A_{\text{in}} \leq C_{\text{in}} \varepsilon_d L$ and $A \leq C_{\text{amp}} \varepsilon_d L$, and workspaces with $S_{\text{tot}} \leq C_{\text{ws}} \varepsilon_d L$;

(b) M does not transpose every input at size d within $C_T L^\kappa$ steps if the run on each input has such a partition up to the step at which its output is complete, with A_{in}, A and the workspaces S_w the same for all inputs (the passes may depend on the input).

Measured against the scale $\sqrt{\varepsilon_d} = d^{-\theta/2}$ of the matrix dimension, the condition of (a) reads $A_{\text{dim}} \log_2(k_b+1) < 1$ for at most $A_{\text{dim}} \log_2(1/\sqrt{\varepsilon_d}) + C_0$ passes (`subcriticalDimPhaseProfile_oblivious_excludes`).

The threshold $\frac{1}{2}$ is the limit of the grouped inequality at every $0 < \theta \leq 2$ and every $k_b \geq 1$, once workspaces of order $\varepsilon_d L$ are allowed: Corollary 10.4 excludes every slope product below $\frac{1}{2}$, and

at no $\theta > 0$ is one above $\frac{1}{2}$ excluded. (For $\theta > 2$ the limit is $1/\theta$.) If $\psi = A_{\text{slope}} \log_2(k_b + 1) > \frac{1}{2}$ and $C_{\text{ws}} > 0$, then for all large d, with $r = \lfloor A_{\text{slope}} \log_2(1/\varepsilon_d) + C_0 \rfloor$ passes and $S_{\text{tot}} = \lfloor C_{\text{ws}} \varepsilon_d L \rfloor$, the largest values that Corollary 10.4 admits, the inequality of Theorem 10.3(a) holds for every $\gamma \geq 1$, every A_{in} and A, every bank of k_b tapes and every machine with at least two work symbols (`groupedBound_not_excluded_above_half`). Indeed $H(k_b, r) \geq (k_b + 1)^{C_0 - 2} d^{\theta\psi}$, and $\sigma_{A,S} \geq S_{\text{tot}}$ gives $(\gamma + 1)db + \lceil d/\gamma \rceil \sigma_{A,S} \geq 2\sqrt{C_{\text{ws}}}\, d^{2-\theta/2} b$, so the right-hand side is at least $2\sqrt{C_{\text{ws}}}\,(k_b + 1)^{C_0 - 2} d^{2+\theta(\psi - 1/2)} b$. Without such workspaces the limit is different: at $C_{\text{ws}} = C_{\text{amp}} = 0$ and $\theta < 2$, taking $\gamma = 1$ makes the grouped inequality exclude slope products up to $\min(1, 1/\theta)$, and up to $1/\theta$ if moreover $C_{\text{in}} = 0$, by a direct computation that is not formalized. At $\theta = 2$ no workspace is needed: with $d = f^P$, $k_b = 2f$ and $r = P + 1$ passes, the inequality of Theorem 10.3(a) holds for every machine and every $P \geq 1$, and the slope products $\log_2(2f + 1)/(2 \log_2 f)$ of these parameters tend to $\frac{1}{2}$ (`groupedBound_thetaTwo_not_excluded_above_half`, `perfectUnshuffleSlopeProduct_tendsto_half`). Proof sketch only: these parameters are realized by an f-way perfect-unshuffle transposer, which deals the entries round-robin onto f tapes, reads the other f tapes backward, repeats this P times, and finds entry boundaries with a counter of $O(b)$ cells; the index identity of its rounds is `unshuffleStep_iter_transpose`. So at $\theta = 2$ no constant above $\frac{1}{2}$ works for every k_b. For a fixed k_b, and for $\theta < 2$, we do not know whether transposers with slope products close to $\frac{1}{2}$ exist.

Over whole runs, boundedly many passes are excluded even with amplitudes and workspaces up to a small fixed fraction of L; choosing γ as a fixed fraction of d gives the following.

Corollary 10.5 (Boundedly many coarse passes)**.** Lean: `boundedCoarsePasses_oblivious_excludes`, `boundedCoarsePasses_adaptive_polyTime_excludes`, `boundedCoarsePasses_sublinear_oblivious_excludes`, `boundedCoarsePasses_sublinear_adaptive_polyTime_excludes` [`CoarseGroupedPass.lean`]

Let M be a machine, B a bank of its work tapes and r_0 a natural number.

(a) There is $\varepsilon > 0$ such that for all large d, M is not, at any running time, an oblivious transposer at size d whose run has a coarse pass partition with at most r_0 passes, amplitudes $A_{\text{in}}, A \leq \varepsilon L$ and workspaces with $S_{\text{tot}} \leq \varepsilon L$.

(b) There is $\varepsilon > 0$ such that for all natural numbers C_T and κ and all large d, M does not transpose every input at size d within $C_T L^\kappa$ steps if the run on each input has such a partition up to the step at which its output is complete, with A_{in}, A and the workspaces S_w the same for all inputs (the passes may depend on the input).

In particular, (a) and (b) hold with $A_{\text{in}}, A, S_{\text{tot}} \leq \omega(d)$ in place of εL, for every function $\omega(d) = o(L)$.

Outline of the formal proof. Let $H_0 = H(k_b, r_0)$, $N = 4(H_0 + 1)$, $\gamma = \lfloor d/N \rfloor + 1$, $c = (3k_b + 1)\lceil \log_2(g + 1) \rceil$ and $\varepsilon = 1/(N(2 + Nc))$, so that $\lceil d/\gamma \rceil \leq N$. If $A_{\text{in}}, A, S_{\text{tot}} \leq \varepsilon L$, then $\sigma_{A,S} \leq c\varepsilon L + O(1)$, and the right-hand side of Theorem 10.3(a) is at most $(2H_0/N)L + O(H_0 db) < L$ for large d. In (b) the extra terms of $\sigma'_{A,S}$ are $O(b)$, because $T \leq C_T L^\kappa$, $S_w \leq L$ and $L \leq 2^{3b}$, so the same ε serves every C_T and κ. The $o(L)$ forms follow, since $\omega(d) \leq \varepsilon L$ for large d. □

A linear-time run may have up to about $4kC/\varepsilon$ legs at scale εL (Lemma 6.3), and three of the cascades of Section 10.3 (the first, third and fourth), whose number of phases at scale $2^{-s}L$ grows linearly in s, have slope products between 25 and 40, far above the threshold of Corollary 10.4 (numerically, at $d = 128$, $b = 7$). So pass counting, like cut counting, cannot exclude linear time one scale at a time. Problem 11.1 asks for a pass bound that counts phases across all scales at once.

10.5 Cascades 3 and 4 fail the rows-to-columns condition

All five cascades are excluded by the regular-recursion obstruction of the second paper [15, Theorem 7.5], as Astra's memo [16] verifies (Problem 11.4). For every fixed work alphabet there are $\delta > 0$ and d_0 such that, at every dyadic $d \geq d_0$, a machine with one of the five traces and states $\{0, \dots, q\}$ that transposes the inputs of a set S satisfies $\lceil \log_2(q+1) \rceil + L - \log_2 |S| \geq \delta L$. That argument concerns the motion of the heads and gives no numerical threshold.

For Cascades 3 and 4 a cut does more. Condition (iv) of the specification already fails for the rows $A_0 = \{1, 2\}$: an explicit set of $o(db)$ steps separates their reads from the outputs they feed. Counting single cells instead of whole steps then bounds the work alphabet and the number of states separately, and both may grow with d. The cut also locates the loss of information.

Definition 10.6 (Cell network)**.** Let a trace have horizon T. For a work tape j and a step $t < T$, let $\mathrm{next}_j(t)$ be the next step after t at which head j scans the cell it scans at step t, if there is one. The *cell network* of the trace has the vertices $0, \dots, T-1$, a *state edge* $t \to t+1$ for every $t+1 < T$, and a *cell edge* $t \to \mathrm{next}_j(t)$ of tape j for every j and t for which $\mathrm{next}_j(t)$ exists. A cell edge $t \to t+1$ is a *wait*.

The cell network runs forward in time and keeps waits as edges of their own. Reversing every edge of the dependency DAG gives edges of the cell network: a state edge $t \to t-1$ becomes the state edge $t-1 \to t$, and the revisit edge of a revisit (p, u) becomes the cell edge $p \to \mathrm{next}_j(p) = u$.

Lemma 10.7 (Cell-level cut-set bound)**.** Not formalized; a complete proof is given below.

Let M be oblivious with trace τ, work alphabet $\{0, \dots, g\}$ and states $\{0, \dots, q\}$, and let it transpose at size d. Let A be a set of input positions, $V \subseteq A$, and C a set of edges of the cell network of τ, consisting of C_{cell} cell edges and C_{state} state edges, such that every path from a step that reads a position of A to a step of $u_0(V)$ uses an edge of C. Then

$$|V| \;\leq\; |C_{\mathrm{cell}}| \log_2(g+1) + |C_{\mathrm{state}}| \log_2(q+1).$$

Proof (not formalized). Fix the input bits outside A. Let U be the set of steps reachable by a path that avoids C from a step that reads a position of A. Every edge from a step of U to a step outside U lies in C. A step reads the state left by the step before it and the symbols last written on the cells it scans (the blank on a first visit), so the state it enters, the symbols it writes and the bit it appends are a function of the values carried by its incoming edges and of the input bit it reads. By induction on t, everything a step $t \notin U$ computes is a function of the values carried by the edges of C: such a step reads no position of A, and each of its incoming edges starts outside U or lies in C. The steps of $u_0(V)$ lie outside U, and at $u_0(v)$ the machine appends the input bit v. So the bits $(x_v)_{v \in V}$ are a function of the values on C. They take all $2^{|V|}$ values as the bits of A vary, and the values on C take at most $(g+1)^{|C_{\mathrm{cell}}|}(q+1)^{|C_{\mathrm{state}}|}$. □

Theorem 10.8 (Cascades 3 and 4)**.** Not formalized; a complete proof is given below.

There are absolute constants c and d_0 with the following property. Let $d = 2^k \geq d_0$, let τ be the trace of Cascade 3 or of Cascade 4 at size d, and put $A_0 = \{1, 2\}$, $A_1 = \{0, \dots, d-1\}$, and $\nu = (db)^{1/5}$ for Cascade 3 and $\nu = (db)^{1/6}$ for Cascade 4.

(a) Some set of at most $c\,db/\nu$ steps separates $u_0(Q^*(A_0, A_1))$ from $\mathsf{In}(A_0)$. Hence, by Lemma 7.1, no oblivious machine with trace τ and $w_{\mathrm{val}} + 1 < \nu/c$ transposes at size d.

(b) No oblivious machine with trace τ, work alphabet $\{0, \dots, g\}$ and states $\{0, \dots, q\}$ with $\log_2(g+1) + \log_2(q+1) < \nu/c$ transposes at size d.

Proof outline. The proof is in Appendix D. Both schedules read rows 1 and 2 only while loading the input, at the $2db$ load steps that read the positions $db, \dots, 3db - 1$. Call a step tainted if it is reachable from these reads in the cell network, with three exceptions: taint never passes along a state edge; it passes along a wait only within one step (two steps on Cascade 4) of the head's arrival on the cell; and a set C_0 of cell edges is cut, namely the tape-1 edges that leave tainted steps of the transfer at one level ρ and next visit their cell after it. Every edge from a tainted step to an untainted step then lies in C_0, or leaves the last step of a maximal interval of tainted steps, a *run* (Lemma D.2). So the cut costs $|C_0|$ plus a bounded number of edges per run, and the tainted steps of $u_0(Q^*)$ are the outputs given up.

Each fold of these schedules starts with a transfer of half of its source, in which tape 1 moves at half speed while tape 0 crosses the first two thirds of that half. Rows 1 and 2 stay in that part at every level, so their window halves at each level: at level ρ it has at most $db/2^{\rho-1} + O(1)$ cells, and the transfer squeezes it into $db/2^{\rho} + O(1)$ cells of tape 1. Cutting the edges out of these cells leaves every deeper level untainted. Below ρ, the backward stage spreads the window again, by a bounded factor per level. At the level we choose it has $O(6^{\rho})$ runs on Cascade 3 and $O(11^{\rho})$ on Cascade 4, and the final sweep gives up $O(6^{\rho}b)$, respectively $O(11^{\rho}b)$, outputs. It also gives up the $4b$ outputs of the entries $(1,1)$, $(1,2)$, $(2,1)$ and $(2,2)$, at whose append steps it reads the loaded copy of rows 1 and 2. Choosing 2^{ρ} near ν makes the cut and the outputs given up both $o(db)$. For (a), the tails of the cut edges together with the roots given up separate in the dependency DAG; for (b), Lemma 10.7 applies to the at least db outputs that the cut separates. □

Remark 10.9 (Other entry widths)**.** The geometric part of the proof holds for every $b \geq 1$ once $d \geq 32$. Choosing 2^{ρ} near $d^{1/5}$ (Cascade 3) or $d^{1/6}$ (Cascade 4) instead, the count of Lemma 10.7 excludes the two traces, for $d \times d$ matrices with b-bit entries and any $b \geq 1$, once $\log_2(g+1)+\log_2(q+1)$ is below $d^{1/5}/c$, respectively $d^{1/6}/c$. In particular it excludes them for matrices of bits.

Remark 10.10 (Rows 0 and 1)**.** Lemma 7.1 counts every step at which the input head is in rows A_0. Both schedules keep the input head on bit 0 throughout their initial positioning, a linear number of steps whose writes later steps revisit, so with $A_0 = \{0, 1\}$ these steps join $\mathsf{In}(A_0)$. The choice of rows in Lemma 7.1 is free, and rows 1 and 2 avoid them.

The two exclusions of Cascades 3 and 4 differ in scope. The memo [16] covers all five cascades, and it excludes every machine with $\lceil \log_2(q+1) \rceil < \delta L$ that is correct on more than a $2^{\lceil \log_2(q+1) \rceil - \delta L}$ fraction of the inputs; but it needs a fixed work alphabet, and its δ and d_0 depend on the alphabet and are obtained by contradiction. Theorem 10.8 covers two cascades and machines that transpose every input, but it allows alphabets and controllers of up to about ν bits, with absolute constants.

It also names the mechanism that fails. The half-speed transfer squeezes the data of rows 1 and 2 into half as many cells at each level. On Cascade 3, after about $\frac{1}{5}\log_2(db)$ levels these data pass through about $(db)^{4/5}$ cells, where the backward stage is still cheap to cut. Cascades 1 and 2 use the same transfer, but before their cascade a shuffle, in which both work heads read and write at every step, spreads each row over several distant intervals of the tape, so that the data of a pair of rows is no longer confined to the squeezed part. We know no cut that excludes Cascades 1, 2 or 5.

11 Open problems

The results of this paper confine an oblivious linear-time transposer to macroscopic queue-like reuse with linear expansion, on one leg of a bounded skeleton, with two long-range heads. The following problems would take the next step; the first three remain open on the side of impossibility, while the fourth, as posed for fixed work alphabets, has since been answered in the negative.

Problem 11.1 (Pass bounds across scales)**.** *For a run and $1 \le j \le \log_2 L$ let r_j be the total number of legs of the skeletons of all its heads, the input head included, at scale $\lfloor 2^{-j} L \rfloor$; coarse pass counting (Theorem 10.3) constrains the input head's motion as well as the work heads'. A linear-time run has $r_j \le (k+1)(4C\,2^j + 1)$, by the argument of Lemma 6.3, and pass counting at a single scale requires only $r_j \gtrsim \frac{j}{2} \log_{k+1} 2$ (Corollary 10.4 and the paragraph after it). Is there, for every k, C and w, a function f with $f(d) \to \infty$ such that no oblivious machine with k work tapes and value width at most w whose runs satisfy $r_j \le Cj$ for all j transposes at size d in time less than $f(d)L$? (The value width must be bounded: a machine with enough states can read the whole input of one size into its control and transpose it in one pass.) More generally, find a coding-proof inequality that involves all the r_j at once and fails for every linear-time run.*

The schedules with $r_j = O(j)$ include, as far as we can tell, the cascades of Section 10.3. A positive answer would exclude every transposer whose motion looks like a cascade at all scales. A negative answer would give machines of this class with bounded normalized running time along an unbounded sequence of dimensions (since the class is finite, one machine along a subsequence); turning this into a transposer of that shape that is correct at every size and faster than $O(L \log d)$ would require a further argument. The interval forests of Theorem P do not obviously help: in our tests (numerically, at $d \le 128$), the smallest charge of such forests over all the group widths we tried was at least L on every transposer, as the cut lemma of the first paper requires, and at least $1.6L$ on the three cascades we tested—about $2L$, $2.5L$ and $4L$ on the first, third and fourth cascades at the dyadic scales with $b = \lceil \log_2 d \rceil$, and $1.63L$ and $1.64L$ on the first and third cascades ($4L$ on the fourth) with scales down to single cells ($d = 64$, $b = 1$)—whereas an exclusion needs a charge below L; localizing the forests to windows in which tapes overlap at two scales did not bring any charge below L.

Problem 11.2 (From the simulator back to the machine)**.** *Let M transpose in linear time, and let N_s be its bit simulator. Does the conclusion of Theorem 5.5 hold in M's own dependency DAG, on a set of traces followed by all but an exponentially small fraction of the inputs? A positive answer would remove the word "oblivious" from Theorem X. The question is whether a macroscopic crossing in the dependency DAG of N_s, which rewrites windows of $3s$ cells in each round, forces a macroscopic crossing of the heads of M.*

Problem 11.3 (Constants and the number of tapes)**.** *The constants of Section 5.1 are polynomial in C, $1/\alpha$ and w_{val}, but exponential in k, through the chain counts $m_0(k)$ and $g(k)$. Does Theorem 5.5 hold with a threshold and an expansion constant that are polynomial in $1/k$ as well? The proof must handle 2^k compatible phase-exit chains on some configurations (*`card_cutValidStepsAux_unitValidTapes_eq_pow_two`*), so this needs a different argument. Numerically, the expansion ratio was 1 at every tested threshold up to $L/2$ (Remark 5.8), far above the proved constants, and nothing we know rules out an expansion constant of order 1 at thresholds of order $1/C$.*

Problem 11.4 (The cascades)**.** *For each of the five cascade schedules of Section 10.3, decide whether some machine with that oblivious trace transposes.* Update: *answered negatively for all five schedules, for every fixed work alphabet at all large dyadic d, at $b = \max(1, \lceil \log_2 d \rceil)$ and at $b = 1$, by the companion memo [16, Theorem 1.1], which verifies on each complete schedule the hypotheses of the regular-recursion obstruction of the second paper [15, Theorem 7.5]. The memo's bound tolerates controllers of $o(L)$ bits, correctness on only part of the inputs, and public initial work contents [16, Proposition 4.3]; for the fifth schedule at $b = \max(1, \lceil \log_2 d \rceil)$ it is formalized in Lean, and the other cases have written proofs. For the third and fourth schedules, condition (iv) of the specification already fails: an explicit separator on rows 1 and 2 violates Lemma 7.1, and a*

count on single cells also allows work alphabets that grow with d (Theorem 10.8). We know no cut that excludes the first, second or fifth schedule.

A linear-time transposer, if one exists, must solve a coding problem on its own schedule: codes of rate approaching one on the cone relations of macroscopic queue-like reuse [6, Corollary O]. The fifteenth paper shows that binary storage codes of rate approaching one exist on suitable separated relations [14]. A construction would need a single uniform tape machine that computes and uses such codes on the relations of a schedule meeting conditions (i)–(iv) and (vi) of Section 10, with all computation and head movement charged, and emits the transpose in the required order; exhibiting storage codes on those relations would not suffice by itself. In the other direction, showing that the cone relations of every such schedule, after a cheap genie, have a fixed rate deficit would exclude oblivious linear-time transposers (Theorem 3.4; [6, Corollary O]); along this route, deciding (T) itself would follow from the native form of Theorem 9.4, machine by machine: a fixed deficit on the cone relations, with arbitrary roots, of the runs of each machine's simulators.

A Catalogue of excluded schedule families

This appendix lists the explicit oblivious linear-time schedule families excluded by the results of this paper. A schedule family prescribes, for each size d in its domain, the motion of the input head and of the work heads and the append steps; a family is *excluded* if for every $C \geq 1$ and every value width w there is d_0 such that, at every size $d \geq d_0$ in its domain, no machine of value width at most w whose trace is the prescribed one transposes within $C(L+1)$ steps.

Status. For rows 1–20 the excluding theorem is formal, and that the schedule belongs to the class is checked by inspection of the schedule. Corollary 4.2 and Theorem 9.7 exclude size by size, with thresholds that depend only on the bounds for the number of tapes and the value width (uniform over all machines within those bounds), so the rows they cover are excluded in the sense above. Theorems 9.2 and 9.3 speak of one machine at all large sizes: for the rows that only they cover (rows 3–7 and 12) they exclude the machines that follow the schedule at all large sizes of its domain, not, size by size, every machine that follows it at one size. For the schedules defined only for even d or for powers of two (rows 3, 5 and 6) they are applied along those sizes, which their formal class predicates, quantified over all large d, do not literally cover. For rows 21–38 the certificate is built from formal lemmas of Section 3, but its verification on the schedule is a written argument; each is stated below as a proof sketch only. *None of these exclusions is a numbered result of this paper.*

Crossings (numerical). The last column records whether Theorem 5.5 alone could exclude the family. "Yes" means that the schedule's work heads stay within $O(b)$ cells, so it has no crossing revisits of linear excursion at all. The other entries are numerical, from exact computations at $d = 64$, $b = 1$ (row 16: $d = 16$, $b = 4$): "no" when the minimum set separating a large output set from the crossing steps of revisits of excursion more than $2W$ had at least 0.25 times as many elements as the output set; "small" when that ratio was at most 0.07, which suggests, as a heuristic, that Theorem 5.5 applies; and "—" when nothing was measured.

	Schedule	k	Excluded by	Crossings
1	Direct copy: output bit ℓ emitted while reading input bit ℓ; work tape idle (computes the identity)	1	Corollary 4.2, Theorem 9.2	yes

	Schedule	k	Excluded by	Crossings
2	Persistent triangles: output (j, i, s) emitted while reading input $(j, -j - i \bmod d, s)$; work head within b cells	1	Corollary 4.2, Theorem 9.3	yes
3	One-pass shear with row rotations, two banks and a row buffer; seven monotone phases (d even)	3	Theorems 9.2 and 9.3	no
4	Ruler schedule: two banks moving one row per episode, a third head making excursions of 2-adic lengths inside one row	3	Theorem 9.3	—
5	Row expander: rows mixed in a two-head deep-nesting workspace of width W, two banks, four monotone phases (d even)	4	Theorem 9.2	—
6	Fan-out cascade fed by fresh input slices at each scale ($d = 2^m$)	2	Theorem 9.3 (suffix form)	—
7	Unit-slope passes: load, reflection, shifted sweep, emit; at most three constant-velocity segments per pass	2	Theorem 9.3	no
8	Bounce: zig-zag input, one tape sweeping $[0, L)$, nested excursions of height at most $L/(4d)$ on the other	2	Theorem 9.3, Corollary 4.2	no
9	Sublinear stack: blocks of $L/(4d)$ outputs pushed and popped on fresh cells while the other tape sweeps	2	Theorem 9.3, Corollary 4.2	—
10–11	Free pseudo-random Dyck walk of length L on one tape while the other sweeps the tape six times (two variants)	2	Theorem 9.3, Corollary 4.2	—
12	One push and one pop of height $2L$ on a stack tape while the other tape bounces	2	Theorem 9.3	—
13–16	One free head with all other excursions $O(L/\log d)$: slot cascades and skewed wavefronts (four variants)	2	Corollary 4.2	small (16)
17–20	No free head, owner load at most 4, family heights in at most 4 scale windows: multi-scale push–pop, two-excursion hub, LIFO Dyck stack, torus shifts	2	Theorem 9.7	—
21–23	One push–pop excursion on a stack tape, with a free tape running a multi-scale cascade or bouncing in a hub (three variants)	2	slab-chunk genie on a two-scale grid	—
24	Multi-scale Dyck stack: d sawtooth waves on a stack tape while the free tape runs a cascade and bounces in a hub ($d = 2^\kappa$)	2	block cut on the hub	no
25–26	Two push–pop excursions on a stack tape, with a mandatory cut at the second excursion (two variants)	2	block cut	—
27	Two-exception inverse-square geometry, swept variant [8] ($d \geq 64$)	10	closed post-region, $O(d^{1/3})$ boundary steps	no
28–29	Two-exception inverse-square geometries [8] ($d \geq 64$)	9, 11	window lemma after two cut steps	small
30	Mirrored dyadic fold cascade	2	window lemma (Lemma 3.9), uncut	no

	Schedule	k	Excluded by	Crossings
31	Unit-slope dyadic cascade ($d = 2^m$)	2	sub-row window, uncut	—
32	Dyadic fold cascade with row blocks of doubling width	2	window lemma, uncut	no
33–34	Two stack excursions with the free tape in a hub of width $L/2^s$ (s fixed) or d	2	block cut; space–time potential	—
35	Daisy-chained stack copies with golden-ratio shifts	2	piecewise-isometric windows	—
36	LIFO fan-out cascades of $\log d$ levels (two variants; $d = 2^\kappa$)	2	laminar cut of $2^{1-p}L + 3$ steps, $p = O(\log w_{\mathrm{val}})$	no
37	Equal-volume tapped cascade with $\log d$ meso-scale stages	2	meso-hub cut of $O(L/\log d)$ steps	no
38	Two-band meso cascade	2	closed post-region, $O(L/\log d)$ boundary steps	—

Rows 1–20. For bounded macro-phases (Theorems 9.2 and 9.3) the crux rate is $1/(4(|B|+1)^{M_{\mathrm{ph}}-1})$ with the bank B and the number of phases read off the schedule: for example, row 3 has banks $\{0,1\}$ and seven phases, row 5 banks $\{0,1\}$ and four phases. Rows 13–16 satisfy the hypotheses of Corollary 4.2 with the free head named and all other excursions $o(L)$; rows 17–20 those of Theorem 9.7 with owner load and window counts read off the schedule. For row 1, Theorem 9.2 gives crux rate $\frac{1}{4}$; the true rate is at most $\frac{1}{2}$, because pairing q with $\mathrm{tpos}(q)$ gives an exclusive-or code of $L/2$ bits.

Rows 21–38. Proof sketch only. Rows 21–23 are excluded by the frozen-trace count used in the proof of Theorem 4.1: on the pop leg of the stack, revisits that span the peak are order-reversing, and a two-scale slab grid with a greedy slicing gives a trapped chunk whose genie costs less than its output surplus. Rows 24–38 are excluded through Theorem 3.4 by an explicit certificate. Rows 24–26 and 33: the cut at a mandatory part of the stack motion makes every relevant output carry at most a blocks of potential sources and every input at most e blocks, and Lemma 3.8 gives an acyclic set of density $1/\binom{a+e}{a}$, uniformly in the depth. Rows 27–30 and 32: the window lemma (Lemmas 3.7 and 3.9) with potentials $\varphi(q) = \mathrm{tpos}(q)$ and $\psi(p) = L - 1 - p$ or a row-local variant, possibly after cutting a sublinear boundary. The geometries of rows 27–29 have hop counts of order L/i^2 in their i-th family; the gap-area memo of the ninth paper shows that its frozen-gap-area certificate cannot exclude this case [8], whereas the certificates here use cut steps and a window potential instead. Row 34 uses a space–time potential, row 35 a bound for piecewise-isometric motion with boundedly many segments, row 36 a bounded-level laminar cut, row 37 a meso-scale hub cut, and row 38 the formal closed-post-region criterion (`isAcyclicIn_ballAvoidSystem_of_closedPostRegion`).

As a numerical check, the seven explicit $\omega(L)$ transposers in `experiments/` (six radix and butterfly transposers of time $\Theta(L \log d)$ and a one-excursion stack transposer of time $\Theta(d^{1/3}L)$, all verified on the unit-vector basis) were tested on the numerical quantities of this paper: all seven had crossing separator ratio 1 at threshold $2W$ ($d = 32$, $b = 1$; the $\Theta(d^{1/3}L)$ transposer also lies outside the linear-time hypothesis of Theorem 5.5 and Corollary 4.2), and every tested coarse-pass slope product and closed interval-forest charge met the proved lower bounds.

B Index of Lean declarations

Every declaration of this paper's library cited in the text, grouped by the Lean module that contains it, relative to the namespace `NativeRelations`. All of them depend only on the axioms `propext`, `Classical.choice` and `Quot.sound` (Appendix C). The declarations of earlier and shared libraries named in the text are checked both in those libraries and in `PaperAxioms.lean`: `DirectedRecovery.IsIndependent` in the fifth paper's library; `HalfRate.IsAcyclicIn` in the seventh paper's library; `BoundedNesting.IsHop` in the eighth paper's library [5]; `MergeDepth.DepStep`, `MergeDepth.IsDepPath`, `MergeDepth.TransposesInput`, `MergeDepth.blockMachine_isStorageCode_fibre`, `MergeDepth.blockWidth`, `MergeDepth.entropy_comp_le` in the eleventh paper's library [10]; `BoundedStackDepth.coneSystem`, `BoundedStackDepth.depCone`, `BoundedStackDepth.exists_indexCode`, `BoundedStackDepth.genie_cost`, `BoundedStackDepth.genie_of_trace` in the fourteenth paper's library [6]; `TapeStorageRates.aligned_low_trace_dimensions`, `TapeStorageRates.skeletonOfTrace_eq` in the fifteenth paper's library [14]; and `LeanMachine.BlockRanking.choose_mul_card_goodPerms_eq_factorial` in the shared `LeanMachine` library.

`NativeRelations.AdaptiveSuffixJitterBM` `adaptiveJitterBM_excludes_transposesAt`, `adaptiveJitterBM_not_linearTimeTransposer`, `adaptiveSuffixJitterBM_excludes_transposesAt`, `adaptiveSuffixJitterBM_not_linearTimeTransposer`, `nativeGapBound_of_adaptiveJitterBM`, `native_machine_preserves_suffixJitterBM`.

`NativeRelations.CoarseGroupedPass` `CoarseWorkspacePartition`, `adaptive_singleScale_coarseWorkspace_closed_lb`, `boundedCoarsePasses_adaptive_polyTime_excludes`, `boundedCoarsePasses_oblivious_excludes`, `boundedCoarsePasses_sublinear_adaptive_polyTime_excludes`, `boundedCoarsePasses_sublinear_oblivious_excludes`, `groupedBound_not_excluded_above_half`, `groupedBound_thetaTwo_not_excluded_above_half`, `oblEntryBits_zero_univ_eq`, `oblivious_singleScale_coarseWorkspace_closed_lb`, `perfectUnshuffleSlopeProduct_tendsto_half`, `subcriticalDimPhaseProfile_oblivious_excludes`, `subcriticalPhaseProfile_adaptive_polyTime_excludes`, `subcriticalPhaseProfile_oblivious_excludes`, `theoremP_coarseWorkspace_grouped_on_bits`, `theoremPprime_coarseWorkspace_grouped_on_bits`, `unshuffleStep_iter_transpose`.

`NativeRelations.CrossingBridge` `appendTime_tposOf_injective`, `card_selfCollisions_singleFamily_le`, `crossingLowerEtaSteps`, `isBallPathSeparator_of_isMengerVertexCut`.

`NativeRelations.DichromaticCrux` `acyclicIn_outside_injOn`, `indexCode_bits_ge_of_isAcyclicIn`, `machineCruxAt_of_isAcyclicIn`, `machineCrux_of_isAcyclicIn`, `oblivious_excludes_of_isAcyclicIn`, `storageCode_card_le_of_isAcyclicIn`.

`NativeRelations.EnvelopeCrux` `HasHopExcursionGTAt`, `HopsCross`, `IsBallPathSeparator`, `MachineCutCruxAt`, `SubpopCutCruxAt`, `acyclicIn_of_lt_on_nbhd`, `ballAvoidSystem`, `coneSystem_subset_ballAvoidSystem`, `crossingCut_nonCrossing_strictAnti`, `cutSet_isAcyclicIn`, `depPath_oneSidedStepInvariant_with_sink`, `exists_isAcyclicIn_card_ge_of_blockCut`, `exists_isAcyclicIn_card_ge_of_uniformBlockCut`, `exists_isAcyclicIn_of_oneSidedEnvelope_maxDeg`, `exists_isAcyclicIn_of_shiftBlockCutFamily_gen`, `exists_isAcyclicIn_of_window_maxDeg`,

exists_two_cuts_covering_nonCrossingLargeHops,
isAcyclicIn_ballAvoidSystem_of_closedPostRegion,
isAcyclicIn_ballAvoid_window_of_constStepInvariant,
isAcyclicIn_ballAvoid_window_of_stepInvariant, machineCutCruxAt_of_ballAvoid_isAcyclicIn,
machineCutCruxAt_zero_iff, machine_cutcrux_excludes,
obliviousLinTime_ballSeparator_card_ge, obliviousLinTime_separator_card_ge_of_isAcyclicIn,
oneSidedEnvelope_isAcyclicIn, subpopCutCruxAt_one_iff, subpop_cutcrux_excludes,
upperEnvelope_isAcyclicIn, window_isAcyclicIn_of_noFarArcs.

NativeRelations.JitterMacroBank CoarseMonoOnInterval, hop_upper_injOn.

NativeRelations.MachineCrux MachineCruxAt, machine_crux_excludes.

NativeRelations.MacroBankCrux BoundedMacroBank, WeakMonoOnInterval.

NativeRelations.MacroscopicSkeleton CrossingCapableSkeletonTuple,
IsScaleFreeCrossingCapable, card_boundedSkeletonTuple,
card_crossingCapableSkeletonTuple_lt, card_tapeCrossingLowers_ge_of_macroOverlap,
crossing_isHop_opposite_polarity, epochGrid_confines_and_exact,
exists_crossingZigzag_of_crossingHopsGT2H, exists_infinite_fiber_of_mapsTo_finset,
isCrossingCapableSkeleton_of_tapeCrossingLowers_nonempty,
isScaleFreeCrossingCapable_of_two_mul_le, isWalk_ivt, legCrossingLowers,
legOfEpoch_le_macroTurns, legOfEpoch_lt_of_isHop_gt2H,
linearTimeTrace_reduces_to_crossingCapableSkeleton_and_singleLeg, macroTurns, milestones,
tapeCrossingLowers_nonempty_of_isDetourProofSkeleton, tapeSkeleton,
tapeSkeleton_length_mul_le, traceSkeletonTuple, traceSkeletonTuple_mem_boundedSkeletonTuple,
two_le_macroTurns_of_isScaleFreeCrossingCapable.

NativeRelations.MacroscopicThreshold bridge_exists_isAcyclicIn_crossingLowerH,
clauseE1H_uncapped, crossingLowerEtaSteps_anti_H,
depPath_nonCrossing_dichotomy_of_isBallPathSeparator_crossingLowerH,
linearTimeTrace_crossingLowerGT2H_ballSep_dichotomy,
linearTimeTrace_crossingLowerGT2H_menger_dichotomy,
obliviousLinTime_macroscopicCrossingLowers_menger_lb_genie, timePullback_blockCut_mais.

NativeRelations.NativeGapCriterion NativeGapBound, fSecond_native_gap,
nativeGapBound_half_iff, native_gap_excludes_linear, native_gap_excludes_transposesAt,
native_gap_exclusion_of_trace_support, native_gap_exists_exclude_linear,
native_gap_tests_exclude_linear, native_machine_k.

NativeRelations.OneFreeHeadSmallExcursion IsMengerVertexCut,
atMostOneWorkTape_lowAdaptivity_excludes, atMostOneWorkTape_oblivious_excludes,
card_smallCrossingLowers_erase_le, depStep_trapped_in_chunk,
exists_offset_slabTransitions_le, exists_slab_chunk_dichotomy,
frozenTrace_subpop_deficit_le, hopExcursionLEOn_of_famd_le,
offCarrier_longRevisits_card_ge, offCarrier_vertexExpansion_of_linearTimeTransposer,
offCarrier_vertexExpansion_of_obliviousLinTime, oneFreeHead_smallExcursion_excludes,
oneFreeHead_sublinearExcursion_excludes, oneFreeHead_sublinearExcursion_oblivious_excludes,
oneFreeHead_sublinearFamHeight_excludes, outIn_card_le_two_indep_add_six_runs,
slabAutonomyEps, slabDimThreshold, slabSmallExcursion, slabWallCap,
twoHeads_manyLongRevisits_of_linearTimeTransposer,
twoHeads_manyLongRevisits_of_oblivious_linearTimeTransposer,

twoMacroscopicHeads_of_linearTimeTransposer,
twoMacroscopicHeads_of_oblivious_linearTimeTransposer.

NativeRelations.ScaleProfile disjointWave_mengerExpansion_counting_le,
isMengerVertexCut_biUnion, obliviousLinTime_expandingWaves_le.

NativeRelations.SimulatorBridge card_correctInputs_le_of_coneDeficit,
correctSubset_isAcyclicIn_coneSystem_deficit_le,
correctSubset_isAcyclicIn_coneSystem_deficit_le_logb,
frozenTrace_isAcyclicIn_ballAvoid_separator_card_ge,
frozenTrace_isAcyclicIn_coneSystem_deficit_le.

NativeRelations.SuffixMassCrux boundedMacroBank_excludes, boundedMacroBank_has_crux,
boundedSuffixMacroBank_of_boundedMacroBank, getLastD_le_of_mem_depPath,
suffixMacroBank_excludes, suffixMacroBank_has_crux, suffixMacroBank_independent_exists.

NativeRelations.TightConstants
adaptive_simulator_heavyTrace_macroscopicExpansion_tight, cE_frozen_eta_tight,
cE_menger_tight, cE_menger_tight_le_one_thirtieth,
card_cutValidStepsAux_unitValidTapes_eq_pow_two, card_microHopStepGuard_tight, chainCapAt,
chainCapAt_le_pow_three, coneSystem_eq_ballAvoidSystem_at_T,
correctSubset_crossingLowerGT2H_menger_lb_tight, exists_commonRefinement_cell_core,
exists_singleCell_guard_coneSystem_isIndependent_crossingLowerGT2H,
exists_singleCell_guard_coneSystem_isIndependent_crossingLowerH,
frozenTrace_crossingLowerGT2H_menger_lb_tight,
frozenTrace_macroscopicCrossingLowers_menger_lb_tight,
frozenTrace_reduces_to_crossingCapableSkeleton_and_singleLeg_tight,
johnsonSetPair_isAcyclicIn_card_le_one,
linearTimeTrace_crossingLowerGT2H_cone_menger_dichotomy_tight,
linearTimeTrace_crossingLowerGT2H_cone_menger_implication_tight,
linearTimeTrace_crossingLowerGT2H_menger_lb_of_coneDeficit_tight, nodeCapAt,
nodeCapAt_le_succ_mul_chainCapAt, obliviousLinTime_coneSystem_isAcyclicIn_card_le,
obliviousLinTime_crossingCapableSkeleton_and_singleLeg_of_le_one_tight,
obliviousLinTime_crossingLowerGT2H_menger_lb_tight,
obliviousLinTime_crossingLowerGT2W_menger_lb_tight,
obliviousLinTime_infinite_pinned_skeleton_and_leg_tight,
obliviousLinTime_macroscopicCrossingLowers_card_and_menger_tight,
obliviousLinTime_macroscopicCrossingLowers_menger_lb_tight,
obliviousLinTime_mixGT2W_menger_lb_tight,
obliviousLinTime_reduces_to_crossingCapableSkeleton_and_singleLeg_tight,
obliviousOn_correctSubset_macroscopicCrossingLowers_menger_lb_tight, sharpCellGuardThresh,
sharpCellGuardThresh_le_pow_three, sharpChainCount_le_pow_three, sharpCounts_small_values,
sharpFamCount, simulator_heavyTrace_expansion_and_skeleton_tight, thetaMacro_tight,
tightBridgeRefCells, tightBridgeRho0, tightBridge_d0, tightBtime, tightTimePullbackRhoMin,
tightTimePullback_d0, timePullback_commonRefinement_mais_tight.

NativeRelations.TranspositionCut bichromaticRect, card_bichromaticRect,
frozenTrace_bichromaticColoring_wave_cut_ge, frozenTrace_bichromatic_tposOf_cut_ge,
inSlabReadSteps, isAcyclicIn_coneSystem_uncutRootedBichromaticRect,
isMengerVertexCut_of_bichromaticColoring, oblivious_bichromatic_tposOf_cut_ge.

NativeRelations.ZeroFreeHeadQWindow zeroFreeHead_entries_qWindow_excludes,
zeroFreeHead_qWindow_excludes, zeroFreeHead_qWindow_oblivious_excludes.

C Building and verifying the formalization

The Lean project is the directory `lean/` next to this paper. It uses Lean and Mathlib, both at version v4.34.1 (`lean-toolchain`, `lakefile.toml`, `lake-manifest.json`), and requires the library of the fifteenth paper [14] through a relative path. Through it, and the libraries it requires in turn, the build also uses the libraries of the first, second, fourth, fifth, seventh, eighth, ninth, tenth, eleventh, thirteenth and fourteenth papers of the series, two memos in the directories of the fourth and seventh papers, and the shared `lean-machine/` library (including `LeanMachine/BlockRanking.lean` and `LeanMachine/ThresholdRefinement.lean`). Keep the repository's directory layout so that these sibling directories stay in place.

Build. Before the first build, fetch the second paper's pinned external dependency as described in its README (`python3 verification/fetch_openai_metric.py`, run in that paper's directory). With `elan` installed, run in `lean/`:

```
lake exe cache get && lake build
```

The first command downloads the compiled Mathlib cache; the second compiles `NativeRelations` and the parts of the earlier libraries that it uses.

Axioms and index. The file `lean/PaperAxioms.lean`, generated from the LaTeX sources of this paper by `tools/lean_index.py`, runs `#print axioms` on every declaration cited in this paper (Appendix B), both in `NativeRelations` and in the earlier and shared libraries:

```
lake env lean PaperAxioms.lean
```

Every declaration depends at most on `propext`, `Classical.choice` and `Quot.sound`. No Lean file of this library uses `sorry`, `admit` or `native_decide`, declares an `axiom`, bypasses the kernel via `unsafe`, `implemented_by` or `extern`, or raises `maxHeartbeats`. Running `python3 tools/lean_index.py` from the paper's root directory regenerates both `paper/sections/leanindex.tex` and `lean/PaperAxioms.lean` (`-index-only` writes only the index; `-check` verifies that every cited name resolves and that both generated files are current).

Where things are. The library `lean/NativeRelations/` has eighteen modules, all imported by the root module `lean/NativeRelations.lean`:

- *Certificates and acyclic-set tools* (Section 3): `MachineCrux.lean`, `DichromaticCrux.lean` and `EnvelopeCrux.lean`.
- *Two long-range heads* (Section 4): `OneFreeHeadSmallExcursion.lean`.
- *Macroscopic crossings* (Section 5): `CrossingBridge.lean` and `MacroscopicThreshold.lean` (crossing sets, the first forms of the pullback, bridge, dichotomy and crossing theorem using uniform block cuts and genie depth), and `TightConstants.lean` (the common-refinement time pullback, the single-cell guarded bridge, the constants of Section 5.1, and the main forms of Lemma 3.5(b) and Theorems D, X, S and E).
- *Skeletons and expanding waves* (Section 6 and Remark 8.3): `MacroscopicSkeleton.lean` and `ScaleProfile.lean`.
- *Rows to columns* (Section 7): `TranspositionCut.lean`.

- *Adaptive machines* (Section 8): `SimulatorBridge.lean` (frozen-trace deficit bounds, used with `TightConstants.lean`).
- *Class exclusions* (Section 9): `MacroBankCrux.lean`, `SuffixMassCrux.lean`, `JitterMacroBank.lean`, `AdaptiveSuffixJitterBM.lean`, `NativeGapCriterion.lean` and `ZeroFreeHeadQWindow.lean`.
- *Coarse pass counting* (Section 10): `CoarseGroupedPass.lean`.

What is not formalized. Complete written proofs in the text, not formalized in Lean, are given for Corollary 7.3, Proposition 10.2, Lemma 10.7 and Theorem 10.8 (with Appendix D). Proof sketches only are given for Remark 8.2, for the perfect-unshuffle transposer realizing the $\theta = 2$ parameters after Corollary 10.4, and for the exclusions of the individual schedule families in Appendix A (built on the formal class theorems of Section 9 and certificate lemmas of Section 3).

Constants and numerical remarks. The script `tools/print_constants.py` (Python 3 standard library only) reproduces the exact rational values and scientific approximations of the constants in Sections 4, 5.1 and 6. The self-contained Rust crate `experiments/` (standard library only; `cargo run -release -offline -locked - check` in `experiments/`) generates the thirty schedules, exports complete time-indexed traces, builds their exact dependency DAGs, verifies the seven $\omega(L)$ transpose controls on the unit-vector basis and the cell-entry and write caps of the third and fourth cascades (Section 10.3), and reproduces every numerical observation reported in the text: the crossing separator ratios and $E = L$ cycle-packing acyclic bounds of Remark 5.8, the running times, Λ^*-skeleton leg counts and single-leg crossing expansion of Remarks 6.8 and 10.1, the coarse-pass slope products after Corollary 10.5 (and in Section 1), the multi-scale leg counts and the one-scale and two-scale closed interval-forest charges of and after Problem 11.1, and the fourteen numerical "Crossings" entries and $\omega(L)$ control checks of Appendix A. Those observations are labelled as numerical throughout and no proof in the paper or the formal library depends on them.

D Proofs for Cascades 3 and 4

This appendix proves Theorem 10.8 and Remark 10.9. Throughout, $d = 2^k \geq 32$, $b \geq 1$, $L = d^2b$, $m_r = L/2^r$, $\mathrm{sh}_r = \lfloor m_r/3 \rfloor + b$ and $r_{\max} = k - 1$, so that $m_r \geq m_{r_{\max}} = 2db \geq 64$ for $r \leq r_{\max}$. Let $A = [db, 3db)$ be the set of input positions of rows 1 and 2. As A_1 is the set of all rows, $Q^*(\{1,2\}, A_1) = A$, and $\mathsf{In}(\{1,2\})$ is the set of steps at which the input head reads a position of A. The traces are those of the constructors `build_c3_ten_entry_cascade` and `build_c4_no_overwrite_cascade` in `experiments/src/schedules.rs`; the head motions below are read off their loops, and we also checked them on the exported traces for $d \leq 128$. A *region* is an interval of cells of one tape, and the *offset* of a cell in a region is its position minus the first position of the region.

D.1 Taint closures

For a work tape j and a step t, let $\mathrm{prev}_j(t)$ be the last step before t at which head j scans the cell it scans at step t, if there is one; the cell edge into t of tape j is then $\mathrm{prev}_j(t) \to t$. The *wait age* $\omega_j(t)$ is 0 if $t = 0$ or head j scans different cells at steps $t - 1$ and t, and $\omega_j(t-1) + 1$ otherwise.

Definition D.1 (Taint closure)**.** Fix a set C_0 of cell edges and an order $h \in \{1, 2\}$. A step t is *tainted* if it reads a position of A, or if for some tape j the cell edge $p \to t$ with $p = \mathrm{prev}_j(t)$ is

not in C_0, the step p is tainted, and either $p \le t-2$ or $\omega_j(t) \le h$. A *run* is a maximal interval of consecutive tainted steps. The *cut* C consists of C_0 and of the state edge and the waits that leave the last step of each run. A cell of tape j is *live tainted* at step t if its last visit $p < t$ is tainted and the edge $p \to \mathrm{next}_j(p)$ is not in C_0.

A step t with $p = \mathrm{prev}_j(t) = t-1$ is a *short wait* if $\omega_j(t) \le h$. For $h = 1$ this means that head j arrived on the cell at step $t-1$.

Lemma D.2 (The closure is cut by C)**.** Not formalized; a complete proof is given below.
Every edge of the cell network from a tainted step to an untainted step lies in C. Let N_{run} be the number of runs and N_{up} the number of tainted steps of $u_0(A)$. Then:

(a) every path in the cell network from a step that reads a position of A to an untainted step uses an edge of C;

(b) the set Γ of the tails of the edges of C and the tainted steps of $u_0(A)$ separates $u_0(A)$ from $\mathsf{In}(\{1,2\})$ in the dependency DAG;

(c) C has at most $|C_0|+3N_{\mathrm{run}}$ cell edges and at most N_{run} state edges, and $|\Gamma| \le |C_0|+N_{\mathrm{run}}+N_{\mathrm{up}}$.

Proof (not formalized). Let $p \to v$ be an edge with p tainted and v untainted. A state edge has $v = p+1$, so p ends a run and the edge is in C. A cell edge with $v \ge p+2$ is in C_0, since otherwise v would be tainted. A wait of tape j that is not in C_0 has $\omega_j(v) > h$, since otherwise v would be tainted; so p ends a run, and the wait is in C. This proves the first claim, and (a): a path from a tainted step to an untainted one has a first untainted vertex, and the edge into it leaves a tainted vertex.

For (b), take a dependency path from $u_0(v)$, $v \in A$, to a step that reads a position of A. If $u_0(v)$ is tainted it lies in Γ. Otherwise the reversed path is a path in the cell network (Definition 10.6) from a tainted step to an untainted one, and the edge into its first untainted vertex lies in C and leaves a vertex of the path, which is a tail of C. For (c), C_0 has $|C_0|$ edges and tails, and every other edge of C leaves the last step of a run: one state edge, none after the last step of the trace, and at most one wait per work tape; both schedules have three. □

So it suffices to choose C_0 so that $|C_0|$, N_{run} and N_{up} are small.

D.2 Cascade 3

Cascade 3 uses the closure of order $h = 1$. Its regions are as follows. On tape 0, R_r starts at a_r, with $a_1 = L$ and $a_{r+1} = a_r + 2m_r$; for the backward stage, IN_r has size m_r and OUT_r has size $2m_r$, with $\mathrm{IN}_{r-1} = \mathrm{OUT}_r$. On tape 1, the level-$r$ scratch starts at some s_r, and the backward stage uses fresh regions CB_r of size m_r. On tape 2, $Z_r = [z_r, z_r + m_r)$ with $z_r = L - 2m_r$. Put $\Delta = \lfloor \lfloor L/3 \rfloor / 2 \rfloor - 1$. The phases that matter are the following.

phase	head motion
load, step $u < L$	input head on u; tapes 0 and 1 on cell u; tape 2 on cell $L + \lceil u/2 \rceil$ while $u \le \lfloor L/3 \rfloor$, then at speed 1
rev	tape 0 down $[L, 2L)$ and tape 2 down to cell L, tape-2 cell x aligned with tape-0 cell $x + \Delta$; then tape 2 parks on cell L; tape 1 idle
fwd r.transfer, step s	tape 0 on R_r offset $\min(s, m_r - 1)$; tape 1 on scratch offset $\lceil s/2 \rceil$ for $s < 2\lfloor m_r/3 \rfloor$, then at speed 1; tape 2 idle

phase	head motion
fwd r.move2	tape 0 up to R_r offset $2m_r - \mathrm{sh}_r$, then idle; tape 1 down the scratch from offset $m_r - 1$ to 0
fwd r.copyS	tapes $0, 1, 2$ on R_r offset $2m_r - \mathrm{sh}_r + u$, scratch offset u, Z_r offset u, for $0 \le u < \mathrm{sh}_r$
fwd r.move3, copyR, move4	tape 0 down to R_r offset m_r; then R_r offset $m_r + u$, scratch offset $\mathrm{sh}_r + u$, Z_r offset $\mathrm{sh}_r + u$; then tape 0 up to R_{r+1} offset $m_r - 1$
fwd r.back, move5	tape 0 down R_{r+1} and tape 2 down Z_r, offset aligned with offset; then tape 2 up through Z_r to z_{r+1}
comb r.move	tape 0 up IN_r from offset 0; tape 1 up from CB_{r+1} offset sh_{r+1} (from the end of the scratch if $r = r_{\max}$) to CB_r offset $m_r - 1$; IN_r offset s is aligned with CB_{r+1} offset $\mathrm{sh}_{r+1} + s$ while $s < m_{r+1} - \mathrm{sh}_{r+1}$, and with CB_r offset $s - c_r$, where $c_r = m_{r+1} + 2 - \mathrm{sh}_{r+1}$, once $s \ge c_r$; then tape 2 moves to $z_r + m_r - 1$: one step from z_{r+1} if $r < r_{\max}$, and up through the offsets $0, \ldots, m_r - 2$ of $Z_{r_{\max}}$ if $r = r_{\max}$
comb r.blocks	2^r blocks of $m_r/2^r$ offsets, from the top: all three heads down together over IN_r, CB_r and Z_r at equal offsets; then tape 1 up and down across the block while tapes 0 and 2 wait
comb r.move2	tape 0 up through IN_r to OUT_r; tapes 1 and 2 idle
comb r.dbl	CB_r offset z aligned with OUT_r offsets $2z$ and $2z + 1$
comb r.move3, shift, move4, rem	tape 1 alone down to offset $\mathrm{sh}_r - 1$; then OUT_r offset y aligned with CB_r offset $(\lfloor y/2 \rfloor + \mathrm{sh}_r) \bmod m_r$, both moving down, with tape 1 alone moving up between the two parts
rewind	tape 1 down from CB_1 offset sh_1 to cell 0; tapes 0 and 2 idle
emit, step u	tape 0 on OUT_1 offset u; tape 1 on cell u; tape 2 on cell $\min(u, \sum_r m_r - 1)$; output bit u appended

The positioning phases omitted from the table (the initial positioning, the moves into and out of rev, and the move that starts each forward level) read no live tainted cell. We use three properties of the trace.

(i) Every phase is a *segment*, or a few of them: an interval of steps in which every head is either idle throughout, or monotone and waiting at most one step at a time. To get segments, cut rev, transfer, move2, comb move and emit where a head stops early, cut each block into its sweeps, and make the last step of copyS, at which all three heads wait, a segment of its own.

(ii) After the current level, the regions of tape 0 below the current one, the scratch regions, and $\mathrm{CB}_{r'}$ for $r' > r + 1$ are not visited again, except by tape 1 in the rewind. Z_r is visited after move5 only in comb r.blocks and in the emit. The exceptions are single idle cells: offset 0 of Z_r while comb r.move2 to rem run and again in comb $(r-1)$.move, and offset 0 of Z_1 and of OUT_1 during the rewind. Each adds at most one run of at most two steps.

(iii) In every segment at most one head moves at half speed (in load, transfer, dbl, shift and rem).

Lemma D.3 (Segments)**.** Not formalized; a complete proof is given below.

In a segment of Cascade 3, every tainted step is (1) a step at which a head that is not idle scans a live tainted cell, (2) one of the first two steps of an idle head on a live tainted cell, (3) a step that reads a position of A, or (4) the step right after one of these, or the step after that when the first is a short wait of an idle head. Hence the number of runs in a segment is at most the number of

maximal intervals of live tainted cells that it reads, over all tapes, plus the number of runs of reads of A; and each run writes one interval of cells on each tape whose head moves.

Proof (not formalized). A monotone head stays on an interval of cells during an interval of steps. Its first step on a cell is a revisit, or at the start of the segment a wait of either kind, and its second step there, if any, is a short wait. A short wait at the first step t_0 of the segment from a tainted step $t_0 - 1$ reads a cell that is live tainted at t_0, so t_0 is of kind (1). An idle head's first two steps are a revisit and a short wait, and the rest are waits that are not short. Cells written inside the segment are revisited inside it only by waits. So a tainted step of none of the kinds (1)–(3) is tainted through a short wait from the tainted step before it. By (iii), all short waits of moving heads in the segment belong to one head j. After its wait at step t, head j moves, so step $t+1$ is no wait of j, and no wait of any other moving head either; so the extension stops after one step. An idle head's short wait at t_0+1 can hand over to a short wait of j at t_0+2, so the extension stops after at most two steps. It is adjacent to its run, so it adds no run and extends each written interval by at most two cells. □

The cut is placed at a level ρ with $1 \le \rho \le r_{\max}$: C_0 is the set of tape-1 edges that leave tainted steps of fwd ρ.transfer and whose next visit lies after the transfer. The waits of tape 1 inside the transfer are not cut. This is well defined: no edge of C_0 ends at or before the end of the transfer, so the taint up to that point does not depend on C_0.

Lemma D.4 (Forward windows)**.** Not formalized; a complete proof is given below.
For $1 \le r \le \rho$ let W_r be the set of offsets of R_r that are live tainted when fwd r.transfer starts. Then $W_1 = \{\lceil u/2 \rceil + \Delta : u \in A\}$ and $W_{r+1} = \{\lceil s/2 \rceil : s \in W_r\}$ for $r < \rho$. Each W_r is an interval with

$$|W_r| \le \frac{db}{2^{r-1}} + 2, \qquad \frac{m_r}{3} - 2 < \min W_r \le \max W_r \le \frac{m_r}{3} + \frac{3db}{2^r} + 1.$$

Moreover $|C_0| \le db/2^\rho + 2$, and the forward stage up to the end of fwd ρ.transfer has $O(\rho)$ runs.

Proof (not formalized). The bounds hold for W_1: the interval $\{\lceil u/2 \rceil : u \in A\}$ has at most $db+1$ elements, from $\lceil db/2 \rceil$ to $\lceil (3db-1)/2 \rceil$, and $m_1/3 - 2 \le \Delta \le m_1/3 - 1$. They pass from W_r to W_{r+1}, because $\lceil s/2 \rceil \in [s/2, (s+1)/2]$ and the image of an interval of n integers has at most $n/2+1$ elements. Since $3db/2^r = 3m_r/d \le 3m_r/32$ and $m_r \ge 64$,

$$\max W_r \le \tfrac{41}{96} m_r + 1 < 2\lfloor m_r/3 \rfloor, \qquad \max\{\lceil s/2 \rceil : s \in W_r\} \le \tfrac{41}{192} m_r + 1 < \mathrm{sh}_r - 2. \tag{1}$$

Load: the tainted steps are the reads of A and, at most, the step after the last one, a short wait of tape 2. They write tape-0 cells, which are not visited again; the loaded copy of rows 1 and 2 on tape 1, cells $[db, 3db]$, which is live until the rewind; and the tape-2 cells $L + \lceil u/2 \rceil$, $u \in A$, as $3db < \lfloor L/3 \rfloor$. Rev reads these tape-2 cells and writes the tape-0 cells $L + \lceil u/2 \rceil + \Delta$, that is, W_1; tape 1 is idle, so its wait is cut, and the tape-2 cells above L are not visited again.

In fwd r.transfer the steps $s \in W_r$ are tainted, and so is $s+1$ for odd s, by a short wait of tape 1. By (1) tape 1 is in its half-speed part, and the scratch cells it writes are the offsets $S_r = \{\lceil s/2 \rceil : s \in W_r\}$. For $r = \rho$ the edges leaving them are C_0, so $|C_0| \le |S_\rho| \le |W_\rho|/2 + 1$. For $r < \rho$, the scratch window S_r lies in $[0, \mathrm{sh}_r - 2)$. It is read in move2, where tape 0 is already idle because $m_r - 1 - \max S_r > m_r - \mathrm{sh}_r + 1$; in copyS, aligned with Z_r offset u and with R_r offset $2m_r - \mathrm{sh}_r + u$; and next in the rewind. The copy on R_r is read in move3 and move4 and then never again. The copy on Z_r is read in back, which writes the offsets S_r of R_{r+1}, so $W_{r+1} = S_r$, and in move5. CopyR touches only scratch and Z_r offsets at least sh_r and R_r offsets below $2m_r - \mathrm{sh}_r$, none of them tainted. Each of transfer, move2, copyS, move3, move4, back and move5 reads at most two intervals of live tainted cells, so it has at most two runs (Lemma D.3). □

Lemma D.5 (Clean middle)**.** Not formalized; a complete proof is given below.

No step of fwd r for $r > \rho$, of fwd ρ after its transfer, or of comb r for $r \geq \rho$ is tainted.

Proof (not formalized). The cut C_0 blocks the scratch window of level ρ, so the steps of move2 that read it are untainted and rewrite it with untainted contents. Z_ρ and $R_{\rho+1}$ are therefore written by untainted steps, and so is everything built from them. These phases visit only regions of levels at least ρ, by (ii), and the tainted windows of lower levels ($Z_{r'}$, the scratch regions, $\mathrm{CB}_{r'}$ and the loaded copy) lie in regions they do not visit. The one tainted window of level ρ is $W_\rho \subseteq R_\rho$; after the transfer, tape 0 visits only offsets of R_ρ at least $m_\rho - 1$, and $W_\rho \subseteq [0, m_\rho - 1)$. □

For $r < \rho$ let U_r be the set of offsets $y \in [0, m_r)$ at which IN_r, Z_r or CB_r is live tainted when comb r.blocks starts, u_r its number of maximal intervals and $\mu_r = |U_r|$; let $h_r = m_r/2^r$.

Lemma D.6 (Backward stage)**.** Not formalized; a complete proof is given below.

We have $u_{\rho-1} \leq 1$ and $\mu_{\rho-1} \leq db/2^{\rho-1} + 2$, and for $r < \rho - 1$

$$u_r \leq 6u_{r+1} + 8, \qquad \mu_r \leq 10\mu_{r+1} + \frac{db}{2^r} + O(u_{r+1}).$$

Comb r has at most $c\,(u_r + u_{r+1} + \mu_r/h_r + 1)$ runs, for an absolute constant c.

Proof (not formalized). Comb $\rho - 1$ reads $\mathrm{IN}_{\rho-1}$ and the comb region of level ρ, which come from comb ρ and are clean (Lemma D.5); so $U_{\rho-1}$ is the window $S_{\rho-1}$ of $Z_{\rho-1}$, one interval with at most $db/2^{\rho-1} + 2$ cells.

In comb r, blocks moves the three heads in alignment, so afterwards all three regions are tainted exactly on U_r, together with one cell at the bottom of a block, where the waiting heads pass taint into the first two steps of the excursion; the excursions only reread tape-1 cells of the same block. Move2 rereads IN_r, which is not visited afterwards. Dbl taints OUT_r on $2U_r$; shift and rem, through $y \mapsto (\lfloor y/2 \rfloor + \mathrm{sh}_r) \bmod m_r$, add $2(U_r - \mathrm{sh}_r)$ to OUT_r and $U_r + \mathrm{sh}_r$ to CB_r. So $\mathrm{IN}_{r-1} = \mathrm{OUT}_r$ and CB_r each have at most $2u_r + 2$ intervals, of total size at most $4\mu_r$ and $2\mu_r$.

In comb $(r-1)$.move the alignments of the table add to IN_{r-1} the image of the comb region of level r (its offsets at least sh_r, shifted by sh_r), and to CB_{r-1} the image of the offsets of IN_{r-1} at least c_{r-1}. The image of CB_r lands at offsets of IN_{r-1} below $m_r - \mathrm{sh}_r < c_{r-1}$, where tape 1 is still in the old comb region, so the steps that read it write no cell of CB_{r-1}, and CB_{r-1} receives at most the $2u_r + 2$ intervals of OUT_r. Hence $u_{r-1} \leq (2u_r + 2) + (2u_r + 2) + 2 + (2u_r + 2)$ and $\mu_{r-1} \leq 4\mu_r + 2\mu_r + |S_{r-1}| + 4\mu_r$, up to $O(1)$ per interval.

Runs: each segment of comb r has at most as many runs as intervals it reads (Lemma D.3). In blocks an interval is read in every block it meets, and the u_r intervals meet at most $\mu_r/h_r + 2u_r$ blocks, three sweeps each. Comb r.move reads OUT_{r+1} and CB_{r+1}, which gives the term u_{r+1}. □

Solving the recurrences gives $u_r \leq 4 \cdot 6^{\rho-1-r}$. For the sizes, $\kappa_r = \mu_r/(db)$ satisfies $\kappa_{r-1} \leq 10\kappa_r + 2^{1-r}$ up to the terms per interval, so

$$\kappa_r \leq \sum_{j=r}^{\rho-1} 10^{j-r} 2^{1-j} = 2^{1-r} \sum_{j=r}^{\rho-1} 5^{j-r} \leq \tfrac{5}{4}\, 2^{1-r}\, 5^{\rho-r},$$

a bound geometric in r; the terms per interval add $O(\sum_j 10^{j-r} 6^{\rho-j}) = O(10^{\rho-r})$ cells at level r. Hence $\sum_{r<\rho} \mu_r \leq c\,(5^\rho db + 10^\rho)$, $\sum_{r<\rho} \mu_r/h_r = \sum_r \mu_r 4^r/L \leq c\,(5^\rho/d + 40^\rho/L)$, and the backward stage has at most $c\,(6^\rho + 5^\rho/d + 40^\rho/L)$ runs.

Rewind and emit. The rewind moves only tape 1, down from CB_1 offset sh_1. It reads the live tainted intervals of tape 1: the comb regions and the scratch windows of levels below ρ, and the loaded copy $[db, 3db]$; so it has at most $c\,6^\rho + 2\rho + 2$ runs. Afterwards only the loaded copy among them is visited again. The emit's step u reads OUT_1 offset u, tape-1 cell u and tape-2 cell $\min(u, \sum_r m_r - 1)$, and appends output bit u. A tainted step of the emit reads a tainted cell of the loaded copy, at $u \in [db, 3db]$; of OUT_1, which has at most $2u_1 + 2$ intervals and size at most $4\mu_1$; or of Z_r for $r < \rho$, which is tainted on U_r. Among the outputs $u \in [db, 3db]$, those of $u_0(A)$ are the $4b$ bits of the entries $(1,1)$, $(1,2)$, $(2,1)$ and $(2,2)$. Elsewhere, the outputs of $u_0(A)$ form one run of $2b$ positions in each block of db output positions, so an interval of length λ contains at most $2b(\lambda/(db) + 2)$ of them. The other tainted intervals of the emit number at most $c\,6^\rho$ and have total length at most $4\mu_1 + \sum_r \mu_r \le c\,(5^\rho db + 10^\rho)$. Hence

$$N_{\mathrm{up}} \le 4b + 2b\big(c\,5^\rho + c\,10^\rho/(db) + 2c\,6^\rho\big) \le 4b + c'\,6^\rho b \qquad \text{once } (5/3)^\rho \le db.$$

Altogether, for every $1 \le \rho \le r_{\max}$,

$$|C_0| \le \frac{db}{2^\rho} + 2, \qquad N_{\mathrm{run}} \le c''\Big(6^\rho + \frac{5^\rho}{d} + \frac{40^\rho}{L} + \rho\Big), \qquad N_{\mathrm{up}} \le 4b + c'\,6^\rho b. \tag{2}$$

D.3 Cascade 4

Cascade 4 uses the closure of order $h = 2$, because its final sweep moves tapes 0 and 1 once every three steps. All its regions are disjoint, separated by two buffer cells. On tape 2: the frames $Z_r = [z_r, z_r + m_r)$ and $\mathrm{BS}_r = [z_r + m_r + 2, z_r + 3m_r + 2)$ for $r = 1, \dots, r_{\max}$, with $z_1 = 0$ and $z_{r+1} = z_r + 3m_r + 4$, ending at $E_2 = \sum_{r \le r_{\max}} (3m_r + 4) < 3L$; then the seed region $\mathrm{SD} = [E_2 + 2, E_2 + 2 + L)$. On tape 0: the loaded copy $\mathrm{LD}_0 = [0, L)$; $X_1 = [L + 2, 2L + 2)$; for $r = 1, \dots, r_{\max}$, a region $\mathrm{H2R}_r$ of size m_r and the region X_{r+1} of size $m_r = 2m_{r+1}$, the input of level $r + 1$; then, for $r = r_{\max}$ down to 1, regions CM_r of size m_r, $\mathrm{U2R}_r$ of size $2m_r$ and B_{r-1} of size $2m_r$, with $B_{r_{\max}} = X_{r_{\max}+1}$ and B_0 of size L. On tape 1: the loaded copy $\mathrm{LD}_1 = [0, L)$; for $r = 1, \dots, r_{\max}$, regions $\mathrm{H1}_r$ and $\mathrm{H2}_r$ of size m_r; then, for $r = r_{\max}$ down to 1, regions MZ_r of size m_r and $\mathrm{U1}_r$, $\mathrm{U2}_r$ of size $2m_r$. In the table, $Y[x]$ is offset x of region Y.

phase	head motion
initial positioning, load, rev	tape 2 up from 0 to SD[0] while the input head stays on bit 0; load, step $u < L$: input head on u, tapes 0 and 1 on $\mathrm{LD}_0[u]$ and $\mathrm{LD}_1[u]$, tape 2 on $\mathrm{SD}[\lceil u/2 \rceil]$ while $u \le \lfloor L/3 \rfloor$, then at speed 1; tape 0 up to $X_1[L-1]$; rev: $X_1[x + \Delta]$ aligned with $\mathrm{SD}[x]$, then tape 2 parks on SD[0]; tape 2 down to $z_1 = 0$
fwd r.move, transfer	tape 0 idle on $X_r[0]$, tape 1 up to $\mathrm{H1}_r[0]$; transfer, step s: $X_r[\min(s, m_r - 1)]$ aligned with $\mathrm{H1}_r[\lceil s/2 \rceil]$ for $s < 2\lfloor m_r/3 \rfloor$, then at speed 1, ending with one step at which no head moves
fwd r.shift, rev2	tape 0 reads $X_r[m_r], \dots, X_r[2m_r - 1]$, cyclically shifted by sh_r, into the fresh $\mathrm{H2}_r$; then $\mathrm{H2}_r$ is reversed into the fresh $\mathrm{H2R}_r$; tape 2 idle on z_r
fwd r.merge.move, merge, back, move5	merge.move, $m_r + 2$ steps: tape 0 down $\mathrm{H2R}_r$ from offset $m_r - 1$ to 0 ($m_r - 1$ steps, then it parks), tape 1 down from $\mathrm{H2}_r[0]$ to $\mathrm{H1}_r[0]$, so that $\mathrm{H1}_r[x]$ is aligned with $\mathrm{H2R}_r[x - 3]$, tape 2 idle on z_r; merge: $\mathrm{H2R}_r[u]$ and $\mathrm{H1}_r[u]$ into $Z_r[u]$; back: $Z_r[u]$ into $X_{r+1}[u]$, downwards; move5 ($r < r_{\max}$): tape 2 up from z_r to z_{r+1} across Z_r and the clean BS_r
comb r.1a.move, 1a	1a.move: tape 0 up $B_r[0..m_r - 1]$; tape 1 up across $\mathrm{U2}_{r+1}$ (or $\mathrm{H2}_{r_{\max}}$) and the fresh MZ_r; tape 2 down from $\mathrm{BS}_{r+1}[0]$ across Z_{r+1} and BS_r to $Z_r[m_r - 1]$ (up from $Z_{r_{\max}}[0]$ if $r = r_{\max}$); 1a: $B_r[u]$ and $Z_r[u]$ into $\mathrm{MZ}_r[u]$, downwards

phase	head motion
comb r.blocks.move, blocks	blocks.move: tape 0 up across B_r into the fresh $\mathrm{CM}_r[m_r-1]$, tape 1 up MZ_r; blocks: 2^r blocks of height $h_r = m_r/2^r$, from the top; in each, tape 1 down h_r-1 cells, up h_r-1 cells, then tapes 0 and 1 down h_r cells together
comb r.dbl, shift, rev2	dbl: $\mathrm{CM}_r[z]$ into $\mathrm{U1}_r[2z]$ and $\mathrm{U1}_r[2z+1]$; shift: $\mathrm{CM}_r[(z+\mathrm{sh}_r) \bmod m_r]$ into $\mathrm{U2}_r[2z]$ and $\mathrm{U2}_r[2z+1]$; rev2: $\mathrm{U2}_r$ reversed into $\mathrm{U2R}_r$; tape 2 idle on z_r
comb r.merge.move, merge, out	merge.move, $2m_r+2$ steps: tape 0 down $\mathrm{U2R}_r$, tape 1 down from $\mathrm{U2}_r[0]$ to $\mathrm{U1}_r[0]$, tape 2 up from $Z_r[0]$ to $\mathrm{BS}_r[0]$ (m_r+2 steps); merge: $\mathrm{U2R}_r[u]$ and $\mathrm{U1}_r[u]$ into $\mathrm{BS}_r[u]$; out: $\mathrm{BS}_r[u]$ into $B_{r-1}[u]$, downwards
rewind, emit	rewind: tape 0 idle on $B_0[0]$, tape 1 down from $\mathrm{U1}_1[L-1]$ to cell 0, tape 2 down from $\mathrm{BS}_1[0]$ to cell 0; emit, steps $3u+i$ for $u<L$ and $i=0,1,2$: tape 0 on $B_0[u]$ and tape 1 on $\mathrm{LD}_1[u]$, both moving at $i=2$, tape 2 on cell $\min(3u+i, E_2-1)$, output bit u appended at $i=2$

Split each phase into segments at the steps where a head stops early or where no head moves, and each block into its sweeps; in particular split fwd r.transfer where tape 0 stops while tape 1 still moves, and the emit where tape 2 parks on cell E_2-1. Then in every segment each head is idle or monotone, waiting at most two consecutive steps on a cell, and whenever more than one moving head waits, which happens only in the emit, the moving heads that wait all wait at the same steps.

Lemma D.7 (Segments of Cascade 4)**.** Not formalized; a complete proof is given below.
In a segment $[t_0, t_1)$ of Cascade 4, the number of runs is at most the number of maximal intervals of live tainted cells read by the moving heads, plus the number of idle heads on a live tainted cell that arrived there at most two steps before t_0, plus the number of runs of reads of A; and each run writes at most one interval of cells on each tape whose head moves.

Proof (not formalized). As in Lemma D.3, a tainted step that neither reads a live tainted cell with a moving head nor is one of the first steps of an idle head on a live tainted cell nor reads A is tainted through a short wait from the step before it. An idle head that parked three or more steps before t_0 has wait age at least 3 throughout the segment and passes no taint. The moving heads that wait do so at the same steps, at most two in a row, and then all advance together with the heads that do not wait; so a chain of short waits ends within two steps. The one longer chain, of at most five tainted steps, occurs when an idle head's short waits at t_0+1, t_0+2 hand over to a moving head's waits at t_0+3, t_0+4, at the step where tape 2 parks in the emit; it stays adjacent to its run. So each extension adds no run and only boundedly many written cells. □

The cut is placed at a level ρ with $1 \le \rho \le r_{\max}$: C_0 is the set of tape-1 edges that leave tainted steps of fwd ρ.transfer and whose next visit lies after the transfer; as for Cascade 3, this is well defined.

Lemma D.8 (Forward windows of Cascade 4)**.** Not formalized; a complete proof is given below.
For $1 \le r \le \rho$ let W_r be the set of offsets of X_r that are live tainted when fwd r.transfer starts, and put $S_r = \{\lceil s/2 \rceil : s \in W_r\}$. Then $W_1 = \{\lceil u/2 \rceil + \Delta : u \in A\}$ and $W_{r+1} = S_r \cup (S_r - 3)$ for $r < \rho$. Each W_r is an interval of at least 6 offsets with

$$|W_r| \le \frac{db}{2^{r-1}} + 8, \qquad \frac{m_r}{3} - 6 \le \min W_r \le \max W_r \le \frac{m_r}{3} + \frac{3db}{2^r} + 1 \le 2\lfloor m_r/3 \rfloor - 3,$$

and $S_r \subseteq [7, \mathrm{sh}_r - 3]$. Moreover $|C_0| = |S_\rho| \le db/2^\rho + 5$, and the forward stage up to the end of fwd ρ.transfer has $O(\rho)$ runs.

Proof (not formalized). Load writes the tainted cells $\mathrm{SD}[\lceil u/2 \rceil]$, $u \in A$, since $3db \le \lfloor L/3 \rfloor$, together with the loaded copy $\mathrm{LD}_1[db..3db]$; rev aligns $\mathrm{SD}[x]$ with $X_1[x+\Delta]$, with no waits of the moving heads and tape 1 parked for more than two steps, which gives W_1. The bounds pass from W_r to W_{r+1} as in Lemma D.4. If K is an interval of at least 3 offsets, $(K-3) \cup K$ is the interval $[\min K - 3, \max K]$; since $|S_r| \ge |W_r|/2 \ge 3$, each W_{r+1} is again one interval, of at least 6 offsets, with $|W_{r+1}| \le |W_r|/2 + 4$, $\min W_{r+1} = \min S_r - 3 \ge m_{r+1}/3 - 6$ and $\max W_{r+1} = \max S_r \le m_{r+1}/3 + 3db/2^{r+1} + 1$. As in (1), $\max W_r \le \frac{41}{96} m_r + 1 \le 2\lfloor m_r/3 \rfloor - 3$, and $S_r \subseteq [\frac{m_r}{6} - 3, \frac{41}{192} m_r + 1] \subseteq [7, \mathrm{sh}_r - 3]$, since $m_r \ge 64$.

So fwd r.transfer stays in its half-speed part on W_r and writes the tainted offsets S_r of $\mathrm{H1}_r$; for $r = \rho$ the edges leaving them are C_0. As $W_r \subseteq [0, m_r - 2]$, the second half of X_r is untainted, so shift and rev2 have no tainted step, and $\mathrm{H2}_r$ and $\mathrm{H2R}_r$ are clean. In fwd r.merge.move, tape 1 visits $\mathrm{H1}_r[x]$, $x \in S_r$, at the same step at which tape 0 visits $\mathrm{H2R}_r[x-3]$, with no waits of either: tape 0 parks on $\mathrm{H2R}_r[0]$ only at step $m_r - 1$ of merge.move, when tape 1 is at offset 3 of $\mathrm{H1}_r$, below $\min S_r \ge 7$. Tape 2 has been parked on z_r since before the transfer. So merge taints Z_r exactly at the offsets $S_r \cup (S_r - 3)$, and back copies Z_r onto X_{r+1}, which gives W_{r+1}; move5 crosses this window and leaves BS_r untainted. Each forward level has a bounded number of segments, each reading at most two intervals, which gives the $O(\rho)$ runs. □

Lemma D.9 (Clean middle of Cascade 4)**.** Not formalized; a complete proof is given below.
No step of fwd ρ after its transfer, of fwd r for $r > \rho$, or of comb r for $r \ge \rho$ is tainted.

Proof (not formalized). By Lemma D.8 the second half of X_ρ and $\mathrm{H1}_\rho[m_\rho - 1]$ are untainted, and C_0 cuts every tape-1 edge from a tainted step of fwd ρ.transfer to the later phases. From the end of that transfer to the end of comb ρ.out, tape 0 visits only cells at or beyond $X_\rho[m_\rho - 1]$, tape 1 only cells at or beyond $\mathrm{H1}_\rho$, and tape 2 only cells of $[z_\rho, E_2)$. The live tainted regions of levels at most ρ (on tape 0, LD_0, the first halves of $X_{r'}$ for $r' \le \rho$ and $\mathrm{H2R}_{r'}$ for $r' < \rho$; on tape 1, LD_1, $\mathrm{H1}_{r'}$ for $r' < \rho$ and the cut cells of $\mathrm{H1}_\rho$; on tape 2, SD and $Z_{r'}$ for $r' < \rho$) are not reached by an uncut edge in these phases. □

For $1 \le r \le \rho - 1$ let $U_r \subseteq [0, m_r)$ be the set of offsets at which B_r or Z_r is live tainted when comb r.1a starts, with u_r maximal intervals and $\mu_r = |U_r|$, and let $V_r \subseteq [0, 2m_r)$ be the set of live tainted offsets of BS_r at that moment, with v_r maximal intervals and $\eta_r = |V_r|$. For $r \le \rho - 1 \le k - 2$ the block height is $h_r = m_r/2^r = d^2 b/4^r \ge 16b \ge 16$.

Lemma D.10 (Backward stage of Cascade 4)**.** Not formalized; a complete proof is given below.
Let $\rho \ge 2$. For $1 \le r \le \rho - 1$, $V_r \subseteq [\frac{3}{2} m_r + 2, 2m_r)$; $u_{\rho-1} \le 1$, $\mu_{\rho-1} \le db/2^{\rho-1} + 8$ and $V_{\rho-1} = \varnothing$; and for $1 < r \le \rho - 1$

$$u_{r-1} \le 10u_r + v_r + 19, \qquad v_{r-1} \le 7u_r + v_r + 13,$$
$$\mu_{r-1} \le 17\mu_r + \eta_r + \frac{db}{2^{r-1}} + 28u_r + 30, \qquad \eta_{r-1} \le 12\mu_r + \eta_r + 20u_r + 12.$$

Comb r has $O(u_r + v_r + u_{r+1} + v_{r+1} + \mu_r/h_r + 1)$ runs, with $u_\rho = v_\rho = 0$, and leaves Z_r and BS_r with $O(u_r + v_r + 1)$ tainted intervals and $O(\mu_r + \eta_r + u_r + 1)$ tainted cells.

Proof (not formalized). At $r = \rho - 1$, comb ρ is clean (Lemma D.9), so comb $(\rho-1)$.1a.move crosses only clean cells and has no tainted step; $U_{\rho-1}$ is the window W_ρ that merge wrote on $Z_{\rho-1}$, and $V_{\rho-1} = \varnothing$. Now follow comb r for $r \le \rho - 1$, assuming $V_r \subseteq [\frac{3}{2} m_r + 2, 2m_r)$.

(1) 1a writes a subset of U_r onto the fresh MZ_r. In blocks.move tape 0 crosses B_r and tape 1 crosses MZ_r; a short wait of tape 2 on $Z_r[0]$ adds at most offset 1. Tape 0 then crosses two clean cells into the fresh CM_r while tapes 1 and 2 have wait age at least 3, so CM_r enters blocks clean.

(2) In each block $[B, B+h_r)$ tape 1 turns on B without waiting, and the third sweep ends by stepping onto the next block with wait age 0 on both moving heads, so no taint spills across blocks. If $B+h_r-1 \in U_r$, the two-step short wait of tape 0 at the top of the block taints at most $\mathrm{MZ}_r[B+h_r-2]$ and $\mathrm{MZ}_r[B+h_r-3]$; otherwise tape 0 reaches the third sweep with wait age $2h_r-2>2$. So the tainted cells of CM_r form at most u_r intervals with at most μ_r+2u_r cells, and blocks has $O(\mu_r/h_r+u_r)$ runs.

(3) Dbl writes $\mathrm{U1}_r$ with at most u_r intervals and $2\mu_r+4u_r$ cells; shift and rev2 write $\mathrm{U2}_r$ and $\mathrm{U2R}_r$ with at most u_r+3 intervals and $2\mu_r+4u_r+2$ cells.

(4) In merge.move, tapes 0, 1 and 2 cross $\mathrm{U2R}_r$, $\mathrm{U1}_r$ and Z_r together, and tape 2 then parks on the clean $\mathrm{BS}_r[0]$, as $0 \notin V_r$. Afterwards Z_r has at most $3u_r+4$ tainted intervals and $5\mu_r+8u_r+3$ tainted cells. Merge unites $\mathrm{U2R}_r$, $\mathrm{U1}_r$ and V_r on BS_r, and out copies BS_r onto B_{r-1}: at most $6u_r+v_r+9$ intervals and $10\mu_r+\eta_r+16u_r+8$ cells on each.

(5) For $r>1$, in comb $(r-1)$.1a.move, $5m_r+7$ steps, tape 0 crosses $B_{r-1}[0..2m_r-1]$, tape 1 crosses $\mathrm{U2}_r$ and enters the fresh MZ_{r-1}, and tape 2 crosses two buffer cells, then Z_r, and enters the clean BS_{r-1} at step m_r+5. The buffer cells z_r+m_r and z_r+m_r+1 were also crossed by tape 2 in comb r.merge.move, so they may be tainted; they meet $B_{r-1}[1..2]$ and add at most one interval and two cells, which the constants 19 and 30 of the lemma absorb. From step $2m_r+3$ on, tape 0 has wait age at least 4, and tapes 1 and 2 scan only clean cells. So MZ_{r-1} stays clean, and BS_{r-1} is tainted only during the steps $[m_r+5, 2m_r+2]$, which reach its offsets in $[3m_r+2, 4m_r) = [\frac{3}{2}m_{r-1}+2, 2m_{r-1})$. Counting the intervals and cells read in these steps gives the bounds on v_{r-1} and η_{r-1}; counting those read in the steps $[0, 2m_r-1]$, together with the window W_r of Z_{r-1} (one interval, at most $db/2^{r-1}+8$ cells), gives the bounds on u_{r-1} and μ_{r-1}. □

To solve the recurrences, put $\hat{u}_r = 7u_r+v_r$ and $\hat{\mu}_r = 12\mu_r+\eta_r$. Then $\hat{u}_{\rho-1} \le 7$ and $\hat{u}_{r-1} \le 77u_r+8v_r+146 \le 11\hat{u}_r+146$, so $\hat{u}_r+14.6 \le 21.6\cdot 11^{\rho-1-r}$ by induction, and $\hat{u}_r \le 22\cdot 11^{\rho-1-r}$. Likewise $\hat{\mu}_{\rho-1} \le 12db/2^{\rho-1}+96$ and $\hat{\mu}_{r-1} \le 216\mu_r+13\eta_r+12db/2^{r-1}+356u_r+372 \le 18\hat{\mu}_r+12db/2^{r-1}+51\hat{u}_r+372$, whence

$$\hat{\mu}_r \le c\,(2^{-r}9^{\rho-r}db+18^{\rho-r}), \qquad \sum_{r<\rho}(\mu_r+\eta_r) \le c\,(9^\rho db+18^\rho), \qquad \sum_{r<\rho}\frac{\mu_r}{h_r} \le c\,\Big(\frac{9^\rho}{d}+\frac{18^\rho}{d^2b}\Big),$$

and the backward stage has $O(11^\rho+9^\rho/d+18^\rho/L)$ runs.

Rewind and emit. In the rewind, tape 0 stays on $B_0[0]$ while tapes 1 and 2 move down to cell 0, in two segments with $O(11^\rho+\rho)$ runs. Tape 1 crosses $\mathrm{U1}_1$ first; when it later reaches LD_1, tapes 0 and 2 have wait age far above 2, so the taint of LD_1 stays in $[db, 3db]$. In the emit, the step that appends output u is tainted only if $B_0[u]$ or $\mathrm{LD}_1[u]$ is live tainted, if one of the tape-2 cells $3u, 3u+1, 3u+2$ below E_2 is live tainted, or if $u = \lceil E_2/3 \rceil$ and $B_0[u-1]$ is live tainted (the park of tape 2 can extend the tainted interval of $u-1$ by one output). Within $[db, 3db]$ the outputs of $u_0(A)$ are the $4b$ bits of the entries $(1,1)$, $(1,2)$, $(2,1)$ and $(2,2)$. Elsewhere, B_0 and the tape-2 frames Z_r, BS_r of levels $r<\rho$ give $O(11^\rho)$ tainted intervals of outputs, of total length $O(9^\rho db+18^\rho)$, each block of db output positions contains at most $2b$ outputs of $u_0(A)$, and $(18/11)^\rho < 2^\rho \le db$. So

$$|C_0| \le \frac{db}{2^\rho}+5, \qquad N_{\mathrm{run}} \le c''\Big(11^\rho+\frac{9^\rho}{d}+\frac{18^\rho}{L}+\rho\Big), \qquad N_{\mathrm{up}} \le 4b+c'\,11^\rho b. \tag{3}$$

D.4 Proof of Theorem 10.8 and Remark 10.9

Let $b = b(d) = k$, let ν be as in Theorem 10.8, and choose ρ with $\nu \le 2^\rho < 2\nu$; for $d \ge d_0$ this gives $1 \le \rho \le r_{\max}$ and $(5/3)^\rho \le db$. For Cascade 3, $6^\rho < 6\nu^{\log_2 6} \le 6(db)^{0.52}$, $5^\rho/d \le 5(db)^{0.47}/d$ and $40^\rho/L \le 40(db)^{1.07}/(d^2 b)$, and the last two tend to 0. For Cascade 4, $11^\rho < 11\nu^{\log_2 11} \le 11(db)^{0.58}$, $9^\rho/d \le 9(db)^{0.53}/d$ and $18^\rho/L \le 18(db)^{0.7}/(d^2 b)$, and again the last two tend to 0. Since $b = \log_2 d$, (2) and (3) give $|C_0|$, N_{run}, $N_{\mathrm{up}} \le c_1\, db/\nu$ for $d \ge d_0$, with an absolute constant c_1.

(a) By Lemma D.2(b), Γ separates $u_0(Q^*) = u_0(A)$ from $\mathsf{In}(\{1,2\})$, and $|\Gamma| \le 3c_1\, db/\nu$. Lemma 7.1 for the oblivious machine ($\eta = 0$) gives $(w_{\mathrm{val}} + 1)|\Gamma| \ge w_{\mathrm{val}}|\Gamma| + |\Gamma \cap u_0(Q^*)| \ge 2db$, so $w_{\mathrm{val}} + 1 \ge 2\nu/(3c_1)$.

(b) Let V be the set of $v \in A$ whose root $u_0(v)$ is untainted, so $|V| \ge 2db - N_{\mathrm{up}} \ge db$ for $d \ge d_0$. By Lemma D.2(a), every path from a read of A to a step of $u_0(V)$ uses an edge of C, which has at most $|C_0| + 3N_{\mathrm{run}}$ cell edges and at most N_{run} state edges. Lemma 10.7 gives $db \le 4c_1(db/\nu)\big(\log_2(g+1) + \log_2(q+1)\big)$, so $\log_2(g+1) + \log_2(q+1) \ge \nu/(4c_1)$.

For Remark 10.9, let $b \ge 1$ be arbitrary and choose $d^{1/5} \le 2^\rho < 2d^{1/5}$ (Cascade 3) or $d^{1/6} \le 2^\rho < 2d^{1/6}$ (Cascade 4). The same estimates give $N_{\mathrm{run}} = O(d^{0.52})$, respectively $O(d^{0.58})$, and $N_{\mathrm{up}} \le 4b + O(d^{0.52}b)$, respectively $4b + O(d^{0.58}b)$, uniformly in b, while $|C_0| \le d^{4/5}b + 2$, respectively $d^{5/6}b + 5$. So $|V| \ge db$ for $d \ge d_0$, and the proof of Lemma 10.7, which uses nothing about the entry width, excludes the trace once $\log_2(g+1) + \log_2(q+1)$ is below $d^{1/5}/c$, respectively $d^{1/6}/c$.

References

[1] Micah Adler, Nicholas J. A. Harvey, Kamal Jain, Robert D. Kleinberg, and April Rasala Lehman. On the capacity of information networks. In *Proceedings of the Seventeenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2006)*, pages 241–250, 2006.

[2] Alok Aggarwal and Jeffrey Scott Vitter. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988. doi:10.1145/48529.48535.

[3] Ziv Bar-Yossef, Yitzhak Birk, T. S. Jayram, and Tomer Kol. Index coding with side information. *IEEE Transactions on Information Theory*, 57(3):1479–1494, 2011.

[4] Mark Braverman and Zhongtian He. The multiple unicast conjecture is false. arXiv:2610.10108, 2026. URL: https://arxiv.org/abs/2610.10108.

[5] Hans Brende and Claude Opus 5.5 (Anthropic). Bounded nesting and depth reduction for matrix transposition on tapes. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision d5576b2391fb81e782540339776318a2abd065a2. URL: https://github.com/HansBrende/papers/tree/d5576b2391fb81e782540339776318a2abd065a2/Bounded-Nesting-and-Depth-Reduction-for-Matrix-Transposition-on-Tapes.

[6] Hans Brende and Claude Opus 5.5 (Anthropic). Bounded-stack DAGs reduce to constant depth. Manuscript and Lean formalization, repository HansBrende/papers, November 2026. Pinned revision d5576b2391fb81e782540339776318a2abd065a2. URL: https://github.com/HansBrende/papers/tree/d5576b2391fb81e782540339776318a2abd065a2/Bounded-Stack-DAGs-Reduce-to-Constant-Depth.

[7] Hans Brende and Claude Opus 5.5 (Anthropic). Curved motion, depth reduction, and storage codes for matrix transposition on tapes. Manuscript and partial Lean formalization, repository HansBrende/papers, October 2026. Pinned revision

d5576b2391fb81e782540339776318a2abd065a2. URL: https://github.com/HansBrende/papers/tree/d5576b2391fb81e782540339776318a2abd065a2/Curved-Motion-Depth-Reduction-and-Storage-Codes-for-Matrix-Transposition-on-Tapes.

[8] Hans Brende and Claude Opus 5.5 (Anthropic). Frozen gap areas and the sharpness of source-cover certificates. Memo to the ninth paper of the series, with Lean formalization, repository HansBrende/papers, October 2026. Pinned revision d5576b2391fb81e782540339776318a2abd065a2. URL: https://github.com/HansBrende/papers/tree/d5576b2391fb81e782540339776318a2abd065a2/Source-Covers-and-Barriers-to-Tape-Transposition/memo-gap-area.

[9] Hans Brende and Claude Opus 5.5 (Anthropic). Matrix transposition and the tape coding conjecture. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision d5576b2391fb81e782540339776318a2abd065a2. URL: https://github.com/HansBrende/papers/tree/d5576b2391fb81e782540339776318a2abd065a2/Matrix-Transposition-and-the-Tape-Coding-Conjecture.

[10] Hans Brende and Claude Opus 5.5 (Anthropic). Merge depth, input steering, and the coding core of matrix transposition on tapes. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision d5576b2391fb81e782540339776318a2abd065a2. URL: https://github.com/HansBrende/papers/tree/d5576b2391fb81e782540339776318a2abd065a2/Merge-Depth-Input-Steering-and-the-Coding-Core-of-Matrix-Transposition-on-Tapes.

[11] Hans Brende and Claude Opus 5.5 (Anthropic). Storage codes above half rate under paired-distance separation. Manuscript and Lean formalization, repository HansBrende/papers, November 2026. Pinned revision d5576b2391fb81e782540339776318a2abd065a2. URL: https://github.com/HansBrende/papers/tree/d5576b2391fb81e782540339776318a2abd065a2/Storage-Codes-above-Half-Rate-under-Paired-Distance-Separation.

[12] Hans Brende and Gemini 4 Argon (Google DeepMind). Directed reciprocity and deep-nesting bounds for matrix transposition on tapes. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision d5576b2391fb81e782540339776318a2abd065a2. URL: https://github.com/HansBrende/papers/tree/d5576b2391fb81e782540339776318a2abd065a2/Directed-Reciprocity-and-Deep-Nesting-Bounds-for-Matrix-Transposition-on-Tapes.

[13] Hans Brende and GPT-6 Astra (OpenAI). Adaptive pass bounds without a runtime assumption. Memo to the first paper of the series, with Lean formalization, repository HansBrende/papers, October 2026. Pinned revision d5576b2391fb81e782540339776318a2abd065a2. URL: https://github.com/HansBrende/papers/tree/d5576b2391fb81e782540339776318a2abd065a2/Matrix-Transposition-and-the-Tape-Coding-Conjecture/memo-passes.

[14] Hans Brende and GPT-6 Astra (OpenAI). High-rate separated storage codes and tape transposition. Manuscript and Lean formalization, repository HansBrende/papers, November 2026. Pinned revision d5576b2391fb81e782540339776318a2abd065a2. URL: https://github.com/HansBrende/papers/tree/d5576b2391fb81e782540339776318a2abd065a2/High-Rate-Separated-Storage-Codes-and-Tape-Transposition.

[15] Hans Brende and GPT-6 Astra (OpenAI). Local coding barriers and the limits of tape geometry for matrix transposition. Manuscript and Lean for-

malization, repository HansBrende/papers, October 2026. Pinned revision d5576b2391fb81e782540339776318a2abd065a2. URL: https://github.com/HansBrende/papers/tree/d5576b2391fb81e782540339776318a2abd065a2/Local-Coding-Barriers-and-the-Limits-of-Tape-Geometry-for-Matrix-Transposition.

[16] Hans Brende and GPT-6 Astra (OpenAI). Regular-recursion obstructions for the five cascade schedules. Memo to the sixteenth paper of the series, with partial Lean formalization, repository HansBrende/papers, November 2026. Pinned revision dc11ddcd22b308c54e6cad0a86976a9bd321064c. URL: https://github.com/HansBrende/papers/tree/dc11ddcd22b308c54e6cad0a86976a9bd321064c/Oblivious-Linear-Time-Tape-Transposers-Need-Macroscopic-Crossings/memo-cascade-obstruction.

[17] Hans Brende and GPT-6 Astra (OpenAI). Source covers and barriers to tape transposition. Manuscript and Lean formalization, repository HansBrende/papers, October 2026. Pinned revision d5576b2391fb81e782540339776318a2abd065a2. URL: https://github.com/HansBrende/papers/tree/d5576b2391fb81e782540339776318a2abd065a2/Source-Covers-and-Barriers-to-Tape-Transposition.

[18] Martin Dietzfelbinger and Wolfgang Maass. The complexity of matrix transposition on one-tape off-line Turing machines with output tape. *Theoretical Computer Science*, 108(2):271–290, 1993.

[19] Martin Dietzfelbinger, Wolfgang Maass, and Georg Schnitger. The complexity of matrix transposition on one-tape off-line Turing machines. *Theoretical Computer Science*, 82(1):113–129, 1991.

[20] Étienne Grandjean. A natural NP-complete problem with a nontrivial lower bound. *SIAM Journal on Computing*, 17(4):786–809, 1988. doi:10.1137/0217050.

[21] Martin Grohe, André Hernich, and Nicole Schweikardt. Lower bounds for processing data with few random accesses to external memory. *Journal of the ACM*, 56(3), 2009. Article 12.

[22] Martin Grohe and Nicole Schweikardt. Lower bounds for sorting with few random accesses to external memory. In *Proceedings of the Twenty-fourth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS 2005)*. ACM, 2005. doi:10.1145/1065167.1065197.

[23] David Harvey and Joris van der Hoeven. Integer multiplication is at least as hard as matrix transposition. arXiv:2503.22848; also in Proc. FOCS 2025, pp. 1366–1381 (numbering as in arXiv), 2025. URL: https://arxiv.org/abs/2503.22848.

[24] F. C. Hennie. One-tape, off-line Turing machine computations. *Information and Control*, 8(6):553–578, 1965.

[25] John Hopcroft, Wolfgang Paul, and Leslie Valiant. On time versus space. *Journal of the ACM*, 24(2):332–337, 1977. doi:10.1145/322003.322015.

[26] Walter W. Kirchherr. Transposition of an $l \times l$ matrix requires $\Omega(\log l)$ reversals on conservative Turing machines. *Information Processing Letters*, 28(2):55–59, 1988. doi:10.1016/0020-0190(88)90163-9.

[27] Wolfgang Maass. Combinatorial lower bound arguments for deterministic and nondeterministic Turing machines. *Transactions of the American Mathematical Society*, 292(2):675–693, 1985.

[28] OpenAI. Integer multiplication below $n \log n$. GitHub release, repository `openai/math`, family 109, September 2026. Preprint dated 23 September 2026. URL: https://github.com/openai/math/blob/main/preprints/Integer-multiplication-below-n-log-n-September-23-2026/paper.pdf.

[29] Wolfgang J. Paul, Nicholas Pippenger, Endre Szemerédi, and William T. Trotter. On determinism versus non-determinism and related problems. In *24th Annual Symposium on Foundations of Computer Science (FOCS 1983)*, pages 429–438, 1983.

[30] Nicholas Pippenger and Michael J. Fischer. Relations among complexity measures. *Journal of the ACM*, 26(2):361–381, 1979.

[31] Hans-Jörg Stoß. Rangierkomplexität von Permutationen. *Acta Informatica*, 2:80–96, 1973.

[32] Xindan Zhang, Baochun Li, and Zongpeng Li. Network coding can beat routing in undirected multiple-unicast networks. arXiv:2610.09367, 2026. URL: https://arxiv.org/abs/2610.09367.